Programming Languages and Compilers (CS 421)

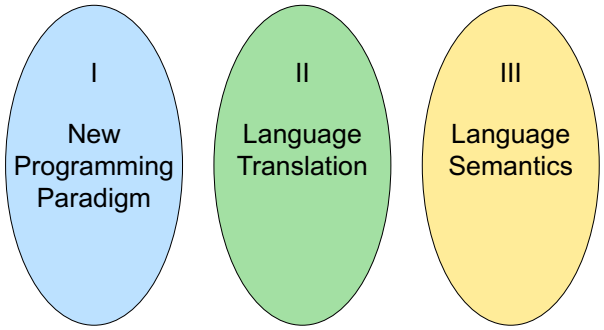
Elsa L Gunter
2112 SC, UIUC
https://courses.engr.illinois.edu/cs421/sp2024



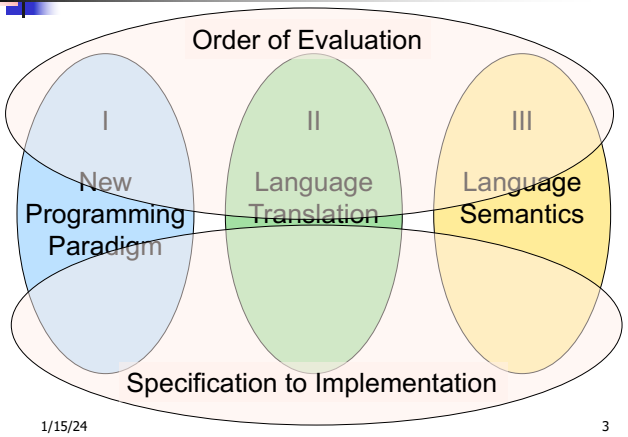
Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Programming Languages & Compilers

Three Main Topics of the Course

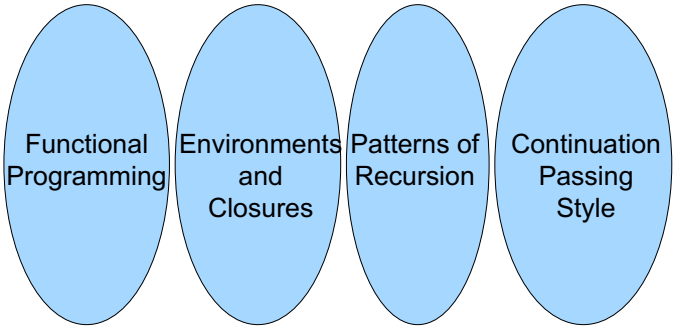


Programming Languages & Compilers

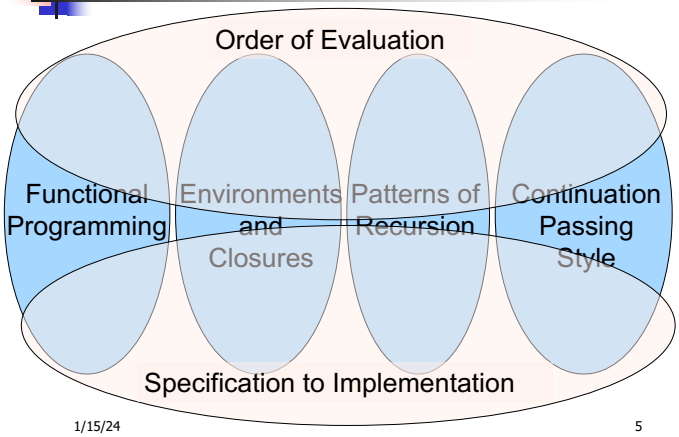


Programming Languages & Compilers

I : New Programming Paradigm

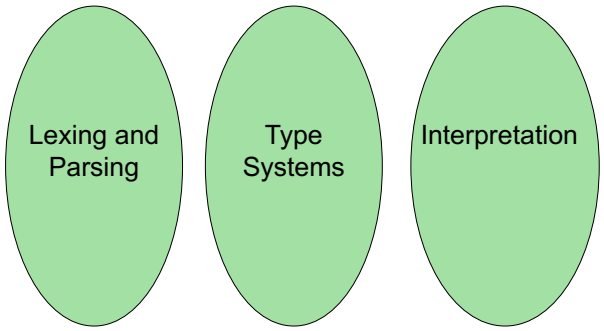


Programming Languages & Compilers

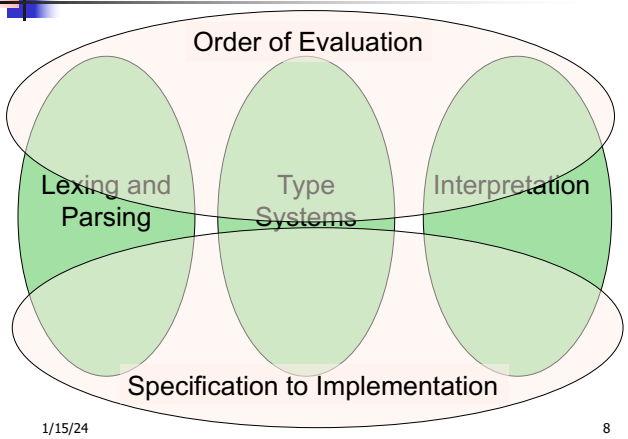


Programming Languages & Compilers

II : Language Translation

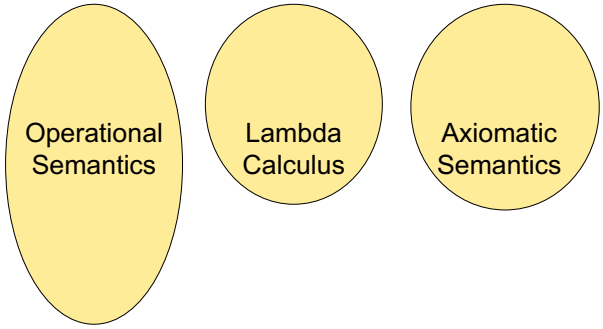


Programming Languages & Compilers

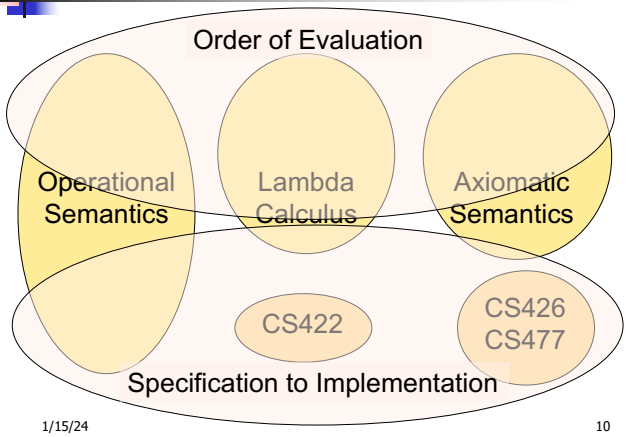


Programming Languages & Compilers

III : Language Semantics



Programming Languages & Compilers



Contact Information - Elsa L Gunter

- Office: 2112 SC
- Office hours:
  - TBD
    - Can attend in zoom
  - Also by appointment
- Email: egunter@illinois.edu

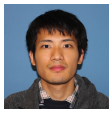
Course TAs


Aruhan


Purvansh Bal


Deeya Bansal


Athena Fung


Yerong Li


James Luo


Siheng Pan


Riya Patel


Mike Qin


Havish Rani

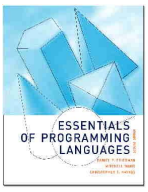

Alan Yao

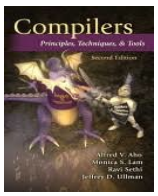
Course Website

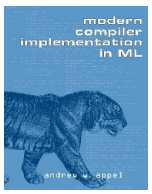
- https://courses.engr.illinois.edu/cs421/sp2024
- Main page - summary of news items
- Policy - rules governing course
- Lectures - syllabus and slides
- MPs - information about assignments
- Exams – Syllabi and review material for Midterms and finals
- Unit Projects - for 4 credit students
- Resources - tools and helpful info
- FAQ

Some Course References

- No required textbook
- Some suggested references







Some Course References

- No required textbook.
- Pictures of the books on previous slide
- Essentials of Programming Languages (2nd Edition) by Daniel P. Friedman, Mitchell Wand and Christopher T. Haynes, MIT Press 2001.
- Compilers: Principles, Techniques, and Tools, (also known as "The Dragon Book"); by Aho, Sethi, and Ullman. Published by Addison-Wesley. ISBN: 0-201-10088-6.
- Modern Compiler Implementation in ML by Andrew W. Appel, Cambridge University Press 1998
- Additional ones for Ocaml given separately

Course Grading

- Assignments 10%
  - Web Assignments (WA) (~3-6%)
  - MPs (in Ocaml) (~4-7%)
  - All WAs and MPs Submitted by **PrairieLearn**
  - Late submission penalty:
    score capped at 80% of total

Course Grading

- Five quizzes - 10% (2% each)
  - In class, BYOD
  - Tentatively Jan 23, Feb 6, Feb 27, Mar 26, Apr 23
- 3 Midterms - 15% each
  - **Feb 12-14, Mar 6-8, Apr 11-13**
  - **BE AVAILABLE FOR THESE DATES!**
- Final 35%
- Tuesday May 7, 7:00pm – 10:00pm
- Percentages based on 3 cr, are approximate

Course Assingments – WA & MP

- You may discuss assignments and their solutions with others
- You may work in groups, but you must **list members with whom you worked** if you share solutions or detailed solution outlines
- **Each student must write up and turn in their own solution separately**
- You may look at examples from class and other similar examples from any source – **cite appropriately**
  - Note: University policy on plagiarism still holds - cite your sources if you are not the sole author of your solution
  - Do not have to cite course notes or course staff

OCAML

- Locally:
  - Will use ocaml inside VSCode inside PrairieLearn problems this semester
- Globally:
  - Main OCAML home: http://ocaml.org
  - To install OCAML on your computer see: http://ocaml.org/docs/install.html
  - To try on the web: https://try.ocamlpro.com
  - More notes on this later

References for OCaml

- Supplemental texts (not required):
- The Objective Caml system release 4.05, by Xavier Leroy, online manual
- Introduction to the Objective Caml Programming Language, by Jason Hickey
- Developing Applications With Objective Caml, by Emmanuel Chailloux, Pascal Manoury, and Bruno Pagano, on O' Reilly
  - Available online from course resources

Features of OCAML

- Higher order applicative language
- Call-by-value parameter passing
- Modern syntax
- Parametric polymorphism
  - Aka structural polymorphism
- Automatic garbage collection
- User-defined algebraic data types

Session in OCAML

```
% ocaml
Objective Caml version 4.07.1
# (* Read-eval-print loop; expressions and
  declarations *)
  2 + 3;;     (* Expression *)
- : int = 5
# 3 < 2;;
- : bool = false
```


Declarations; Sequencing of Declarations

```
# let x = 2 + 3;;   (* declaration *)
val x : int = 5
# let test = 3 < 2;;
val test : bool = false
# let a = 1 let b = a + 4;; (* Sequence of dec
  *)
val a : int = 1
val b : int = 5
```

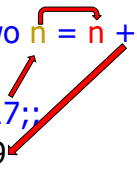

Functions

```
# let plus_two n = n + 2;;
val plus_two : int -> int = <fun>
# plus_two 17;;
- : int = 19
```


Functions

```
let plus_two n = n + 2;;

plus_two 17;;
- : int = 19
```



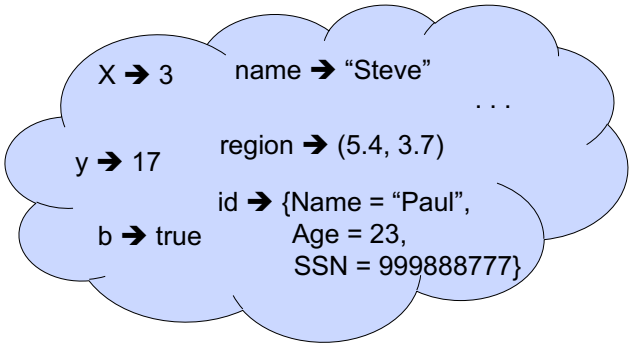
Environments

- *Environments* record what value is associated with a given identifier
- Central to the semantics and implementation of a language
- Notation

  $\rho = \{name_1 \rightarrow value_1, name_2 \rightarrow value_2, \ldots\}$

  Using set notation, but describes a partial function
- Often stored as list, or stack
  - To find value start from left and take first match

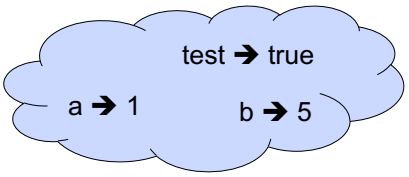
Environments



Global Variable Creation

```
# 2 + 3;;     (* Expression *)
// doesn't affect the environment
# let test = 3 < 2;;       (* Declaration *)
val test : bool = false
//  ρ1 = {test → false}
# let a = 1 let b = a + 4;; (* Seq of dec *)
//  ρ2 = {b → 5, a → 1, test → false}
```


Environments



New Bindings Hide Old

```
//  ρ2 = {b → 5, a → 1, test → false}
let test = 3.7;;
```

- What is the environment after this declaration?

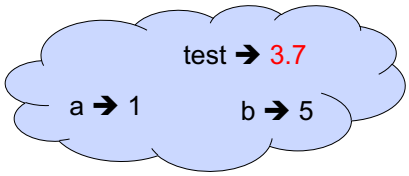
New Bindings Hide Old

```
//  ρ2 = {b → 5, a → 1, test → false}
let test = 3.7;;
```

- What is the environment after this declaration?

```
//  ρ3 = {test → 3.7, a → 1, b → 5}
```

Environments

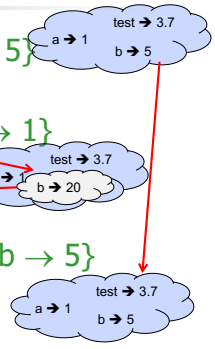


Now it's your turn

You should be able to do WA1-IC Problem 1 , parts (* 1 *) - (* 3 *)

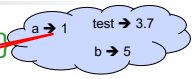
Local Variable Creation

// ρ_3 = {test → 3.7, a → 1, b → 5}
let b = 5 * 4
// ρ_4 = {b → 20, test → 3.7, a → 1}
  in 2 * b;;
- : int = 40
// ρ_5 = ρ_3= {test → 3.7, a → 1, b → 5}
b;;
- : int = 5



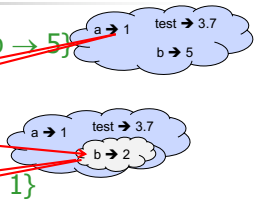
Local let binding

// ρ_5 =ρ_3 ={test → 3.7, a → 1, b → 5}
let c =
  let b = a + a
// ρ_6 = {b → 2} + ρ_3
// ={b → 2, test → 3.7, a → 1}
  in b * b;;
val c : int = 4
// ρ_7 = {c → 4, test → 3.7, a → 1, b → 5}
b;;
- : int = 5



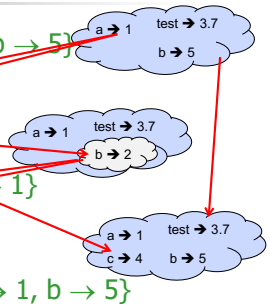
Local let binding

// ρ_5 =ρ_3 ={test → 3.7, a → 1, b → 5}
let c =
  let b = a + a
// ρ_6 = {b → 2} + ρ_3
// ={b → 2, test → 3.7, a → 1}
  in b * b;;
val c : int = 4
// ρ_7 = {c → 4, test → 3.7, a → 1, b → 5}
b;;
- : int = 5



Local let binding

// ρ_5 =ρ_3 ={test → 3.7, a → 1, b → 5}
let c =
  let b = a + a
// ρ_6 = {b → 2} + ρ_3
// ={b → 2, test → 3.7, a → 1}
  in b * b;;
val c : int = 4
// ρ_7 = {c → 4, test → 3.7, a → 1, b → 5}
b;;
- : int = 5



Functions

```
# let plus_two n = n + 2;;
val plus_two : int -> int = <fun>
# plus_two 17;;
- : int = 19
```


Functions

```
let plus_two n = n + 2;;

plus_two 17;;
- : int = 19
```


Nameless Functions (aka Lambda Terms)

```
fun n -> n + 2;;

(fun n -> n + 2) 17;;
- : int = 19
```



Functions

```
# let plus_two n = n + 2;;
val plus_two : int -> int = <fun>
# plus_two 17;;
- : int = 19
# let plus_two = fun n -> n + 2;;
val plus_two : int -> int = <fun>
# plus_two 14;;
- : int = 16
```

First definition syntactic sugar for second

Using a nameless function

```
# (fun x -> x * 3) 5;;   (* An application *)
- : int = 15
# ((fun y -> y +. 2.0), (fun z -> z * 3));;
  (* As data *)
- : (float -> float) * (int -> int) = (<fun>,
  <fun>)
```

Note: in fun v -> exp(v), scope of variable is only the body exp(v)

Values fixed at declaration time

```
# let x = 12;;
val x : int = 12
# let plus_x y = y + x;;
val plus_x : int -> int = <fun>
# plus_x 3;;
```

X ➔ 12
…

What is the result?

Values fixed at declaration time

```
# let x = 12;;
val x : int = 12
# let plus_x y = y + x;;
val plus_x : int -> int = <fun>
# plus_x 3;;
- : int = 15
```


Values fixed at declaration time

```
# let x = 7;;   (* New declaration, not an
  update *)
val x : int = 7

# plus_x 3;;
```

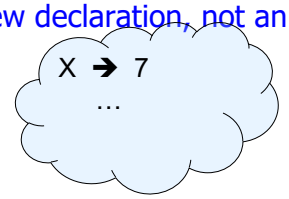
What is the result this time?

Values fixed at declaration time

```
# let x = 7;;   (* New declaration, not an
  update *)
val x : int = 7
```




```
# plus_x 3;;
```



What is the result this time?

Values fixed at declaration time

```
# let x = 7;;   (* New declaration, not an
  update *)
val x : int = 7

# plus_x 3;;
- : int = 15
```


Question

- Observation: Functions are first-class values in this language
- Question: What value does the environment record for a function variable?
- Answer: a closure

Save the Environment!

- A *closure* is a pair of an environment and an association of a formal parameter (the input variables)* with an expression (the function body), written:

  $f \rightarrow < (v1,\ldots,vn) \rightarrow exp, \rho_f >$

- Where ρ_f is the environment in effect when f is defined (if f is a simple function)

* Will come back to the "formal parameter"

Closure for plus_x

- When plus_x was defined, had environment:

$$\rho_{plus_x} = \{\ldots, x \rightarrow 12, \ldots\}$$

- Recall: let plus_x y = y + x

  is really let plus_x = fun y -> y + x
- Closure for fun y -> y + x:

$$<y \rightarrow y + x, \rho_{plus_x}>$$

- Environment just after plus_x defined:

$$\{plus_x \rightarrow <y \rightarrow y + x, \rho_{plus_x}>\} + \rho_{plus_x}$$

Now it's your turn

You should be able complete ACT1

Functions with more than one argument

```
# let add_three x y z = x + y + z;;
val add_three : int -> int -> int -> int = <fun>
# let t = add_three 6 3 2;;
val t : int = 11
# let add_three =
   fun x -> (fun y -> (fun z -> x + y + z));;
val add_three : int -> int -> int -> int = <fun>
```

Again, first syntactic sugar for second

Functions with more than one argument

```
# let add_three x y z = x + y + z;;
val add_three : int -> int -> int -> int = <fun>
```

- What is the value of add_three?
- Let ρ_{add_three} be the environment before the declaration
- Remember:

```
let add_three =
 fun x -> (fun y -> (fun z -> x + y + z));;
```

Value: $<x \rightarrow$fun y -> (fun z -> x + y + z), $\rho_{add_three}>$

Partial application of functions

```
let add_three x y z = x + y + z;;

# let h = add_three 5 4;;
val h : int -> int = <fun>
# h 3;;
- : int = 12
# h 7;;
- : int = 16
```


Partial application of functions

```
let add_three x y z = x + y + z;;

# let h = add_three 5 4;;
val h : int -> int = <fun>
# h 3;;
- : int = 12
# h 7;;
- : int = 16
```

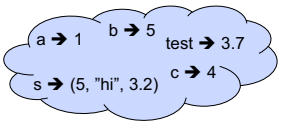
- Partial application also called *sectioning*

Functions as arguments

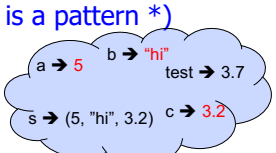
```
# let thrice f x = f (f (f x));;
val thrice : ('a -> 'a) -> 'a -> 'a = <fun>
# let g = thrice plus_two;;
val g : int -> int = <fun>
# g 4;;
- : int = 10
# thrice (fun s -> "Hi! " ^ s) "Good-bye!";;
- : string = "Hi! Hi! Hi! Good-bye!"
```

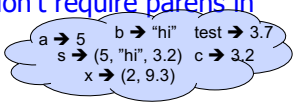

Pattern Matching with Tuples

// ρ_8 = {s → (5, "hi", 3.2),
c → 4, test → 3.7,
a → 1, b → 5}




```
# let (a,b,c) = s;;  (* (a,b,c) is a pattern *)
val a : int = 5
val b : string = "hi"
val c : float = 3.2
# let x = 2, 9.3;; (* tuples don't require parens in
  Ocaml *)
val x : int * float = (2, 9.3)
```



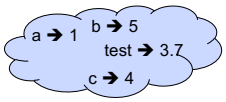


Functions on tuples

```
# let plus_pair (n,m) = n + m;;
val plus_pair : int * int -> int = <fun>
# plus_pair (3,4);;
- : int = 7
# let double x = (x,x);;
val double : 'a -> 'a * 'a = <fun>
# double 3;;
- : int * int = (3, 3)
# double "hi";;
- : string * string = ("hi", "hi")
```

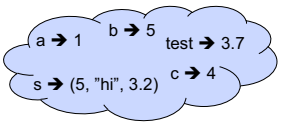

Tuples as Values

// ρ_7 = {c → 4, test → 3.7,
a → 1, b → 5}




```
# let s = (5,"hi",3.2);;
val s : int * string * float = (5, "hi", 3.2)
```

// ρ_8 = {s → (5, "hi", 3.2),
c → 4, test → 3.7,
a → 1, b → 5}



Nested Tuples

```
#  (*Tuples can be nested *)
let d = ((1,4,62),("bye",15),73.95);;
val d : (int * int * int) * (string * int) * float =
  ((1, 4, 62), ("bye", 15), 73.95)
#  (*Patterns can be nested *)
let (p,(st,_),_) = d;; (* _ matches all, binds nothing
   *)
val p : int * int * int = (1, 4, 62)
val st : string = "bye"
```


Match Expressions

```
# let triple_to_pair triple =
  match triple
  with (0, x, y) -> (x, y)
  | (x, 0, y) -> (x, y)
  | (x, y, _) -> (x, y);;
val triple_to_pair : int * int * int -> int * int =
  <fun>
```

- Each clause: pattern on left, expression on right
- Each x, y has scope of only its clause
- Use first matching clause

Closure for plus_pair

- Assume ρ_{plus_pair} was the environment just before plus_pair defined
- Closure for plus_pair:

$$<(n,m) \rightarrow n + m, \rho_{plus_pair}>$$

- Environment just after plus_pair defined:

$$\{plus_pair \rightarrow <(n,m) \rightarrow n + m, \rho_{plus_pair} >\} + \rho_{plus_pair}$$

Save the Environment!

- A *closure* is a pair of an environment and an association of a pattern (e.g. (v1,…,vn) giving the input variables) with an expression (the function body), written:

$$< (v1,\ldots,vn) \rightarrow exp, \rho >$$

- Where ρ is the environment in effect when the function is defined (for a simple function)

Evaluating declarations

- Evaluation uses an environment ρ
- To evaluate a (simple) declaration let x = e
  - Evaluate expression e in ρ to value v
  - Update ρ with x v: $\{x \rightarrow v\} + \rho$
- Update: $\rho_1 + \rho_2$ has all the bindings in ρ_1 and all those in ρ_2 that are not rebound in ρ_1

$\{x \rightarrow 2, y \rightarrow 3, a \rightarrow$ "hi"$\} + \{y \rightarrow 100, b \rightarrow 6\}$
$= \{x \rightarrow 2, y \rightarrow 3, a \rightarrow$ "hi", $b \rightarrow 6\}$

Evaluating expressions in OCaml

- Evaluation uses an environment ρ
- A constant evaluates to itself, including primitive operators like + and =
- To evaluate a variable, look it up in ρ: $\rho(v)$
- To evaluate a tuple $(e_1,\ldots,e_n)$,
  - Evaluate each e_i to v_i, right to left for Ocaml
  - Then make value $(v_1,\ldots,v_n)$

Evaluating expressions in OCaml

- To evaluate uses of +, _ , etc, eval args, then do operation
- Function expression evaluates to its closure
- To evaluate a local dec: let x = e1 in e2
  - Eval e1 to v, then eval e2 using $\{x \rightarrow v\} + \rho$
- To evaluate a conditional expression: if b then e1 else e2
  - Evaluate b to a value v
  - If v is True, evaluate e1
  - If v is False, evaluate e2

Evaluation of Application with Closures

- Given application expression f e
- In Ocaml, evaluate e to value v
- In environment ρ, evaluate left term to closure, $c = <(x_1,\ldots,x_n) \rightarrow b, \rho'>$
  - $(x_1,\ldots,x_n)$ variables in (first) argument
  - v must have form $(v_1,\ldots,v_n)$
- Update the environment ρ' to

$\rho'' = \{x_1 \rightarrow v_1,\ldots, x_n \rightarrow v_n\} + \rho'$

- Evaluate body b in environment ρ''

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC



https://courses.engr.illinois.edu/cs421/fa2023/CS421D

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Functions with more than one argument

```
# let add_three x y z = x + y + z;;
val add_three : int -> int -> int -> int = <fun>
# let t = add_three 6 3 2;;
val t : int = 11
# let add_three =
   fun x -> (fun y -> (fun z -> x + y + z));;
val add_three : int -> int -> int -> int = <fun>
```

Again, first syntactic sugar for second

Functions with more than one argument

```
# let add_three x y z = x + y + z;;
val add_three : int -> int -> int -> int = <fun>
```

- What is the value of add_three?
- Let ρ_{add_three} be the environment before the declaration
- Remember:

```
let add_three =
 fun x -> (fun y -> (fun z -> x + y + z));;
```

Value: <x ->fun y -> (fun z -> x + y + z), ρ_{add_three} >

Partial application of functions

```
let add_three x y z = x + y + z;;
```

```
# let h = add_three 5 4;;
val h : int -> int = <fun>
# h 3;;
- : int = 12
# h 7;;
- : int = 16
```


Partial application of functions

```
let add_three x y z = x + y + z;;
```

```
# let h = add_three 5 4;;
val h : int -> int = <fun>
# h 3;;
- : int = 12
# h 7;;
- : int = 16
```

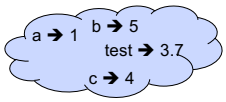
- Partial application also called *sectioning*

Functions as arguments

```
# let thrice f x = f (f (f x));;
val thrice : ('a -> 'a) -> ('a -> 'a) = <fun>
# let g = thrice plus_two;;
val g : int -> int = <fun>
# g 4;;
- : int = 10
# thrice (fun s -> "Hi! " ^ s) "Good-bye!";;
- : string = "Hi! Hi! Hi! Good-bye!"
```

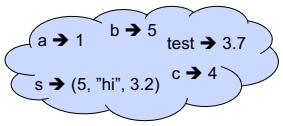
Tuples as Values

// ρ_7 = {c → 4, test → 3.7,
a → 1, b → 5}



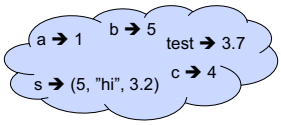

```
# let s = (5,"hi",3.2);;
val s : int * string * float = (5, "hi", 3.2)
```

// ρ_8 = {s → (5, "hi", 3.2),
c → 4, test → 3.7,
a → 1, b → 5}

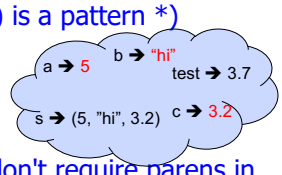


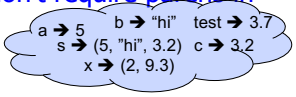
Pattern Matching with Tuples

/ ρ_8 = {s → (5, "hi", 3.2),
c → 4, test → 3.7,
a → 1, b → 5}




```
# let (a,b,c) = s;;  (* (a,b,c) is a pattern *)
val a : int = 5
val b : string = "hi"
val c : float = 3.2
# let x = 2, 9.3;; (* tuples don't require parens in
  Ocaml *)
val x : int * float = (2, 9.3)
```





Nested Tuples

```
#  (*Tuples can be nested *)
let d = ((1,4,62),("bye",15),73.95);;
val d : (int * int * int) * (string * int) * float =
  ((1, 4, 62), ("bye", 15), 73.95)
#  (*Patterns can be nested *)
let (p,(st,_),_) = d;; (* _ matches all, binds nothing
   *)
val p : int * int * int = (1, 4, 62)
val st : string = "bye"
```


Functions on tuples

```
# let plus_pair (n,m) = n + m;;
val plus_pair : int * int -> int = <fun>
# plus_pair (3,4);;
- : int = 7
# let double x = (x,x);;
val double : 'a -> 'a * 'a = <fun>
# double 3;;
- : int * int = (3, 3)
# double "hi";;
- : string * string = ("hi", "hi")
```


Curried vs Uncurried

- Recall

```
val add_three : int -> int -> int -> int = <fun>
```

- How does it differ from

```
# let add_triple (u,v,w) = u + v + w;;
val add_triple : int * int * int -> int = <fun>
```

- add_three is *curried*;
- add_triple is *uncurried*

Curried vs Uncurried

```
# add_triple (6,3,2);;
- : int = 11
# add_triple 5 4;;
Characters 0-10:
  add_triple 5 4;;
  ^^^^^^^^^^
```

This function is applied to too many arguments, maybe you forgot a `;'

```
# fun x -> add_triple (5,4,x);;
: int -> int = <fun>
```

Match Expressions

```
# let triple_to_pair triple =
  match triple
  with (0, x, y) -> (x, y)
  | (x, 0, y) -> (x, y)
  | (x, y, _) -> (x, y);;
val triple_to_pair : int * int * int -> int * int =
  <fun>
```

- Each clause: pattern on left, expression on right
- Each x, y has scope of only its clause
- Use first matching clause

Save the Environment!

- A *closure* is a pair of an environment and an association of a pattern (e.g. (v1,…,vn) giving the input variables) with an expression (the function body), written:

  $< (v1,\ldots,vn) \rightarrow exp, \rho >$

- Where ρ is the environment in effect when the function is defined (for a simple function)

Closure for plus_pair

- Assume ρ_{plus_pair} was the environment just before plus_pair defined
- Closure for fun (n,m) -> n + m:

  $<(n,m) \rightarrow n + m, \rho_{plus_pair}>$

- Environment just after plus_pair defined:

  $\{plus_pair \rightarrow <(n,m) \rightarrow n + m, \rho_{plus_pair} >\}$

  $+ \rho_{plus_pair}$

Evaluating declarations

- Evaluation uses an environment ρ
- To evaluate a (simple) declaration let x = e
  - Evaluate expression e in ρ to value v
  - Update ρ with x $\rightarrow$ v: $\{x \rightarrow v\} + \rho$

Evaluating declarations

- Evaluation uses an environment ρ
- To evaluate a (simple) declaration let x = e
  - Evaluate expression e in ρ to value v
  - Update ρ with x v: $\{x \rightarrow v\} + \rho$
- Update: $\rho_1 + \rho_2$ has all the bindings in ρ_1 and all those in ρ_2 that are not rebound in ρ_1

$\{x \rightarrow 2, y \rightarrow 3, a \rightarrow$ "hi"$\} + \{y \rightarrow 100, b \rightarrow 6\}$
$= \{x \rightarrow 2, y \rightarrow 3, a \rightarrow$ "hi", $b \rightarrow 6\}$

Evaluating expressions in OCaml

- Evaluation uses an environment ρ
- A constant evaluates to itself, including primitive operators like + and =

Evaluating expressions in OCaml

- Evaluation uses an environment ρ
- A constant evaluates to itself, including primitive operators like + and =
- To evaluate a variable, look it up in ρ: $\rho(v)$

Evaluating expressions in OCaml

- Evaluation uses an environment ρ
- A constant evaluates to itself, including primitive operators like + and =
- To evaluate a variable, look it up in ρ: $\rho(v)$
- To evaluate a tuple $(e_1,\ldots,e_n)$,
  - Evaluate each e_i to v_i, right to left for Ocaml
  - Then make value $(v_1,\ldots,v_n)$

Evaluating expressions in OCaml

- To evaluate uses of +, - , etc, eval args, then do operation

Evaluating expressions in OCaml

- To evaluate uses of +, - , etc, eval args, then do operation
- Function expression evaluates to its closure

Evaluating expressions in OCaml

- To evaluate uses of +, - , etc, eval args, then do operation
- Function expression evaluates to its closure
- To evaluate a local dec: let x = e1 in e2
  - Eval e1 to v, then eval e2 using $\{x \rightarrow v\} + \rho$

Evaluating expressions in OCaml

- To evaluate uses of +, - , etc, eval args (right to left for Ocaml), then do operation
- Function expression evaluates to its closure
- To evaluate a local dec: let x = e1 in e2
  - Eval e1 to v, then eval e2 using $\{x \rightarrow v\} + \rho$
- To evaluate a conditional expression: if b then e1 else e2
  - Evaluate b to a value v
  - If v is True, evaluate e1
  - If v is False, evaluate e2

Evaluation of Application with Closures

- Given application expression f e
- In Ocaml, evaluate e to value v
- In environment ρ, evaluate left term to closure, $c = <(x_1,\ldots,x_n) \rightarrow b, \rho'>$
  - $(x_1,\ldots,x_n)$ variables in (first) argument
  - v must have form $(v_1,\ldots,v_n)$
- Update the environment ρ' to
  $\rho'' = \{x_1 \rightarrow v_1,\ldots, x_n \rightarrow v_n\} + \rho'$
- Evaluate body b in environment ρ''

Recursive Functions

```
# let rec factorial n =
    if n = 0 then 1 else n * factorial (n - 1);;
  val factorial : int -> int = <fun>
# factorial 5;;
- : int = 120
# (* rec  is needed for recursive function
  declarations *)
```


Recursion Example

Compute n^2 recursively using:
$n^2 = (2 * n - 1) + (n - 1)^2$

```
# let rec nthsq n =          (* rec for recursion *)
  match n                    (* pattern matching for cases *)
  with 0 -> 0                (* base case *)
  | n -> (2 * n -1)          (* recursive case *)
        + nthsq (n -1);;     (* recursive call *)
val nthsq : int -> int = <fun>
# nthsq 3;;
- : int = 9
```

Structure of recursion similar to inductive proof

Recursion and Induction

```
# let rec nthsq n = match n with 0 -> 0
      | n -> (2 * n - 1) + nthsq (n - 1) ;;
```

- Base case is the last case; it stops the computation
- Recursive call must be to arguments that are somehow smaller - must progress to base case
- **if** or **match** must contain base case
- Failure of these may cause failure of termination

Lists

- List can take one of two forms:
  - Empty list, written [ ]
  - Non-empty list, written x :: xs
    - x is head element, xs is tail list, :: called "cons"
  - Syntactic sugar: [x] == x :: [ ]
  - [x1; x2; …; xn] == x1 :: x2 :: … :: xn :: [ ]

Lists

```
# let fib5 = [8;5;3;2;1;1];;
val fib5 : int list = [8; 5; 3; 2; 1; 1]
# let fib6 = 13 :: fib5;;
val fib6 : int list = [13; 8; 5; 3; 2; 1; 1]
# (8::5::3::2::1::1::[ ]) = fib5;;
- : bool = true
# fib5 @ fib6;;
- : int list = [8; 5; 3; 2; 1; 1; 13; 8; 5; 3; 2; 1;
  1]
```

Lists are Homogeneous

```
# let bad_list = [1; 3.2; 7];;
Characters 19-22:
  let bad_list = [1; 3.2; 7];;
                      ^^^
This expression has type float but is here
  used with type int
```


Question

- Which one of these lists is invalid?

1. [2; 3; 4; 6]
2. [2,3; 4,5; 6,7]
3. [(2.3,4); (3.2,5); (6,7.2)]
4. [[“hi”; “there”]; [“wahcha”]; [ ]; [“doin”]]

Answer

- Which one of these lists is invalid?

1. [2; 3; 4; 6]
2. [2,3; 4,5; 6,7]
3. [(2.3,4); (3.2,5); (6,7.2)]
4. [[“hi”; “there”]; [“wahcha”]; [ ]; [“doin”]]

- 3 is invalid because of last pair

Functions Over Lists

```
# let rec double_up list =
    match list
    with [ ] -> [ ]  (* pattern before ->,
                        expression after *)
      | (x :: xs) -> (x :: x :: double_up xs);;
val double_up : 'a list -> 'a list = <fun>
# let fib5_2 = double_up fib5;;
val fib5_2 : int list = [8; 8; 5; 5; 3; 3; 2; 2; 1;
  1; 1; 1]
```


Functions Over Lists

```
# let silly = double_up ["hi"; "there"];;
val silly : string list = ["hi"; "hi"; "there"; "there"]
# let rec poor_rev list =
  match list
  with [] -> []
    | (x::xs) -> poor_rev xs @ [x];;
val poor_rev : 'a list -> 'a list = <fun>
# poor_rev silly;;
- : string list = ["there"; "there"; "hi"; "hi"]
```


Structural Recursion

- Functions on recursive datatypes (eg lists) tend to be recursive
- Recursion over recursive datatypes generally by structural recursion
  - Recursive calls made to components of structure of the same recursive type
  - Base cases of recursive types stop the recursion of the function

Question: Length of list

- Problem: write code for the length of the list
  - How to start?

```
let rec length list =
```


Question: Length of list

- Problem: write code for the length of the list
  - How to start?

```
let rec length list =
    match list with
```


Question: Length of list

- Problem: write code for the length of the list
  - What patterns should we match against?

```
let rec length list =
    match list with
```


Question: Length of list

- Problem: write code for the length of the list
  - What patterns should we match against?

```
let rec length list =
    match list with [] ->
     | (a :: bs) ->
```


Question: Length of list

- Problem: write code for the length of the list
  - What result do we give when list is empty?

```
let rec length list =
    match list with [] -> 0
     | (a :: bs) ->
```


Question: Length of list

- Problem: write code for the length of the list
  - What result do we give when list is not empty?

```
let rec length list =
    match list with [] -> 0
     | (a :: bs) ->
```

Question: Length of list

- Problem: write code for the length of the list
  - What result do we give when list is not empty?

```
let rec length list =
    match list with [] -> 0
     | (a :: bs) -> 1 + length bs
```


Structural Recursion : List Example

```
# let rec length list = match list
  with [ ] -> 0   (* Nil case *)
  | a :: bs -> 1 + length bs;;  (* Cons case *)
val length : 'a list -> int = <fun>
# length [5; 4; 3; 2];;
- : int = 4
```

- Nil case [ ] is base case
- Cons case recurses on component list bs

Same Length

- How can we efficiently answer if two lists have the same length?

Same Length

- How can we efficiently answer if two lists have the same length?

```
let rec same_length list1 list2 =
    match list1 with [] ->
      (match list2 with [] -> true
        | (y::ys) -> false)
    | (x::xs) ->
      (match list2 with [] -> false
        | (y::ys) -> same_length xs ys)
```


Your turn: doubleList : int list -> int list

- Write a function that takes a list of int and returns a list of the same length, where each element has been multiplied by 2

```
let rec doubleList list =
```


Your turn: doubleList : int list -> int list

- Write a function that takes a list of int and returns a list of the same length, where each element has been multiplied by 2

```
let rec doubleList list =
  match list
    with [] ->[]
      |  x :: xs -> (2 * x) :: doubleList xs
```

Your turn: doubleList : int list -> int list

- Write a function that takes a list of int and returns a list of the same length, where each element has been multiplied by 2

```
let rec doubleList list =
  match list
    with [] ->[]
     | x :: xs -> (2 * x) :: doubleList xs
```


Higher-Order Functions Over Lists

```
# let rec map f list =
  match list
  with [] -> []
  | (h::t) -> (f h) :: (map f t);;
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
# map plus_two fib5;;
- : int list = [10; 7; 5; 4; 3; 3]
# map (fun x -> x - 1) fib6;;
: int list = [12; 7; 4; 2; 1; 0; 0]
```


Higher-Order Functions Over Lists

```
# let rec map f list =
  match list
  with [] -> []
  | (h::t) -> (f h) :: (map f t);;
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
# map plus_two fib5;;
- : int list = [10; 7; 5; 4; 3; 3]
# map (fun x -> x - 1) fib6;;
: int list = [12; 7; 4; 2; 1; 0; 0]
```


Mapping Recursion

- Can use the higher-order recursive map function instead of direct recursion

```
# let doubleList list =
    List.map (fun x -> 2 * x) list;;
val doubleList : int list -> int list = <fun>
# doubleList [2;3;4];;
- : int list = [4; 6; 8]
```


Mapping Recursion

- Can use the higher-order recursive map function instead of direct recursion

```
# let doubleList list =
    List.map (fun x -> 2 * x) list;;
val doubleList : int list -> int list = <fun>
# doubleList [2;3;4];;
- : int list = [4; 6; 8]
```

- Same function, but no explicit recursion

Folding Recursion

- Another common form “folds” an operation over the elements of the structure

```
# let rec multList list = match list
  with [ ] -> 1
  | x::xs -> x * multList xs;;
val multList : int list -> int = <fun>
# multList [2;4;6];;
- : int = 48
```

- Computes (2 * (4 * (6 * 1)))

Folding Recursion : Length Example

```
# let rec length list = match list
  with [ ] -> 0   (* Nil case *)
  | a :: bs -> 1 + length bs;;  (* Cons case *)
val length : 'a list -> int = <fun>
# length [5; 4; 3; 2];;
- : int = 4
```

- Nil case [ ] is base case, 0 is the base value
- Cons case recurses on component list bs
- What do multList and length have in common?

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC



https://courses.engr.illinois.edu/cs421/fa2017/CS421D

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Evaluating declarations

- Evaluation uses an environment ρ
- To evaluate a (simple) declaration let x = e
  - Evaluate expression e in ρ to value v
  - Update ρ with x $\rightarrow$ v: $\{x \rightarrow v\} + \rho$

Evaluating declarations

- Evaluation uses an environment ρ
- To evaluate a (simple) declaration let x = e
  - Evaluate expression e in ρ to value v
  - Update ρ with x v: $\{x \rightarrow v\} + \rho$
- Update: $\rho_1 + \rho_2$ has all the bindings in ρ_1 and all those in ρ_2 that are not rebound in ρ_1

$\{x \rightarrow 2, y \rightarrow 3, a \rightarrow$ "hi"$\} + \{y \rightarrow 100, b \rightarrow 6\}$
$= \{x \rightarrow 2, y \rightarrow 3, a \rightarrow$ "hi", $b \rightarrow 6\}$

Evaluating expressions in OCaml

- Evaluation uses an environment ρ
- A constant evaluates to itself, including primitive operators like + and =

Evaluating expressions in OCaml

- Evaluation uses an environment ρ
- A constant evaluates to itself, including primitive operators like + and =
- To evaluate a variable, look it up in ρ: $\rho(v)$

Evaluating expressions in OCaml

- Evaluation uses an environment ρ
- A constant evaluates to itself, including primitive operators like + and =
- To evaluate a variable, look it up in ρ: $\rho(v)$
- To evaluate a tuple $(e_1,\ldots,e_n)$,
  - Evaluate each e_i to v_i, right to left for Ocaml
  - Then make value $(v_1,\ldots,v_n)$

Evaluating expressions in OCaml

- To evaluate uses of +, - , etc, eval args, then do operation

Evaluating expressions in OCaml

- To evaluate uses of +, - , etc, eval args, then do operation
- Function expression evaluates to its closure

Evaluating expressions in OCaml

- To evaluate uses of +, - , etc, eval args, then do operation
- Function expression evaluates to its closure
- To evaluate a local dec: let x = e1 in e2
  - Eval e1 to v, then eval e2 using $\{x \rightarrow v\} + \rho$

Evaluating expressions in OCaml

- To evaluate uses of +, - , etc, eval args (right to left for Ocaml), then do operation
- Function expression evaluates to its closure
- To evaluate a local dec: let x = e1 in e2
  - Eval e1 to v, then eval e2 using $\{x \rightarrow v\} + \rho$
- To evaluate a conditional expression: if b then e1 else e2
  - Evaluate b to a value v
  - If v is True, evaluate e1
  - If v is False, evaluate e2

Evaluation of Application with Closures

- Given application expression f e
- In Ocaml, evaluate e to value v
- In environment ρ, evaluate left term to closure, $c = <(x_1,\ldots,x_n) \rightarrow b, \rho'>$
  - $(x_1,\ldots,x_n)$ variables in (first) argument
  - v must have form $(v_1,\ldots,v_n)$
- Update the environment ρ' to $\rho'' = \{x_1 \rightarrow v_1,\ldots, x_n \rightarrow v_n\} + \rho'$
- Evaluate body b in environment ρ''

Recursive Functions

```
# let rec factorial n =
    if n = 0 then 1 else n * factorial (n - 1);;
  val factorial : int -> int = <fun>
# factorial 5;;
- : int = 120
# (* rec  is needed for recursive function
  declarations *)
```

Recursion Example

Compute n^2 recursively using:
$n^2 = (2 * n - 1) + (n - 1)^2$

```
# let rec nthsq n =          (* rec for recursion *)
   match n                 (* pattern matching for cases *)
   with 0 -> 0                  (* base case *)
   | n -> (2 * n -1)            (* recursive case *)
          + nthsq (n -1);;   (* recursive call *)
val nthsq : int -> int = <fun>
# nthsq 3;;
- : int = 9
```

Structure of recursion similar to inductive proof

Recursion and Induction

```
# let rec nthsq n = match n with 0 -> 0
      | n -> (2 * n - 1) + nthsq (n - 1) ;;
```

- Base case is the last case; it stops the computation
- Recursive call must be to arguments that are somehow smaller - must progress to base case
- **if** or **match** must contain base case
- Failure of these may cause failure of termination

Lists

- List can take one of two forms:
  - Empty list, written [ ]
  - Non-empty list, written x :: xs
    - x is head element, xs is tail list, :: called "cons"
  - Syntactic sugar: [x] == x :: [ ]
  - [x1; x2; …; xn] == x1 :: x2 :: … :: xn :: [ ]

Lists

```
# let fib5 = [8;5;3;2;1;1];;
val fib5 : int list = [8; 5; 3; 2; 1; 1]
# let fib6 = 13 :: fib5;;
val fib6 : int list = [13; 8; 5; 3; 2; 1; 1]
# (8::5::3::2::1::1::[ ]) = fib5;;
- : bool = true
# fib5 @ fib6;;
- : int list = [8; 5; 3; 2; 1; 1; 13; 8; 5; 3; 2; 1;
  1]
```


Lists are Homogeneous

```
# let bad_list = [1; 3.2; 7];;
Characters 19-22:
  let bad_list = [1; 3.2; 7];;
                     ^^^
This expression has type float but is here
  used with type int
```


Question

- Which one of these lists is invalid?

1. [2; 3; 4; 6]
2. [2,3; 4,5; 6,7]
3. [(2.3,4); (3.2,5); (6,7.2)]
4. [["hi"; "there"]; ["wahcha"]; [ ]; ["doin"]]

Answer

- Which one of these lists is invalid?

1. [2; 3; 4; 6]
2. [2,3; 4,5; 6,7]
3. [(2.3,4); (3.2,5); (6,7.2)]
4. [[“hi”; “there”]; [“wahcha”]; [ ]; [“doin”]]

- 3 is invalid because of last pair

Functions Over Lists

```
# let rec double_up list =
    match list
    with [ ] -> [ ]  (* pattern before ->,
                        expression after *)
       | (x :: xs) -> (x :: x :: double_up xs);;
val double_up : 'a list -> 'a list = <fun>
# let fib5_2 = double_up fib5;;
val fib5_2 : int list = [8; 8; 5; 5; 3; 3; 2; 2; 1;
  1; 1; 1]
```


Functions Over Lists

```
# let silly = double_up ["hi"; "there"];;
val silly : string list = ["hi"; "hi"; "there"; "there"]
# let rec poor_rev list =
  match list
  with [] -> []
     | (x::xs) -> poor_rev xs @ [x];;
val poor_rev : 'a list -> 'a list = <fun>
# poor_rev silly;;
- : string list = ["there"; "there"; "hi"; "hi"]
```


Structural Recursion

- Functions on recursive datatypes (eg lists) tend to be recursive
- Recursion over recursive datatypes generally by structural recursion
  - Recursive calls made to components of structure of the same recursive type
  - Base cases of recursive types stop the recursion of the function

Question: Length of list

- Problem: write code for the length of the list
  - How to start?

```
let rec length list =
```


Question: Length of list

- Problem: write code for the length of the list
  - How to start?

```
let rec length list =
    match list with
```

Question: Length of list

- Problem: write code for the length of the list
  - What patterns should we match against?

```
let rec length list =
    match list with
```


Question: Length of list

- Problem: write code for the length of the list
  - What patterns should we match against?

```
let rec length list =
    match list with [] ->
     | (a :: bs) ->
```


Question: Length of list

- Problem: write code for the length of the list
  - What result do we give when list is empty?

```
let rec length list =
    match list with [] ->
     | (a :: bs) ->
```


Question: Length of list

- Problem: write code for the length of the list
  - What result do we give when list is empty?

```
let rec length list =
    match list with [] -> 0
     | (a :: bs) ->
```


Question: Length of list

- Problem: write code for the length of the list
  - What result do we give when list is not empty?

```
let rec length list =
    match list with [] -> 0
     | (a :: bs) ->
```


Question: Length of list

- Problem: write code for the length of the list
  - What result do we give when list is not empty?

```
let rec length list =
    match list with [] -> 0
     | (a :: bs) -> 1 + length bs
```

Structural Recursion : List Example

```
# let rec length list = match list
  with [ ] -> 0   (* Nil case *)
  | a :: bs -> 1 + length bs;;  (* Cons case *)
val length : 'a list -> int = <fun>
# length [5; 4; 3; 2];;
- : int = 4
```

- Nil case [ ] is base case
- Cons case recurses on component list bs

Same Length

- How can we efficiently answer if two lists have the same length?

Same Length

- How can we efficiently answer if two lists have the same length?

```
let rec same_length list1 list2 =
    match list1 with [] ->


    | (x::xs) ->
```


Same Length

- How can we efficiently answer if two lists have the same length?

```
let rec same_length list1 list2 =
    match list1 with [] ->
      (match list2 with [] -> true
        | (y::ys) -> false)
    | (x::xs) ->
```


Same Length

- How can we efficiently answer if two lists have the same length?

```
let rec same_length list1 list2 =
    match list1 with [] ->
      (match list2 with [] ->
        | (y::ys) ->       )
    | (x::xs) ->
```


Same Length

- How can we efficiently answer if two lists have the same length?

```
let rec same_length list1 list2 =
    match list1 with [] ->
      (match list2 with [] -> true
        | (y::ys) -> false)
    | (x::xs) ->
      (match list2 with [] ->
        | (y::ys) ->                    )
```

Same Length

- How can we efficiently answer if two lists have the same length?

```
let rec same_length list1 list2 =
   match list1 with [] ->
     (match list2 with [] -> true
       | (y::ys) -> false)
   | (x::xs) ->
     (match list2 with [] -> false
       | (y::ys) ->                    )
```


Same Length

- How can we efficiently answer if two lists have the same length?

```
let rec same_length list1 list2 =
   match list1 with [] ->
     (match list2 with [] -> true
       | (y::ys) -> false)
   | (x::xs) ->
     (match list2 with [] -> false
       | (y::ys) -> same_length xs ys)
```


Your turn: doubleList : int list -> int list

- Write a function that takes a list of int and returns a list of the same length, where each element has been multiplied by 2

```
let rec doubleList list =
```


Your turn: doubleList : int list -> int list

- Write a function that takes a list of int and returns a list of the same length, where each element has been multiplied by 2

```
let rec doubleList list =
  match list
    with [] ->[]
      |  x :: xs -> (2 * x) :: doubleList xs
```


Your turn: doubleList : int list -> int list

- Write a function that takes a list of int and returns a list of the same length, where each element has been multiplied by 2

```
let rec doubleList list =
  match list
    with [] ->[]
      |  x :: xs -> (2 * x) :: doubleList xs
```


Folding Recursion

- Another common form "folds" an operation over the elements of the structure

```
# let rec multList list = match list
  with [ ] -> 1
  | x::xs -> x * multList xs;;
val multList : int list -> int = <fun>
# multList [2;4;6];;
- : int = 48
```

- Computes (2 * (4 * (6 * 1)))

Folding Recursion : Length Example

```
# let rec length list = match list
  with [ ] -> 0   (* Nil case *)
  | a :: bs -> 1 + length bs;;  (* Cons case *)
val length : 'a list -> int = <fun>
# length [5; 4; 3; 2];;
- : int = 4
```

- Nil case [ ] is base case, 0 is the base value
- Cons case recurses on component list bs
- What do multList and length have in common?

Forward Recursion

- In Structural Recursion, split input into components and (eventually) recurse
- Forward Recursion form of Structural Recursion
- In forward recursion, **first** call the function recursively on all recursive components, and then build final result from partial results
- Wait until whole structure has been traversed to start building answer

Forward Recursion: Examples

```
# let rec double_up list =
    match list
    with [ ] -> [ ]
      | (x :: xs) -> (x :: x :: double_up xs);;
val double_up : 'a list -> 'a list = <fun>

# let rec poor_rev list =
  match list
  with [] -> []
    | (x::xs) -> let r = poor_rev xs in r @ [x];;
val poor_rev : 'a list -> 'a list = <fun>
```


Forward Recursion: Examples

```
# let rec double_up list =
    match list
    with [ ] -> [ ]
      | (x :: xs) -> (x :: x :: double_up xs);;
val double_up : 'a list -> 'a list = <fun>
```

Base Case | Operator | Recursive Call

```
# let rec poor_rev list =
  match list
  with [] -> []
    | (x::xs) -> let r = poor_rev xs in r @ [x];;
val poor_rev : 'a list -> 'a list = <fun>
```

Base Case | Operator | Recursive Call

Recursing over lists



The Primitive Recursion Fairy

```
# let rec fold_right f list b =
  match list
  with [] -> b
  | (x :: xs) -> f x (fold_right f xs b);;
val fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b =
  <fun>
# fold_right
    (fun s -> fun () -> print_string s)
    ["hi"; "there"]
    ();;
therehi- : unit = ()
```


Folding Recursion : Length Example

```
# let rec length list = match list
  with [ ] -> 0   (* Nil case *)
  | a :: bs -> 1 + length bs;;  (* Cons case *)
val length : 'a list -> int = <fun>
# let length list =
fold_right (fun a -> fun r -> 1 + r) list 0;;
val length : 'a list -> int = <fun>
# length [5; 4; 3; 2];;
- : int = 4
```

Forward Recursion: Examples

```
# let rec double_up list =
    match list
    with [ ] -> [ ]
      | (x :: xs) -> (x :: x :: double_up xs);;
val double_up : 'a list -> 'a list = <fun>
# let double_up =
    fold_right (fun x -> fun r -> x :: x :: r) list [ ]
# double_up  ["a";"b"];;
- : string list = ["a"; "a"; "b"; "b"]
```

Base Case | Operator | Recursive Call

Operator | Recursive result | Base Case

- let rec multList_fr list =
  match list
  with [] -> 1
  | (x::xs) -> let r = (multList_fr ns) in
  (x * r)

Folding Recursion

- multList folds to the right
- Same as:

```
# let multList list =
    List.fold_right
    (fun x -> fun p -> x * p)
    list 1;;
val multList : int list -> int = <fun>
# multList [2;4;6];;
- : int = 48
```


Terminology

- Available: A function call that can be executed by the current expression
- The fastest way to be unavailable is to be guarded by an abstraction (anonymous function, lambda lifted).
  - if (h x) then f x else (x + g x)
  - if (h x) then (fun x -> f x) else (g (x + x))

Not available

Terminology

- Tail Position: A subexpression s of expressions e, which is available and such that if evaluated, will be taken as the value of e (last thing done in this expression)
  - if (x>3) then x + 2 else x - 4
  - let x = 5 in x + 4
- Tail Call: A function call that occurs in tail position
  - if (h x) then f x else (x + g x)

Tail Recursion

- A recursive program is tail recursive if all recursive calls are tail calls
- Tail recursive programs may be optimized to be implemented as loops, thus removing the function call overhead for the recursive calls
- Tail recursion generally requires extra "accumulator" arguments to pass partial results
  - May require an auxiliary function

Tail Recursion - length

- How can we write length with tail recursion?

```
let length list =
   let rec length_aux list acc_length =
         match list
          with [ ] -> acc_length
            | (x::xs) ->
              length_aux xs (1 + acc_length)
    in length_aux list  0
```


Your turn: num_neg – tail recursive

```
# let num_neg list =
```


Your turn: num_neg – tail recursive

```
# let num_neg list =
 let rec num_neg_aux list curr_neg =




  in num_neg_aux  ?  ?
```


Your turn: num_neg – tail recursive

```
# let num_neg list =
 let rec num_neg_aux list curr_neg =
    match list with [] ->
      | (x :: xs) ->




  in num_neg_aux  ?  ?
```


Your turn: num_neg – tail recursive

```
# let num_neg list =
 let rec num_neg_aux list curr_neg =
    match list with [] -> curr_neg
      | (x :: xs) ->




  in num_neg_aux  ?  ?
```


Your turn: num_neg – tail recursive

```
# let num_neg list =
 let rec num_neg_aux list curr_neg =
    match list with [] -> curr_neg
      | (x :: xs) ->
        num_neg_aux xs  ?




  in num_neg_aux  ?  ?
```

Your turn: num_neg – tail recursive

```
# let num_neg list =
 let rec num_neg_aux list curr_neg =
    match list with [] -> curr_neg
      | (x :: xs) ->
        num_neg_aux xs
          (if x < 0 then 1 + curr_neg
           else curr_neg)
  in num_neg_aux  ?  ?
```


Your turn: num_neg – tail recursive

```
# let num_neg list =
 let rec num_neg_aux list curr_neg =
    match list with [] -> curr_neg
      | (x :: xs) ->
        num_neg_aux xs
          (if x < 0 then 1 + curr_neg
           else curr_neg)
  in num_neg_aux list  ?
```


Your turn: num_neg – tail recursive

```
# let num_neg list =
 let rec num_neg_aux list curr_neg =
    match list with [] -> curr_neg
      | (x :: xs) ->
        num_neg_aux xs
          (if x < 0 then 1 + curr_neg
           else curr_neg)
  in num_neg_aux list 0
```



```
let num_neg list =
List.fold_left
  (fun curr_neg -> (fun x ->
     (if x < 0 then 1 + curr_neg else curr_neg)
                    )
  )
0
list
```


Folding

```
# let rec fold_left f a list = match list
  with [] -> a | (x :: xs) -> fold_left f (f a x) xs;;
val fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a =
   <fun>
```

fold_left f a [x_1; x_2;…;x_n] = f(…(f (f a x_1) x_2)…)x_n

```
# let rec fold_right f list b = match list
  with [ ] -> b | (x :: xs) -> f x (fold_right f xs b);;
val fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b =
   <fun>
```

fold_right f [x_1; x_2;…;x_n] b = f x_1(f x_2 (…(f x_n b)…))

Folding

- Can replace recursion by fold_right in any forward primitive recursive definition
  - Primitive recursive means it only recurses on immediate subcomponents of recursive data structure
- Can replace recursion by fold_left in any tail primitive recursive definition

Mapping Recursion

```
# let rec map f list =
  match list
  with [] -> []
  | (h::t) -> (f h) :: (map f t);;
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
# map plus_two fib5;;
- : int list = [10; 7; 5; 4; 3; 3]
# map (fun x -> x - 1) fib6;;
: int list = [12; 7; 4; 2; 1; 0; 0]
```


Map is forward recursive

```
# let rec map f list =
  match list
  with [] -> []
  | (h::t) -> (f h) :: (map f t);;
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
# let map f list =
    List.fold_right (fun h -> fun r -> (f h) :: r)
                    list  [];;
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
```


Mapping Recursion

- Can use the higher-order recursive map function instead of direct recursion

```
# let doubleList list =
    List.map (fun x -> 2 * x) list;;
val doubleList : int list -> int list = <fun>
# doubleList [2;3;4];;
- : int list = [4; 6; 8]
```


Mapping Recursion

- Can use the higher-order recursive map function instead of direct recursion

```
# let doubleList list =
    List.map (fun x -> 2 * x) list;;
val doubleList : int list -> int list = <fun>
# doubleList [2;3;4];;
- : int list = [4; 6; 8]
```

- Same function, but no explicit recursion

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Folding

```
# let rec fold_left f a list = match list
  with [] -> a | (x :: xs) -> fold_left f (f a x) xs;;
val fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a =
  <fun>
```

fold_left f a [x_1; x_2;...;x_n] = f(...(f (f a x_1) x_2)...)x_n

```
# let rec fold_right f list b = match list
  with [ ] -> b | (x :: xs) -> f x (fold_right f xs b);;
val fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b =
  <fun>
```

fold_right f [x_1; x_2;...;x_n] b = f x_1(f x_2 (...(f x_n b)...))

Folding

- Can replace recursion by fold_right in any forward primitive recursive definition
  - Primitive recursive means it only recurses on immediate subcomponents of recursive data structure
- Can replace recursion by fold_left in any tail primitive recursive definition

Mapping Recursion

```
# let rec map f list =
  match list
  with [] -> []
  | (h::t) -> (f h) :: (map f t);;
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
# map plus_two fib5;;
- : int list = [10; 7; 5; 4; 3; 3]
# map (fun x -> x - 1) fib6;;
: int list = [12; 7; 4; 2; 1; 0; 0]
```


Map is forward recursive

```
# let rec map f list =
  match list
  with [] -> []
  | (h::t) -> (f h) :: (map f t);;
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
# let map f list =
    List.fold_right (fun h -> fun r -> (f h) :: r)
                    list  [];;
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
```


Mapping Recursion

- Can use the higher-order recursive map function instead of explicit recursion

```
# let doubleList list =
    List.map (fun x -> 2 * x) list;;
val doubleList : int list -> int list = <fun>
# doubleList [2;3;4];;
- : int list = [4; 6; 8]
```

Mapping Recursion

- Can use the higher-order recursive map function instead of explicit recursion

```
# let doubleList list =
    List.map (fun x -> 2 * x) list;;
val doubleList : int list -> int list = <fun>
# doubleList [2;3;4];;
- : int list = [4; 6; 8]
```

- Same function, but no explicit recursion

Continuations

- A programming technique for all forms of "non-local" control flow:
  - non-local jumps
  - exceptions
  - general conversion of non-tail calls to tail calls
- Essentially it's a higher-order function version of GOTO

Continuations

- Idea: Use functions to represent the control flow of a program
- Method: Each procedure takes a function as an extra argument to which to pass its result; outer procedure "returns" no result
- Function receiving the result called a continuation
- Continuation acts as "accumulator" for work still to be done

Continuation Passing Style

- Writing procedures such that all procedure calls take a continuation to which to give (pass) the result, and return no result, is called continuation passing style (CPS)

Continuation Passing Style

- A compilation technique to implement non-local control flow, especially useful in interpreters.
- A formalization of non-local control flow in denotational semantics
- Possible intermediate state in compiling functional code

Why CPS?

- Makes order of evaluation explicitly clear
- Allocates variables (to become registers) for each step of computation
- Essentially converts functional programs into imperative ones
  - Major step for compiling to assembly or byte code
- Tail recursion easily identified
- Strict forward recursion converted to tail recursion
  - At the expense of building large closures in heap

Other Uses for Continuations

- CPS designed to preserve order of evaluation
- Continuations used to express order of evaluation
- Can be used to change order of evaluation
- Implements:
  - Exceptions and exception handling
  - Co-routines
  - (pseudo, aka green) threads

Example

- Simple reporting continuation:

```
# let report x = (print_int x; print_newline( ) );;
val report : int -> unit = <fun>
```

- Simple function using a continuation:

```
# let addk (a, b) k = k (a + b);;
val addk : int * int -> (int -> 'a) -> 'a = <fun>
# addk (22, 20) report;;
2
- : unit = ()
```


Simple Functions Taking Continuations

- Given a primitive operation, can convert it to pass its result forward to a continuation
- Examples:

```
# let subk (x, y) k = k(x - y);;
val subk : int * int -> (int -> 'a) -> 'a = <fun>
# let eqk (x, y) k = k(x = y);;
val eqk : 'a * 'a -> (bool -> 'b) -> 'b = <fun>
# let timesk (x, y) k = k(x * y);;
val timesk : int * int -> (int -> 'a) -> 'a = <fun>
```


Nesting Continuations

```
# let add_triple (x, y, z) = (x + y) + z;;
val add_triple : int * int * int -> int = <fun>
# let add_triple (x,y,z)=let p = x + y in p + z;;
val add_triple : int * int * int -> int = <fun>
# let add_triple_k (x, y, z) k =
   addk (x, y) (fun p -> addk (p, z) k );;
val add_triple_k: int * int * int -> (int -> 'a) ->
  'a = <fun>
```


add_three: a different order

- # let add_triple (x, y, z) = x + (y + z);;
- How do we write add_triple_k to use a different order?

- let add_triple_k (x, y, z) k =

add_three: a different order

- # let add_triple (x, y, z) = x + (y + z);;
- How do we write add_triple_k to use a different order?

- let add_triple_k (x, y, z) k =

```
      addk (y,z) (fun r -> addk(x,r) k)
```

Recursive Functions

- Recall:

```
# let rec factorial n =
    if n = 0 then 1 else n * factorial (n - 1);;
  val factorial : int -> int = <fun>
# factorial 5;;
- : int = 120
```


Terms

- A function is in Direct Style when it returns its result back to the caller.
- A function is in Continuation Passing Style when it, and every function call in it, passes its result to another function.
- Instead of returning the result to the caller, we pass it forward to another function giving the computation after the call.

Recursive Functions

```
# let rec factorial n =
   let b = (n = 0) in (* First computation *)
   if b then 1 (* Returned value *)
   else let s = n – 1 in (* Second computation *)
         let r = factorial s in  (* Third computation *)
         n * r (* Returned value *) ;;
val factorial : int -> int = <fun>
# factorial 5;;
- : int = 120
```


Recursive Functions

```
# let rec factorialk n k =
   eqk (n, 0)
   (fun b ->  (* First computation *)
    if b then k 1 (* Passed value *)
    else subk (n, 1)  (* Second computation *)
    (fun s -> factorialk s  (* Third computation *)
      (fun r -> timesk (n, r) k))) (* Passed value *)
val factorialk : int -> (int -> 'a) -> 'a = <fun>
# factorialk 5 report;;
120
- : unit = ()
```


Recursive Functions

- To make recursive call, must build intermediate continuation to
  - take recursive value: r
  - build it to final result: n * r
  - And pass it to final continuation:
  - times (n, r) k = k (n * r)

425 minutes

Example: CPS for length

```
let rec length list = match list with [] -> 0
    | (a :: bs) -> 1 + length bs
```

What is the let-expanded version of this?

Example: CPS for length

```
let rec length list = match list with [] -> 0
    | (a :: bs) -> 1 + length bs
```

What is the let-expanded version of this?

```
Let rec length list = match list with [] -> 0
 | (a :: bs) -> let n = length bs in 1 + n

Let lengthk list k = match list with [] -> k 0
| (a :: bs) -> lengthk bs (fun n -> addk (1,n)
k)
```


Example: CPS for length

```
let rec length list = match list with [] -> 0
    | (a :: bs) -> 1 + length bs
```

What is the let-expanded version of this?

```
let rec length list = match list with [] -> 0
    | (a :: bs) -> let r1 = length bs in 1 + r1
```


Example: CPS for length

```
#let rec length list = match list with [] -> 0
    | (a :: bs) -> let r1 = length bs in 1 + r1
```

What is the CSP version of this?

Example: CPS for length

```
#let rec length list = match list with [] -> 0
    | (a :: bs) -> let r1 = length bs in 1 + r1
```

What is the CSP version of this?

```
#let rec lengthk list k = match list with [ ] -> k 0
   | x :: xs -> lengthk xs (fun r -> addk (1,r) k);;
val lengthk : 'a list -> (int -> 'b) -> 'b = <fun>
# lengthk [2;4;6;8] report;;
4
- : unit = ()
```


450 minutes

CPS for sum

```
# let rec sum list = match list with [ ] -> 0
    | x :: xs -> x + sum xs ;;
val sum : int list -> int = <fun>
```


CPS for sum

```
# let rec sum list = match list with [ ] -> 0
    | x :: xs -> x + sum xs ;;
val sum : int list -> int = <fun>
# let rec sum list = match list with [ ] -> 0
    | x :: xs -> let r1 = sum xs  in x + r1;;
```


CPS for sum

```
# let rec sum list = match list with [ ] -> 0
    | x :: xs -> x + sum xs ;;
val sum : int list -> int = <fun>
# let rec sum list = match list with [ ] -> 0
    | x :: xs -> let r1 = sum xs  in x + r1;;
val sum : int list -> int = <fun>
# let rec sumk list k = match list with [ ] -> k 0
     | x :: xs -> sumk xs  (fun r1 -> addk x r1 k);;
```


CPS for sum

```
# let rec sum list = match list with [ ] -> 0
    | x :: xs -> x + sum xs ;;
val sum : int list -> int = <fun>
# let rec sum list = match list with [ ] -> 0
    | x :: xs -> let r1 = sum xs  in x + r1;;
val sum : int list -> int = <fun>
# let rec sumk list k = match list with [ ] -> k 0
     | x :: xs -> sumk xs  (fun r1 -> addk (x, r1) k);;
val sumk : int list -> (int -> 'a) -> 'a = <fun>
# sumk [2;4;6;8] report;;
20
- : unit = ()
```


CPS for Higher Order Functions

- In CPS, every procedure / function takes a continuation to receive its result
- Procedures passed as arguments take continuations
- Procedures returned as results take continuations
- CPS version of higher-order functions must expect input procedures to take continuations

Example: all

```
#let rec all (p, l) = match l with [] -> true
     | (x :: xs) -> let b = p x in
          if b then all (p, xs) else false
val all : ('a -> bool) -> 'a list -> bool = <fun>
```

- What is the CPS version of this?

Example: all

```
#let rec all (p, l) = match l with [] -> true
    | (x :: xs) -> let b = p x in
        if b then all (p, xs) else false
val all : ('a -> bool) -> 'a list -> bool = <fun>
```

- What is the CPS version of this?

```
#let rec allk (pk, l) k =
```


Example: all

```
#let rec all (p, l) = match l with [] -> true
    | (x :: xs) -> let b = p x in
        if b then all (p, xs) else false
val all : ('a -> bool) -> 'a list -> bool = <fun>
```

- What is the CPS version of this?

```
#let rec allk (pk, l) k = match l with [] ->     true
```


Example: all

```
#let rec all (p, l) = match l with [] -> true
    | (x :: xs) -> let b = p x in
        if b then all (p, xs) else false
val all : ('a -> bool) -> 'a list -> bool = <fun>
```

- What is the CPS version of this?

```
#let rec allk (pk, l) k = match l with [] ->  k true
```


Example: all

```
#let rec all (p, l) = match l with [] -> true
    | (x :: xs) -> let b = p x in
        if b then all (p, xs) else false
val all : ('a -> bool) -> 'a list -> bool = <fun>
```

- What is the CPS version of this?

```
#let rec allk (pk, l) k = match l with [] ->  k true
 | (x :: xs) ->
```


Example: all

```
#let rec all (p, l) = match l with [] -> true
    | (x :: xs) -> let b = p x in
        if b then all (p, xs) else false
val all : ('a -> bool) -> 'a list -> bool = <fun>
```

- What is the CPS version of this?

```
#let rec allk (pk, l) k = match l with [] ->  k true
 | (x :: xs) -> pk x
```


Example: all

```
#let rec all (p, l) = match l with [] -> true
    | (x :: xs) -> let b = p x in
        if b then all (p, xs) else false
val all : ('a -> bool) -> 'a list -> bool = <fun>
```

- What is the CPS version of this?

```
#let rec allk (pk, l) k = match l with [] ->  k true
 | (x :: xs) -> pk x
          (fun b -> if b then                else
      )
```

Example: all

```
#let rec all (p, l) = match l with [] -> true
    | (x :: xs) -> let b = p x in
       if b then all (p, xs) else false
val all : ('a -> bool) -> 'a list -> bool = <fun>
```

- What is the CPS version of this?

```
#let rec allk (pk, l) k = match l with [] ->  k true
 | (x :: xs) -> pk x
          (fun b -> if b then allk (pk, xs) k else k
false)
val allk : ('a -> (bool -> 'b) -> 'b) * 'a list ->
(bool -> 'b) -> 'b = <fun>
```


475 minutes

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Terminology: Review

- A function is in Direct Style when it returns its result back to the caller.
- A function is in Continuation Passing Style when it, and every function call in it, passes its result to another function.
- A Tail Call occurs when a function returns the result of another function call without any more computations (eg tail recursion)
- Instead of returning the result to the caller, we pass it forward to another function giving the computation after the call.

CPS Transformation

- Step 1: Add continuation argument to any function definition:
  - let f arg = e ⇒ let f arg k = e
  - Idea: Every function takes an extra parameter saying where the result goes
- Step 2: A simple expression in tail position should be passed to a continuation instead of returned:
  - return a ⇒ k a
  - Assuming a is a constant or variable.
  - "Simple" = "No available function calls."

CPS Transformation

- Step 3: Pass the current continuation to every function call in tail position
  - return f arg ⇒ f arg k
  - The function "isn't going to return," so we need to tell it where to put the result.

CPS Transformation

- Step 4: Each function call not in tail position needs to be converted to take a new continuation (containing the old continuation as appropriate)
  - return op (f arg) ⇒ f arg (fun r -> k(op r))
  - op represents a primitive operation
  - return g(f arg) ⇒ f arg (fun r-> g r k)

Example

Before:

```
let rec add_list lst =
match lst with
  [ ] -> 0
| 0 :: xs -> add_list xs
| x :: xs -> (+) x
   (add_list xs);;
```

After:

```
let rec add_listk lst k =
                    (* rule 1 *)
match lst with
| [ ] -> k 0 (* rule 2 *)
| 0 :: xs -> add_listk xs k
                     (* rule 3 *)
| x :: xs -> add_listk xs
          (fun r -> k ((+) x r));;
                (* rule 4 *)
```

Example

Before:

```
let rec mem (y,lst) =
match lst with
  [ ] -> false
| x :: xs ->
  if (x = y)
   then true
   else mem(y,xs);;
```

After:

```
let rec memk (y,lst) k =
                    (* rule 1 *)
```


Example

Before:

```
let rec mem (y,lst) =
match lst with
  [ ] -> false
| x :: xs ->
  if (x = y)
   then true
   else mem(y,xs);;
```

After:

```
let rec memk (y,lst) k =
                    (* rule 1 *)

          k false (* rule 2 *)


          k true (* rule 2 *)
```


Example

Before:

```
let rec mem (y,lst) =
match lst with
  [ ] -> false
| x :: xs ->
  if (x = y)
   then true
   else mem(y,xs);;
```

After:

```
let rec memk (y,lst) k =
                    (* rule 1 *)

          k false (* rule 2 *)


      k true (* rule 2 *)
        memk (y, xs) k (* rule 3 *)
```


Example

Before:

```
let rec mem (y,lst) =
match lst with
  [ ] -> false
| x :: xs ->
  if (x = y)
   then true
   else mem(y,xs);;
```

After:

```
let rec memk (y,lst) k =
                    (* rule 1 *)

          k false (* rule 2 *)

 eqk (x, y)
  (fun b ->   b (* rule 4 *)
      k true (* rule 2 *)
        memk (y, xs) (* rule 3 *)
```


Example

Before:

```
let rec mem (y,lst) =
match lst with
  [ ] -> false
| x :: xs ->
  if (x = y)
   then true
   else mem(y,xs);;
```

After:

```
let rec memk (y,lst) k =
                    (* rule 1 *)

          k false (* rule 2 *)

 eqk (x, y)
  (fun b ->if b (* rule 4 *)
then k true (* rule 2 *)
   else memk (y, xs) (* rule 3 *)
```


Example

Before:

```
let rec mem (y,lst) =
match lst with
  [ ] -> false
| x :: xs ->
  if (x = y)
   then true
   else mem(y,xs);;
```

After:

```
let rec memk (y,lst) k =
                    (* rule 1 *)
match lst with
| [ ] -> k false (* rule 2 *)
| x :: xs ->
 eqk (x, y)
  (fun b ->if b (* rule 4 *)
then k true (* rule 2 *)
   else memk (y, xs) k (* rule 3 *)
```

Example

Before:

```
let rec mem (y,lst) =
match lst with
  [ ] -> false
| x :: xs ->
  if (x = y)
   then true
   else mem(y,xs);;
```

After:

```
let rec memk (y,lst) k =
                  (* rule 1 *)
match lst with
| [ ] -> k false (* rule 2 *)
| x :: xs ->
 eqk (x, y)
  (fun b ->if b (* rule 4 *)
then k true (* rule 2 *)
   else memk (y, xs) k (* rule 3 *)
```


Data type in Ocaml: lists

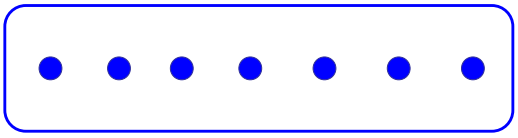
- Frequently used lists in recursive program
- Matched over two structural cases
  - [ ] - the empty list
  - (x :: xs) a non-empty list
- Covers all possible lists
- type 'a list = [ ] | (::) of 'a * 'a list
  - Not quite legitimate declaration because of special syntax

Variants - Syntax (slightly simplified)

- type *name* = C_1 [of ty_1] | . . . | C_n [of ty_n]
- Introduce a type called *name*
- (fun x -> C_i x) : ty_1 -> *name*
- C_i is called a *constructor*; if the optional type argument is omitted, it is called a *constant*
- Constructors are the basis of almost all pattern matching

Enumeration Types as Variants

An enumeration type is a collection of distinct values



In C and Ocaml they have an order structure; order by order of input

Enumeration Types as Variants

```
# type weekday = Monday | Tuesday | Wednesday
  | Thursday | Friday | Saturday | Sunday;;
type weekday =
    Monday
  | Tuesday
  | Wednesday
  | Thursday
  | Friday
  | Saturday
  | Sunday
```


Functions over Enumerations

```
# let day_after day = match day with
      Monday -> Tuesday
  | Tuesday -> Wednesday
  | Wednesday -> Thursday
  | Thursday -> Friday
  | Friday -> Saturday
  | Saturday -> Sunday
  | Sunday -> Monday;;
val day_after : weekday -> weekday = <fun>
```

Functions over Enumerations

```
# let rec days_later n day =
    match n with 0 -> day
    | _ -> if n > 0
         then day_after (days_later (n - 1) day)
        else days_later (n + 7) day;;
val days_later : int -> weekday -> weekday
  = <fun>
```


Functions over Enumerations

```
# days_later 2 Tuesday;;
- : weekday = Thursday
# days_later (-1) Wednesday;;
- : weekday = Tuesday
# days_later (-4) Monday;;
- : weekday = Thursday
```


Problem:

```
# type weekday = Monday | Tuesday |
  Wednesday
  | Thursday | Friday | Saturday | Sunday;;
```

- Write function is_weekend : weekday -> bool

```
let is_weekend day =
```


Problem:

```
# type weekday = Monday | Tuesday |
  Wednesday
  | Thursday | Friday | Saturday | Sunday;;
```

- Write function is_weekend : weekday -> bool

```
let is_weekend day =
    match day with Saturday -> true
     | Sunday -> true
     | _ -> false
```

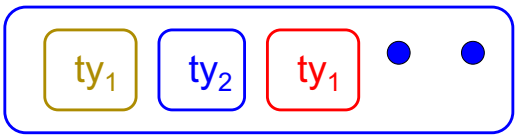

Example Enumeration Types

```
# type bin_op = IntPlusOp |  IntMinusOp
        |  EqOp | CommaOp | ConsOp

# type mon_op = HdOp | TlOp | FstOp
        | SndOp
```


Disjoint Union Types

- Disjoint union of types, with some possibly occurring more than once



- We can also add in some new singleton elements

Disjoint Union Types

```
# type id = DriversLicense of int
  | SocialSecurity of int | Name of string;;
type id = DriversLicense of int | SocialSecurity
  of int | Name of string
# let check_id id = match id with
      DriversLicense num ->
       not (List.mem num [13570; 99999])
   | SocialSecurity num -> num < 900000000
   | Name str -> not (str = "John Doe");;
val check_id : id -> bool = <fun>
```


Problem

- Create a type to represent the currencies for US, UK, Europe and Japan

Problem

- Create a type to represent the currencies for US, UK, Europe and Japan

```
type currency =
  Dollar of int
 | Pound of int
 | Euro of int
 | Yen of int
```


Example Disjoint Union Type

```
# type const =
     BoolConst of bool
   | IntConst of int
   | FloatConst of float
   | StringConst of string
   | NilConst
   | UnitConst
```


Example Disjoint Union Type

```
# type const = BoolConst of bool
   | IntConst of int | FloatConst of float
   | StringConst of string  | NilConst
   | UnitConst
```

- How to represent 7 as a const?
- Answer: IntConst 7

Polymorphism in Variants

- The type 'a option is gives us something to represent non-existence or failure

```
# type 'a option = Some of 'a | None;;
type 'a option = Some of 'a | None
```

- Used to encode partial functions
- Often can replace the raising of an exception

Functions producing option

```
# let rec first p list =
    match list with [ ] -> None
    | (x::xs) -> if p x then Some x else first p xs;;
val first : ('a -> bool) -> 'a list -> 'a option = <fun>
# first (fun x -> x > 3) [1;3;4;2;5];;
- : int option = Some 4
# first (fun x -> x > 5) [1;3;4;2;5];;
- : int option = None
```


Functions over option

```
# let result_ok r =
   match r with None -> false
   | Some _ -> true;;
val result_ok : 'a option -> bool = <fun>
# result_ok (first (fun x -> x > 3) [1;3;4;2;5]);;
- : bool = true
# result_ok (first (fun x -> x > 5) [1;3;4;2;5]);;
- : bool = false
```


Problem

- Write a hd and tl on lists that doesn't raise an exception and works at all types of lists.

Problem

- Write a hd and tl on lists that doesn't raise an exception and works at all types of lists.

```
- let hd list =
      match list with [] -> None
       | (x::xs) -> Some x
- let tl list =
      match list with [] -> None
       | (x::xs) -> Some xs
```


Mapping over Variants

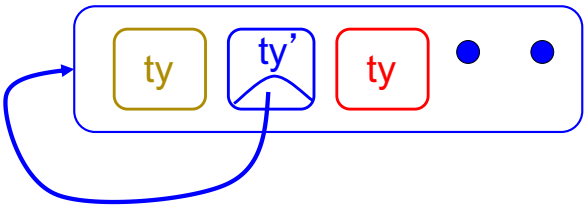
```
# let optionMap f opt =
    match opt with None -> None
    | Some x -> Some (f x);;
val optionMap : ('a -> 'b) -> 'a option -> 'b
  option = <fun>
# optionMap
  (fun x -> x - 2)
  (first (fun x -> x > 3) [1;3;4;2;5]);;
- : int option = Some 2
```


Folding over Variants

```
# let optionFold someFun noneVal opt =
    match opt with None -> noneVal
    | Some x -> someFun x;;
val optionFold : ('a -> 'b) -> 'b -> 'a option ->
  'b = <fun>
# let optionMap f opt =
    optionFold (fun x -> Some (f x)) None opt;;
val optionMap : ('a -> 'b) -> 'a option -> 'b
  option = <fun>
```

Recursive Types

- The type being defined may be a component of itself



Recursive Data Types

```
# type int_Bin_Tree =
 Leaf of int | Node of (int_Bin_Tree *
  int_Bin_Tree);;

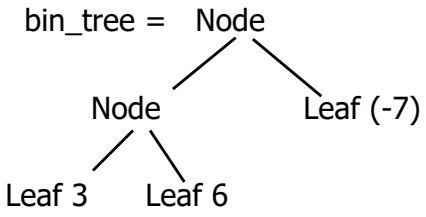
type int_Bin_Tree = Leaf of int | Node of
  (int_Bin_Tree * int_Bin_Tree)
```


Recursive Data Type Values

```
# let bin_tree =
 Node(Node(Leaf 3, Leaf 6),Leaf (-7));;

val bin_tree : int_Bin_Tree = Node (Node
  (Leaf 3, Leaf 6), Leaf (-7))
```


Recursive Data Type Values



Recursive Functions

```
# let rec first_leaf_value tree =
    match tree with (Leaf n) -> n
    | Node (left_tree, right_tree) ->
     first_leaf_value left_tree;;
val first_leaf_value : int_Bin_Tree -> int =
  <fun>
# let left = first_leaf_value bin_tree;;
val left : int = 3
```


Recursive Data Types

```
# type exp =
    VarExp of string
  | ConstExp of const
  | MonOpAppExp of mon_op * exp
  | BinOpAppExp of bin_op * exp * exp
  | IfExp of exp* exp * exp
  | AppExp of exp * exp
  | FunExp of string * exp
```

Recursive Data Types

```
# type bin_op = IntPlusOp |  IntMinusOp
        |  EqOp | CommaOp | ConsOp | …
# type const = BoolConst of bool | IntConst of int |
…
# type exp = VarExp of string | ConstExp of const
  | BinOpAppExp of bin_op * exp * exp  | …
```

- How to represent 6 as an exp?

Recursive Data Types

```
# type bin_op = IntPlusOp |  IntMinusOp
        |  EqOp | CommaOp | ConsOp | …
# type const = BoolConst of bool | IntConst of int |
…
# type exp = VarExp of string | ConstExp of const
  | BinOpAppExp of bin_op * exp * exp  | …
```

- How to represent 6 as an exp?
- Answer: ConstExp (IntConst 6)

Recursive Data Types

```
# type bin_op = IntPlusOp |  IntMinusOp
        |  EqOp | CommaOp | ConsOp | …
# type const = BoolConst of bool | IntConst of int |
…
# type exp = VarExp of string | ConstExp of const
  | BinOpAppExp of bin_op * exp * exp  | …
```

- How to represent (6, 3) as an exp?

Recursive Data Types

```
# type bin_op = IntPlusOp |  IntMinusOp
        |  EqOp | CommaOp | ConsOp | …
# type const = BoolConst of bool | IntConst of int |
…
# type exp = VarExp of string | ConstExp of const
  | BinOpAppExp of bin_op * exp * exp  | …
```

- How to represent (6, 3) as an exp?
- BinOpAppExp (CommaOp, ConstExp (IntConst 6),
  ConstExp (IntConst 3))

Recursive Data Types

```
# type bin_op = IntPlusOp |  IntMinusOp
        |  EqOp | CommaOp | ConsOp | …
# type const = BoolConst of bool | IntConst of int |
…
# type exp = VarExp of string | ConstExp of const
  | BinOpAppExp of bin_op * exp * exp  | …
```

- How to represent [(6, 3)] as an exp?
- BinOpAppExp (ConsOp, BinOpAppExp (CommaOp, ConstExp (IntConst 6), ConstExp (IntConst 3)), ConstExp NilConst))));;

Problem

```
type int_Bin_Tree =Leaf of int
| Node of (int_Bin_Tree * int_Bin_Tree);;
```

- Write sum_tree : int_Bin_Tree -> int
- Adds all ints in tree

```
let rec sum_tree t =
```

Problem

```
type int_Bin_Tree =Leaf of int
| Node of (int_Bin_Tree * int_Bin_Tree);;
```

- Write sum_tree : int_Bin_Tree -> int
- Adds all ints in tree

```
let rec sum_tree t =
    match t with Leaf n -> n
    | Node(t1,t2) -> sum_tree t1 + sum_tree t2
```


Recursion over Recursive Data Types

```
# type exp = VarExp of string | ConstExp of const
  | BinOpAppExp of bin_op * exp * exp
  | FunExp of string * exp | AppExp of exp * exp
```

- How to count the number of variables in an exp?

Recursion over Recursive Data Types

```
# type exp = VarExp of string | ConstExp of const
  | BinOpAppExp of bin_op * exp * exp
  | FunExp of string * exp | AppExp of exp * exp
```

- How to count the number of variables in an exp?

```
# let rec varCnt exp =
  match exp with VarExp x ->
   | ConstExp c ->
   | BinOpAppExp (b, e1, e2) ->
   | FunExp (x,e) ->
   | AppExp (e1, e2) ->
```


Recursion over Recursive Data Types

```
# type exp = VarExp of string | ConstExp of const
  | BinOpAppExp of bin_op * exp * exp
  | FunExp of string * exp | AppExp of exp * exp
```

- How to count the number of variables in an exp?

```
# let rec varCnt exp =
  match exp with VarExp x -> 1
   | ConstExp c -> 0
   | BinOpAppExp (b, e1, e2) -> varCnt e1 + varCnt e2
   | FunExp (x,e) -> 1 + varCnt e
   | AppExp (e1, e2) -> varCnt e1 + varCnt e2
```


Your turn now

Try Problem 3 on MP5

Mapping over Recursive Types

```
# let rec ibtreeMap f tree =
    match tree with (Leaf n) -> Leaf (f n)
    | Node (left_tree, right_tree) ->
     Node (ibtreeMap f left_tree,
           ibtreeMap f right_tree);;
val ibtreeMap : (int -> int) -> int_Bin_Tree ->
  int_Bin_Tree = <fun>
```

Mapping over Recursive Types

```
# ibtreeMap ((+) 2) bin_tree;;

- : int_Bin_Tree = Node (Node (Leaf 5, Leaf
  8), Leaf (-5))
```


Folding over Recursive Types

```
# let rec ibtreeFoldRight leafFun nodeFun tree =
    match tree with Leaf n -> leafFun n
    | Node (left_tree, right_tree) ->
      nodeFun
      (ibtreeFoldRight leafFun nodeFun left_tree)
      (ibtreeFoldRight leafFun nodeFun right_tree);;
val ibtreeFoldRight : (int -> 'a) -> ('a -> 'a -> 'a) ->
  int_Bin_Tree -> 'a = <fun>
```


Folding over Recursive Types

```
# let tree_sum =
    ibtreeFoldRight (fun x -> x) (+);;
val tree_sum : int_Bin_Tree -> int = <fun>
# tree_sum bin_tree;;
- : int = 2
```


600 minutes

Mutually Recursive Types

```
# type 'a tree = TreeLeaf of 'a
   | TreeNode of 'a treeList
and 'a treeList = Last of 'a tree
   | More of ('a tree * 'a treeList);;
type 'a tree = TreeLeaf of 'a | TreeNode of 'a
  treeList
and 'a treeList = Last of 'a tree | More of ('a
  tree * 'a treeList)
```

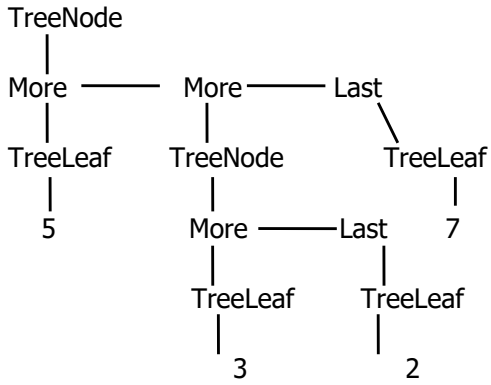

Mutually Recursive Types - Values

```
# let tree =
   TreeNode
    (More (TreeLeaf 5,
          (More (TreeNode
                (More (TreeLeaf 3,
                      Last (TreeLeaf 2))),
                Last (TreeLeaf 7)))));;
```

Mutually Recursive Types - Values

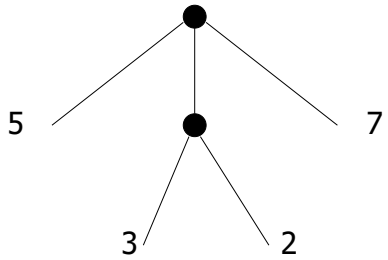
```
val tree : int tree =
 TreeNode
  (More
    (TreeLeaf 5,
     More
      (TreeNode (More (TreeLeaf 3, Last
 (TreeLeaf 2))), Last (TreeLeaf 7))))
```


Mutually Recursive Types - Values



Mutually Recursive Types - Values

A more conventional picture



Mutually Recursive Functions

```
# let rec fringe tree =
    match tree with (TreeLeaf x) -> [x]
  | (TreeNode list) -> list_fringe list
and list_fringe tree_list =
    match tree_list with (Last tree) -> fringe tree
  | (More (tree,list)) ->
    (fringe tree) @ (list_fringe list);;

val fringe : 'a tree -> 'a list = <fun>
val list_fringe : 'a treeList -> 'a list = <fun>
```


Mutually Recursive Functions

```
# fringe tree;;
- : int list = [5; 3; 2; 7]
```


Problem

```
# type 'a tree = TreeLeaf of 'a | TreeNode of 'a treeList
and 'a treeList = Last of 'a tree | More of ('a tree * 'a treeList);;
```

Define tree_size

Problem

```
# type 'a tree = TreeLeaf of 'a | TreeNode of 'a treeList
and 'a treeList = Last of 'a tree | More of ('a tree * 'a treeList);;
Define tree_size
let rec tree_size t =
      match t with TreeLeaf _ ->
      | TreeNode ts ->
```


Problem

```
# type 'a tree = TreeLeaf of 'a | TreeNode of 'a treeList
and 'a treeList = Last of 'a tree | More of ('a tree * 'a treeList);;
Define tree_size
let rec tree_size t =
      match t with TreeLeaf _ -> 1
      | TreeNode ts -> treeList_size  ts
```


Problem

```
# type 'a tree = TreeLeaf of 'a | TreeNode of 'a treeList
and 'a treeList = Last of 'a tree | More of ('a tree * 'a treeList);;
Define tree_size and treeList_size
let rec tree_size t =
      match t with TreeLeaf _ -> 1
      | TreeNode ts -> treeList_size  ts
and treeList_size ts =
```


Problem

```
# type 'a tree = TreeLeaf of 'a | TreeNode of 'a treeList
and 'a treeList = Last of 'a tree | More of ('a tree * 'a treeList);;
Define tree_size and treeList_size
let rec tree_size t =
      match t with TreeLeaf _ -> 1
      | TreeNode ts -> treeList_size  ts
and treeList_size ts =
      match ts with Last t ->
      | More t ts' ->
```


Problem

```
# type 'a tree = TreeLeaf of 'a | TreeNode of 'a treeList
and 'a treeList = Last of 'a tree | More of ('a tree * 'a treeList);;
Define tree_size and treeList_size
let rec tree_size t =
      match t with TreeLeaf _ -> 1
      | TreeNode ts -> treeList_size  ts
and treeList_size ts =
      match ts with Last t -> tree_size t
      | More t ts' -> tree_size t + treeList_size ts'
```


Problem

```
# type 'a tree = TreeLeaf of 'a | TreeNode of 'a treeList
and 'a treeList = Last of 'a tree | More of ('a tree * 'a treeList);;
Define tree_size and treeList_size
let rec tree_size t =
      match t with TreeLeaf _ -> 1
      | TreeNode ts -> treeList_size  ts
and treeList_size ts =
      match ts with Last t -> tree_size t
      | More t ts' -> tree_size t + treeList_size ts'
```

Nested Recursive Types

```
# type 'a labeled_tree =
 TreeNode of ('a * 'a labeled_tree
  list);;
type 'a labeled_tree = TreeNode of ('a
  * 'a labeled_tree list)
```

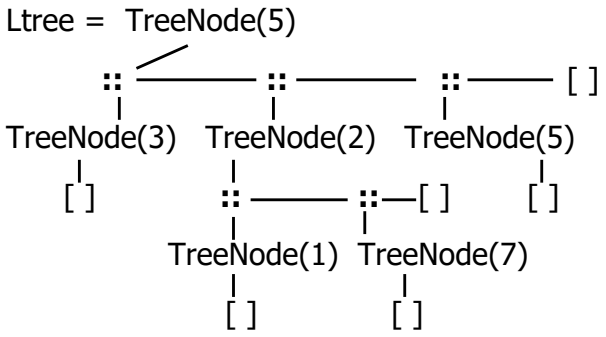

Nested Recursive Type Values

```
# let ltree =
  TreeNode(5,
    [TreeNode (3, []);
     TreeNode (2, [TreeNode (1, []);
                   TreeNode (7, [])]);
     TreeNode (5, [])]);;
```

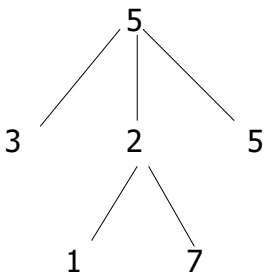

Nested Recursive Type Values

```
val ltree : int labeled_tree =
  TreeNode
   (5,
    [TreeNode (3, []); TreeNode (2,
  [TreeNode (1, []); TreeNode (7, [])]);
     TreeNode (5, [])])
```


Nested Recursive Type Values



Nested Recursive Type Values



Mutually Recursive Functions

```
# let rec flatten_tree labtree =
    match labtree with TreeNode (x,treelist)
      -> x::flatten_tree_list treelist
    and flatten_tree_list treelist =
    match treelist with [] -> []
    | labtree::labtrees
      -> flatten_tree labtree
        @ flatten_tree_list labtrees;;
```

Mutually Recursive Functions

```
val flatten_tree : 'a labeled_tree -> 'a list =
  <fun>
val flatten_tree_list : 'a labeled_tree list -> 'a
  list = <fun>
# flatten_tree ltree;;
- : int list = [5; 3; 2; 1; 7; 5]
```

- Nested recursive types lead to mutually recursive functions

Why Data Types?

- Data types play a key role in:
  - *Data abstraction* in the design of programs
  - *Type checking* in the analysis of programs
  - *Compile-time code generation* in the translation and execution of programs
    - Data layout (how many words; which are data and which are pointers) dictated by type

Terminology

- Type: A type *t* defines a set of possible data values
  - E.g. short in C is $\{x | 2^{15} - 1 \geq x \geq -2^{15}\}$
  - A value in this set is said to have type *t*
- Type system: rules of a language assigning types to expressions

Types as Specifications

- Types describe properties
- Different type systems describe different properties, eg
  - Data is read-write versus read-only
  - Operation has authority to access data
  - Data came from "right" source
  - Operation might or could not raise an exception
- Common type systems focus on types describing same data layout and access methods

Sound Type System

- If an expression is assigned type *t*, and it evaluates to a value *v*, then *v* is in the set of values defined by *t*
- SML, OCAML, Scheme and Ada have sound type systems
- Most implementations of C and C++ do not

Strongly Typed Language

- When no application of an operator to arguments can lead to a run-time type error, language is *strongly typed*
  - Eg: 1 + 2.3;;
- Depends on definition of "type error"

Strongly Typed Language

- C++ claimed to be "strongly typed", but
  - Union types allow creating a value at one type and using it at another
  - Type coercions may cause unexpected (undesirable) effects
  - No array bounds check (in fact, no runtime checks at all)
- SML, OCAML "strongly typed" but still must do dynamic array bounds checks, runtime type case analysis, and other checks

Static vs Dynamic Types

- *Static type*: type assigned to an expression at compile time
- *Dynamic type*: type assigned to a storage location at run time
- *Statically typed language*: static type assigned to every expression at compile time
- *Dynamically typed language*: type of an expression determined at run time

Type Checking

- When is op(arg1,…,argn) allowed?
- *Type checking* assures that operations are applied to the right number of arguments of the right types
  - Right type may mean same type as was specified, or may mean that there is a predefined implicit coercion that will be applied
- Used to resolve overloaded operations

Type Checking

- Type checking may be done *statically* at compile time or *dynamically* at run time
- Dynamically typed (aka untyped) languages (eg LISP, Prolog) do only dynamic type checking
- Statically typed languages can do most type checking statically

Dynamic Type Checking

- Performed at run-time before each operation is applied
- Types of variables and operations left unspecified until run-time
  - Same variable may be used at different types

Dynamic Type Checking

- Data object must contain type information
- Errors aren't detected until violating application is executed (maybe years after the code was written)

Static Type Checking

- Performed after parsing, before code generation
- Type of every variable and signature of every operator must be known at compile time

Static Type Checking

- Can eliminate need to store type information in data object if no dynamic type checking is needed
- Catches many programming errors at earliest point
- Can't check types that depend on dynamically computed values
  - Eg: array bounds

Static Type Checking

- Typically places restrictions on languages
  - Garbage collection
  - References instead of pointers
  - All variables initialized when created
  - Variable only used at one type
    - Union types allow for work-arounds, but effectively introduce dynamic type checks

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Nested Recursive Types

```
# type 'a labeled_tree =
  TreeNode of ('a * 'a labeled_tree
    list);;
type 'a labeled_tree = TreeNode of ('a
    * 'a labeled_tree list)
```

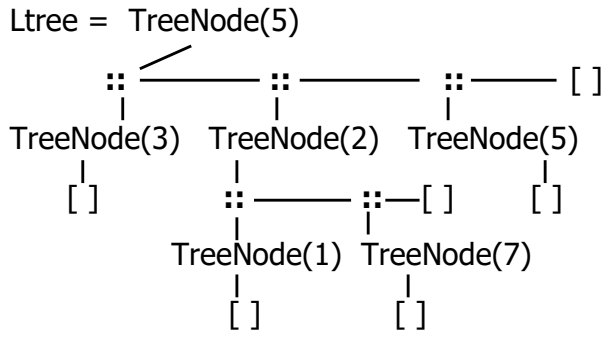

Nested Recursive Type Values

```
# let ltree =
  TreeNode(5,
    [TreeNode (3, []);
     TreeNode (2, [TreeNode (1, []);
                   TreeNode (7, [])]);
     TreeNode (5, [])]);;
```

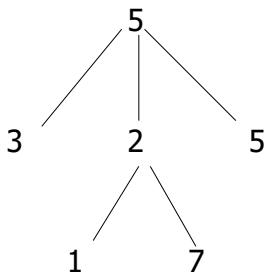

Nested Recursive Type Values

```
val ltree : int labeled_tree =
  TreeNode
   (5,
    [TreeNode (3, []); TreeNode (2,
   [TreeNode (1, []); TreeNode (7, [])]);
     TreeNode (5, [])])
```


Nested Recursive Type Values



Nested Recursive Type Values



Mutually Recursive Functions

```
# let rec flatten_tree labtree =
    match labtree with TreeNode (x,treelist)
      -> x::flatten_tree_list treelist
    and flatten_tree_list treelist =
    match treelist with [] -> []
    | labtree::labtrees
      -> flatten_tree labtree
        @ flatten_tree_list labtrees;;
```


Mutually Recursive Functions

```
val flatten_tree : 'a labeled_tree -> 'a list =
  <fun>
val flatten_tree_list : 'a labeled_tree list -> 'a
  list = <fun>
# flatten_tree ltree;;
- : int list = [5; 3; 2; 1; 7; 5]
```

- Nested recursive types lead to mutually recursive functions

Why Data Types?

- Data types play a key role in:
  - *Data abstraction* in the design of programs
  - *Type checking* in the analysis of programs
  - *Compile-time code generation* in the translation and execution of programs
    - Data layout (how many words; which are data and which are pointers) dictated by type

Terminology

- Type: A type *t* defines a set of possible data values
  - E.g. short in C is $\{x \mid 2^{15} - 1 \geq x \geq -2^{15}\}$
  - A value in this set is said to have type *t*
- Type system: rules of a language assigning types to expressions

Types as Specifications

- Types describe properties
- Different type systems describe different properties, eg
  - Data is read-write versus read-only
  - Operation has authority to access data
  - Data came from "right" source
  - Operation might or could not raise an exception
- Common type systems focus on types describing same data layout and access methods

Sound Type System

- If an expression is assigned type *t*, and it evaluates to a value *v*, then *v* is in the set of values defined by *t*
- SML, OCAML, Scheme and Ada have sound type systems
- Most implementations of C and C++ do not

Strongly Typed Language

- When no application of an operator to arguments can lead to a run-time type error, language is *strongly typed*
  - Eg: 1 + 2.3;;
- Depends on definition of "type error"

Strongly Typed Language

- C++ claimed to be "strongly typed", but
  - Union types allow creating a value at one type and using it at another
  - Type coercions may cause unexpected (undesirable) effects
  - No array bounds check (in fact, no runtime checks at all)
- SML, OCAML "strongly typed" but still must do dynamic array bounds checks, runtime type case analysis, and other checks

Static vs Dynamic Types

- *Static type*: type assigned to an expression at compile time
- *Dynamic type*: type assigned to a storage location at run time
- *Statically typed language*: static type assigned to every expression at compile time
- *Dynamically typed language*: type of an expression determined at run time

Type Checking

- When is op(arg1,…,argn) allowed?
- *Type checking* assures that operations are applied to the right number of arguments of the right types
  - Right type may mean same type as was specified, or may mean that there is a predefined implicit coercion that will be applied
- Used to resolve overloaded operations

Type Checking

- Type checking may be done *statically* at compile time or *dynamically* at run time
- Dynamically typed (aka untyped) languages (eg LISP, Prolog) do only dynamic type checking
- Statically typed languages can do most type checking statically

Dynamic Type Checking

- Performed at run-time before each operation is applied
- Types of variables and operations left unspecified until run-time
  - Same variable may be used at different types

Dynamic Type Checking

- Data object must contain type information
- Errors aren't detected until violating application is executed (maybe years after the code was written)

Static Type Checking

- Performed after parsing, before code generation
- Type of every variable and signature of every operator must be known at compile time

Static Type Checking

- Can eliminate need to store type information in data object if no dynamic type checking is needed
- Catches many programming errors at earliest point
- Can't check types that depend on dynamically computed values
  - Eg: array bounds

Static Type Checking

- Typically places restrictions on languages
  - Garbage collection
  - References instead of pointers
  - All variables initialized when created
  - Variable only used at one type
    - Union types allow for work-arounds, but effectively introduce dynamic type checks

Type Declarations

- *Type declarations*: explicit assignment of types to variables (signatures to functions) in the code of a program
  - Must be checked in a strongly typed language
  - Often not necessary for strong typing or even static typing (depends on the type system)

Type Inference

- *Type inference*: A program analysis to assign a type to an expression from the program context of the expression
  - Fully static type inference first introduced by Robin Miller in ML
  - Haskle, OCAML, SML all use type inference
    - Records are a problem for type inference

Format of Type Judgments

- A *type judgement* has the form

  Γ |- exp : τ

- Γ is a typing environment
  - Supplies the types of variables (and function names when function names are not variables)
  - Γ is a set of the form { x:σ , . . . }
  - For any x at most one σ such that (x : $\sigma \in \Gamma$)
- exp is a program expression
- τ is a type to be assigned to exp
- |- pronounced "turnstyle", or "entails" (or "satisfies" or, informally, "shows")

Axioms – Constants (Monomorphic)

$$\frac{}{\Gamma \text{ |- } n : \text{int}}$$ (assuming n is an integer constant)

$$\frac{}{\Gamma \text{ |- true : bool}} \qquad \frac{}{\Gamma \text{ |- false : bool}}$$

- These rules are true with any typing environment
- Γ, n are meta-variables

Axioms – Variables (Monomorphic Rule)

Notation: Let $\Gamma(x) = \sigma$ if $x : \sigma \in \Gamma$

Note: if such σ exits, its unique

Variable axiom:

$$\frac{}{\Gamma \text{ |- } x : \sigma} \quad \text{if } \Gamma(x) = \sigma$$

Simple Rules – Arithmetic (Mono)

Primitive Binary operators ($\oplus \in$ { +, -, *, …}):

$$\frac{\Gamma \text{ |- } e_1 : \tau_1 \quad \Gamma \text{ |- } e_2 : \tau_2 \quad (\oplus) : \tau_1 \rightarrow \tau_2 \rightarrow \tau_3}{\Gamma \text{ |- } e_1 \oplus e_2 : \tau_3}$$

Special case: Relations (~ $\in$ { < , > , =, <=, >= }):

$$\frac{\Gamma \text{ |- } e_1 : \tau \quad \Gamma \text{ |- } e_2 : \tau \quad (\sim) : \tau \rightarrow \tau \rightarrow \text{bool}}{\Gamma \text{ |- } e_1 \sim e_2 : \text{bool}}$$

For the moment, think τ is int

Example: {x:int} |- x + 2 = 3 :bool

What do we need to show first?

{x:int} |- x + 2 = 3 : bool

Example: {x:int} |- x + 2 = 3 :bool

What do we need for the left side?

$$\frac{\{x : \text{int}\} \text{ |- } x + 2 : \text{int} \qquad \{x:\text{int}\} \text{ |- } 3 : \text{int}}{\{x:\text{int}\} \text{ |- } x + 2 = 3 : \text{bool}}\ \text{Bin}$$

Example: {x:int} |- x + 2 = 3 :bool

How to finish?

$$\frac{\{x:int\} \vdash x:int \quad \{x:int\} \vdash 2:int}{\{x:int\} \vdash x + 2 : int}\text{Bin} \qquad \{x:int\} \vdash 3 :int}{\{x:int\} \vdash x + 2 = 3 : bool}\text{Bin}$$

Example: {x:int} |- x + 2 = 3 :bool

Complete Proof (type derivation)

$$\frac{\dfrac{\dfrac{}{\{x:int\} \vdash x:int}\text{Var} \quad \dfrac{}{\{x:int\} \vdash 2:int}\text{Const}}{\{x:int\} \vdash x + 2 : int}\text{Bin} \qquad \dfrac{}{\{x:int\} \vdash 3 :int}\text{Const}}{\{x:int\} \vdash x + 2 = 3 : bool}\text{Bin}$$

Simple Rules - Booleans

Connectives

$$\frac{\Gamma \vdash e_1 : bool \quad \Gamma \vdash e_2 : bool}{\Gamma \vdash e_1 \;\&\&\; e_2 : bool}$$

$$\frac{\Gamma \vdash e_1 : bool \quad \Gamma \vdash e_2 : bool}{\Gamma \vdash e_1 \;||\; e_2 : bool}$$

Type Variables in Rules

- If_then_else rule:

$$\frac{\Gamma \vdash e_1 : bool \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{if } e_1 \text{ then } e_2 \text{ else } e_3) : \tau}$$

- τ is a type variable (meta-variable)
- Can take any type at all
- All instances in a rule application must get same type
- Then branch, else branch and if_then_else must all have same type

Example derivation: if-then-else-

- Γ = {x:int, int_of_float:float -> int, y:float}

$$\frac{\Gamma \vdash (\text{fun y -> } y > 3)\ x : bool \quad \Gamma \vdash x+2 : int \quad \Gamma \vdash \text{int_of_float } y : int}{\Gamma \vdash \text{if (fun y -> y > 3) x then x + 2 else int_of_float y : int}}$$

Function Application

- Application rule:

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash (e_1\ e_2) : \tau_2}$$

- If you have a function expression e_1 of type $\tau_1 \to \tau_2$ applied to an argument e_2 of type τ_1, the resulting expression $e_1 e_2$ has type τ_2

750 minutes

Example: Application

- Γ = {x:int, int_of_float:float -> int, y:float}

$$\frac{\Gamma \text{ |- (fun y -> y > 3) : int -> bool} \qquad \Gamma \text{ |- x : int}}{\Gamma \text{ |- (fun y -> y > 3) x : bool}}$$

Fun Rule

- Rules describe types, but also how the environment Γ may change
- Can only do what rule allows!
- fun rule:

$$\frac{\{x : \tau_1\} + \Gamma \text{ |- } e : \tau_2}{\Gamma \text{ |- fun } x \text{ -> } e : \tau_1 \rightarrow \tau_2}$$

Fun Examples

$$\frac{\{y : int\} + \Gamma \text{ |- } y + 3 : int}{\Gamma \text{ |- fun y -> } y + 3 : int \rightarrow int}$$

$$\frac{\{f : int \rightarrow bool\} + \Gamma \text{ |- } f\ \ 2 :: [true] : bool\ list}{\Gamma \text{ |- (fun f -> (f 2) :: [true]) : (int} \rightarrow \text{bool)} \rightarrow \text{bool list}}$$

(Monomorphic) Let and Let Rec

- let rule:

$$\frac{\Gamma \text{ |- } e_1 : \tau_1 \qquad \{x : \tau_1\} + \Gamma \text{ |- } e_2 : \tau_2}{\Gamma \text{ |- (let x = } e_1 \text{ in } e_2 \text{)} : \tau_2}$$

- let rec rule:

$$\frac{\{x : \tau_1\} + \Gamma \text{ |- } e_1 : \tau_1 \qquad \{x : \tau_1\} + \Gamma \text{ |- } e_2 : \tau_2}{\Gamma \text{ |- (let rec x = } e_1 \text{ in } e_2 \text{)} : \tau_2}$$

Curry - Howard Isomorphism

- Type Systems are logics; logics are type systems
- Types are propositions; propositions are types
- Terms are proofs; proofs are terms
- Function space arrow corresponds to implication; application corresponds to modus ponens

Curry - Howard Isomorphism

- Modus Ponens

$$\frac{A \Rightarrow B \quad A}{B}$$

- Application

$$\frac{\Gamma \vdash e_1 : \alpha \to \beta \quad \Gamma \vdash e_2 : \alpha}{\Gamma \vdash (e_1\ e_2) : \beta}$$

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Type Inference

- *Type inference*: A program analysis to assign a type to an expression from the program context of the expression
  - Fully static type inference first introduced by Robin Miller in ML
  - Haskle, OCAML, SML all use type inference
    - Records are a problem for type inference

Format of Type Judgments

- A *type judgement* has the form

  Γ |- exp : τ
- Γ is a typing environment
  - Supplies the types of variables (and function names when function names are not variables)
  - Γ is a set of the form { x:σ , . . . }
  - For any x at most one σ such that (x : $\sigma \in \Gamma$)
- exp is a program expression
- τ is a type to be assigned to exp
- |- pronounced "turnstyle", or "entails" (or "satisfies" or, informally, "shows")

Axioms - Constants

$$\frac{}{\Gamma \text{ |- } n : \text{int}}$$ (assuming n is an integer constant)

$$\frac{}{\Gamma \text{ |- true : bool}} \qquad \frac{}{\Gamma \text{ |- false : bool}}$$

- These rules are true with any typing environment
- Γ, n are meta-variables

Axioms – Variables (Monomorphic Rule)

Notation: Let $\Gamma(x) = \sigma$ if $x : \sigma \in \Gamma$

Note: if such σ exits, its unique

Variable axiom:

$$\frac{}{\Gamma \text{ |- } x : \sigma} \quad \text{if } \Gamma(x) = \sigma$$

Simple Rules - Arithmetic

Primitive Binary operators ($\oplus \in \{ +, -, *, \ldots \}$):

$$\frac{\Gamma \text{ |- } e_1 : \tau_1 \quad \Gamma \text{ |- } e_2 : \tau_2 \quad (\oplus) : \tau_1 \to \tau_2 \to \tau_3}{\Gamma \text{ |- } e_1 \oplus e_2 : \tau_3}$$

Special case: Relations ($\sim \in \{ <, >, =, <=, >= \}$):

$$\frac{\Gamma \text{ |- } e_1 : \tau \quad \Gamma \text{ |- } e_2 : \tau \quad (\sim) : \tau \to \tau \to \text{bool}}{\Gamma \text{ |- } e_1 \sim e_2 : \text{bool}}$$

For the moment, think τ is int

Example: {x:int} |- x + 2 = 3 :bool

What do we need to show first?

$$\{x{:}int\} \vdash x + 2 = 3 : bool$$

Example: {x:int} |- x + 2 = 3 :bool

What do we need for the left side?

$$\frac{\{x : int\} \vdash x + 2 : int \qquad \{x{:}int\} \vdash 3 : int}{\{x{:}int\} \vdash x + 2 = 3 : bool}\ Bin$$

Example: {x:int} |- x + 2 = 3 :bool

How to finish?

$$\frac{\dfrac{\{x{:}int\} \vdash x{:}int \quad \{x{:}int\} \vdash 2{:}int}{\{x : int\} \vdash x + 2 : int}\ Bin \qquad \{x{:}int\} \vdash 3 : int}{\{x{:}int\} \vdash x + 2 = 3 : bool}\ Bin$$

Example: {x:int} |- x + 2 = 3 :bool

Complete Proof (type derivation)

$$\frac{\dfrac{\dfrac{}{\{x{:}int\} \vdash x{:}int}\ Var \quad \dfrac{}{\{x{:}int\} \vdash 2{:}int}\ Const}{\{x : int\} \vdash x + 2 : int}\ Bin \qquad \dfrac{}{\{x{:}int\} \vdash 3 : int}\ Const}{\{x{:}int\} \vdash x + 2 = 3 : bool}\ Bin$$

Simple Rules - Booleans

Connectives

$$\frac{\Gamma \vdash e_1 : bool \qquad \Gamma \vdash e_2 : bool}{\Gamma \vdash e_1 \,\&\&\, e_2 : bool}$$

$$\frac{\Gamma \vdash e_1 : bool \qquad \Gamma \vdash e_2 : bool}{\Gamma \vdash e_1 \,||\, e_2 : bool}$$

Type Variables in Rules

- If_then_else rule:

$$\frac{\Gamma \vdash e_1 : bool \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{if } e_1 \text{ then } e_2 \text{ else } e_3) : \tau}$$

- τ is a type variable (meta-variable)
- Can take any type at all
- All instances in a rule application must get same type
- Then branch, else branch and if_then_else must all have same type

Function Application

- Application rule:

$$\frac{\Gamma \text{ |- } e_1 : \tau_1 \to \tau_2 \quad \Gamma \text{ |- } e_2 : \tau_1}{\Gamma \text{ |- } (e_1\ e_2) : \tau_2}$$

- If you have a function expression e_1 of type $\tau_1 \to \tau_2$ applied to an argument e_2 of type τ_1, the resulting expression $e_1 e_2$ has type τ_2

Fun Rule

- Rules describe types, but also how the environment Γ may change
- Can only do what rule allows!
- fun rule:

$$\frac{\{x : \tau_1\} + \Gamma \text{ |- } e : \tau_2}{\Gamma \text{ |- fun } x \text{ -> } e : \tau_1 \to \tau_2}$$

Fun Examples

$$\frac{\{y : \text{int}\} + \Gamma \text{ |- } y + 3 : \text{int}}{\Gamma \text{ |- fun } y \text{ -> } y + 3 : \text{int} \to \text{int}}$$

$$\frac{\{f : \text{int} \to \text{bool}\} + \Gamma \text{ |- } f\ 2 :: [\text{true}] : \text{bool list}}{\Gamma \text{ |- (fun } f \text{ -> } (f\ 2) :: [\text{true}]) : (\text{int} \to \text{bool}) \to \text{bool list}}$$

(Monomorphic) Let and Let Rec

- let rule:

$$\frac{\Gamma \text{ |- } e_1 : \tau_1 \quad \{x : \tau_1\} + \Gamma \text{ |- } e_2 : \tau_2}{\Gamma \text{ |- (let x = } e_1 \text{ in } e_2) : \tau_2}$$

- let rec rule:

$$\frac{\{x : \tau_1\} + \Gamma \text{ |- } e_1 : \tau_1 \quad \{x : \tau_1\} + \Gamma \text{ |- } e_2 : \tau_2}{\Gamma \text{ |- (let rec x = } e_1 \text{ in } e_2) : \tau_2}$$

Example

- Which rule do we apply?

$$\frac{?}{\{\} \text{ |- (let rec one = 1 :: one in let x = 2 in fun y -> (x :: y :: one)) : int} \to \text{int list}}$$

Example

- Let rec rule:

$$\frac{\textcircled{1}\ \{\text{one : int list}\} \text{ |- } (1 :: \text{one}) : \text{int list} \qquad \textcircled{2}\ \{\text{one : int list}\} \text{ |- (let x = 2 in fun y -> (x :: y :: one)) : int} \to \text{int list}}{\{\} \text{ |- (let rec one = 1 :: one in let x = 2 in fun y -> (x :: y :: one)) : int} \to \text{int list}}$$

Proof of 1

- Which rule?

{one : int list} |- (1 :: one) : int list

Proof of 1

- Binary Operator

$$\frac{\text{③ } \{\text{one : int list}\} \text{ |- 1: int} \qquad \text{④ } \{\text{one : int list}\} \text{ |- one : int list}}{\{\text{one : int list}\} \text{ |- (1 :: one) : int list}}$$

where (::) : int → int list → int list

Proof of 1

$$\frac{\text{③ } \dfrac{}{\{\text{one : int list}\} \text{ |- 1: int}}\ \text{Constant Rule} \qquad \text{④ } \dfrac{}{\{\text{one : int list}\} \text{ |- one : int list}}\ \text{Variable Rule}}{\{\text{one : int list}\} \text{ |- (1 :: one) : int list}}$$

Proof of 2

- Let Rule

$$\frac{\{\text{one : int list}\} \text{ |- 2:int} \qquad \{\text{x:int; one : int list}\} \text{ |- fun y -> (x :: y :: one)} : \text{int} \to \text{int list}}{\{\text{one : int list}\} \text{ |- (let x = 2 in fun y -> (x :: y :: one)) : int} \to \text{int list}}$$

Proof of 2

- Constant

$$\frac{\text{⑤ } \{\text{x:int; one : int list}\} \text{ |- fun y -> (x :: y :: one)} : \text{int} \to \text{int list}}{\{\text{one : int list}\} \text{ |- (let x = 2 in fun y -> (x :: y :: one)) : int} \to \text{int list}}$$

Proof of 5

$$\frac{?}{\{\text{x:int; one : int list}\} \text{ |- fun y -> (x :: y :: one)} : \text{int} \to \text{int list}}$$

Proof of 5

?

{y:int; x:int; one : int list} |- (x :: y :: one) : int list

{x:int; one : int list} |- fun y -> (x :: y :: one))
: int → int list

By the Fun Rule

Proof of 5

⑥ ⑦

{y:int; x:int; one:int list} |- x:int

{y:int; x:int; one:int list} |- (y :: one) : int list

{y:int; x:int; one : int list} |- (x :: y :: one) : int list

{x:int; one : int list} |- fun y -> (x :: y :: one))
: int → int list

By BinOp where (::) : int → int list → int list

Proof of 6

⑥ ⑦

Variable Rule

{y:int; x:int; one:int list} |- x:int

{y:int; x:int; one:int list} |- (y :: one) : int list

{y:int; x:int; one : int list} |- (x :: y :: one) : int list

{x:int; one : int list} |- fun y -> (x :: y :: one))
: int → int list

Proof of 7

- Binary Operation Rule

{…; one:int list;…}

{y:int; …} |- y:int |- one : int list

{y:int; x:int; one : int list}|- (y :: one) : int list

By BinOp where (::) : int → int list → int list

Proof of 7

Variable Rule

{y:int; …} |- y:int

Variable Rule

{…; one:int list;…}

|- one : int list

{y:int; x:int; one : int list}|- (y :: one) : int list

Curry - Howard Isomorphism

- Type Systems are logics; logics are type systems
- Types are propositions; propositions are types
- Terms are proofs; proofs are terms
- Function space arrow corresponds to implication; application corresponds to modus ponens

Curry - Howard Isomorphism

- Modus Ponens

$$\frac{A \Rightarrow B \quad A}{B}$$

- Application

$$\frac{\Gamma \vdash e_1 : \alpha \rightarrow \beta \quad \Gamma \vdash e_2 : \alpha}{\Gamma \vdash (e_1\ e_2) : \beta}$$

Review: In Class Activity

ACT 4

Mea Culpa

- The above system can't handle polymorphism as in OCAML
- No type variables in type language (only meta-variable in the logic)
- Would need:
  - Object level type variables and some kind of type quantification
  - **let** and **let rec** rules to introduce polymorphism
  - Explicit rule to eliminate (instantiate) polymorphism

Support for Polymorphic Types

- Monomorpic Types (τ):
  - Basic Types: int, bool, float, string, unit, …
  - Type Variables: α, β, γ, δ, ε
  - Compound Types: $\alpha \rightarrow \beta$, int * string, bool list, …
- Polymorphic Types:
  - Monomorphic types τ
  - Universally quantified monomorphic types
  - $\forall \alpha_1, \ldots, \alpha_n \,.\, \tau$
  - Can think of τ as same as $\forall \,.\, \tau$

Example FreeVars Calculations

- Vars('a -> (int -> 'b) -> 'a) ={'a , 'b}
- FreeVars (All 'b. 'a -> (int -> 'b) -> 'a) =
- {'a , 'b} – {'b}= {'a}
- FreeVars {x : All 'b. 'a -> (int -> 'b) -> 'a,
- id: All 'c. 'c -> 'c,
- y: All 'c. 'a -> 'b -> 'c} =
- {'a} U {} U {'a, 'b} = {'a, 'b}

Support for Polymorphic Types

- Typing Environment Γ supplies polymorphic types (which will often just be monomorphic) for variables
- Free variables of monomorphic type just type variables that occur in it
  - Write FreeVars(τ)
- Free variables of polymorphic type removes variables that are universally quantified
  - FreeVars($\forall \alpha_1, \ldots, \alpha_n \,.\, \tau$) = FreeVars($\tau$) – $\{\alpha_1, \ldots, \alpha_n\}$
- FreeVars(Γ) = all FreeVars of types in range of Γ

Monomorphic to Polymorphic

- Given:
  - type environment Γ
  - monomorphic type τ
  - τ shares type variables with Γ
- Want most polymorphic type for τ that doesn't break sharing type variables with Γ
- $Gen(\tau, \Gamma) = \forall \alpha_1, \ldots, \alpha_n . \tau$ where $\{\alpha_1, \ldots, \alpha_n\} = freeVars(\tau) - freeVars(\Gamma)$

Polymorphic Typing Rules

- A *type judgement* has the form
  Γ |- exp : τ
  - Γ uses polymorphic types
  - τ still monomorphic
- Most rules stay same (except use more general typing environments)
- Rules that change:
  - Variables
  - Let and Let Rec
  - Allow polymorphic constants
- Worth noting functions again

Polymorphic Let and Let Rec

- let rule:

$$\frac{\Gamma \text{ |- } e_1 : \tau_1 \quad \{x : Gen(\tau_1, \Gamma)\} + \Gamma \text{ |- } e_2 : \tau_2}{\Gamma \text{ |- (let x = } e_1 \text{ in } e_2 \text{)} : \tau_2}$$

- let rec rule:

$$\frac{\{x : \tau_1\} + \Gamma \text{ |- } e_1 : \tau_1 \quad \{x : Gen(\tau_1, \Gamma)\} + \Gamma \text{ |- } e_2 : \tau_2}{\Gamma \text{ |- (let rec x = } e_1 \text{ in } e_2 \text{)} : \tau_2}$$

Polymorphic Variables (Identifiers)

Variable axiom:

$$\frac{}{\Gamma \text{ |- } x : \varphi(\tau)} \quad \text{if } \Gamma(x) = \forall \alpha_1, \ldots, \alpha_n . \tau$$

- Where φ replaces all occurrences of $\alpha_1, \ldots, \alpha_n$ by monotypes $\tau_1, \ldots, \tau_n$
- Note: Monomorphic rule special case:

$$\frac{}{\Gamma \text{ |- } x : \tau} \quad \text{if } \Gamma(x) = \tau$$

- Constants treated same way

Fun Rule Stays the Same

- fun rule:

$$\frac{\{x : \tau_1\} + \Gamma \text{ |- } e : \tau_2}{\Gamma \text{ |- fun } x \text{ -> } e : \tau_1 \rightarrow \tau_2}$$

- Types τ_1, τ_2 monomorphic
- Function argument must always be used at same type in function body

Polymorphic Example

- Assume additional constants and primitive operators:
- hd : $\forall\alpha.\ \alpha$ list -> α
- tl: $\forall\alpha.\ \alpha$ list -> α list
- is_empty : $\forall\alpha.\ \alpha$ list -> bool
- (::) : $\forall\alpha.\ \alpha$ -> α list -> α list
- [] : $\forall\alpha.\ \alpha$ list

Polymorphic Example

- Show:

?

{} |- let rec length =
fun l -> if is_empty l then 0
else 1 + length (tl l)
in length (2 :: []) + length(true :: []) : int

Polymorphic Example: Let Rec Rule

- Show: (1) (2)

{length:α list -> int} {length:∀α. α list -> int}
|- fun l -> … |- length (2 :: []) +
: α list -> int length(true :: []) : int

{} |- let rec length =
fun l -> if is_empty l then 0
else 1 + length (tl l)
in length (2 :: []) + length(true :: []) : int

Polymorphic Example (1)

- Show:

?

{length:α list -> int} |-
fun l -> if is_empty l then 0
else 1 + length (tl l)
: α list -> int

Polymorphic Example (1): Fun Rule

- Show: (3)

{length:α list -> int, l: α list } |-
if is_empty l then 0
else length (hd l) + length (tl l) : int

{length:α list -> int} |-
fun l -> if is_empty l then 0
else 1 + length (tl l)
: α list -> int

Polymorphic Example (3)

- Let Γ ={length:α list -> int, l: α list }
- Show

?

Γ|- if is_empty l then 0
else 1 + length (tl l) : int

Polymorphic Example (3):IfThenElse

- Let Γ ={length:α list -> int, l: α list }
- Show

(4) (5) (6)
Γ|- is_empty l Γ|- 0:int Γ|- 1 + length (tl l)
: bool : int

Γ|- if is_empty l then 0
else 1 + length (tl l) : int

Polymorphic Example (4)

- Let Γ ={length:α list -> int, l: α list }
- Show

$$\frac{?}{\Gamma \vdash \text{is_empty l} : \text{bool}}$$

Polymorphic Example (4):Application

- Let Γ ={length:α list -> int, l: α list }
- Show

$$\frac{\dfrac{?}{\Gamma \vdash \text{is_empty} : \alpha \text{ list} \to \text{bool}} \quad \dfrac{?}{\Gamma \vdash \text{l} : \alpha \text{ list}}}{\Gamma \vdash \text{is_empty l} : \text{bool}}$$

Polymorphic Example (4)

- Let Γ ={length:α list -> int, l: α list }
- Show

By Const since α list -> bool is instance of $\forall\alpha$. α list -> bool

$$\frac{\dfrac{}{\Gamma \vdash \text{is_empty} : \alpha \text{ list} \to \text{bool}} \quad \dfrac{?}{\Gamma \vdash \text{l} : \alpha \text{ list}}}{\Gamma \vdash \text{is_empty l} : \text{bool}}$$

Polymorphic Example (4)

- Let Γ ={length:α list -> int, l: α list }
- Show

By Const since α list -> bool is instance of $\forall\alpha$. α list -> bool

By Variable Γ(l) = α list

$$\frac{\dfrac{}{\Gamma \vdash \text{is_empty} : \alpha \text{ list} \to \text{bool}} \quad \dfrac{}{\Gamma \vdash \text{l} : \alpha \text{ list}}}{\Gamma \vdash \text{is_empty l} : \text{bool}}$$

- This finishes (4)

Polymorphic Example (5):Const

- Let Γ ={length:α list -> int, l: α list }
- Show

By Const Rule

$$\frac{}{\Gamma \vdash \text{0:int}}$$

Polymorphic Example (6):Arith Op

- Let Γ ={length:α list -> int, l: α list }
- Show

$$\frac{\dfrac{}{\Gamma \vdash \text{1:int}} \quad \dfrac{\dfrac{\text{By Variable}}{\Gamma \vdash \text{length} : \alpha \text{ list} \to \text{int}} \quad \dfrac{(7)}{\Gamma \vdash (\text{tl l}) : \alpha \text{ list}}}{\Gamma \vdash \text{length (tl l)} : \text{int}}}{\Gamma \vdash \text{1 + length (tl l)} : \text{int}}$$

Polymorphic Example (7):App Rule

- Let Γ ={length:α list -> int, l: α list }
- Show

$$\frac{\dfrac{}{\Gamma \vdash tl : \alpha\ list \rightarrow \alpha\ list}\ \text{By Const} \qquad \dfrac{}{\Gamma \vdash l : \alpha\ list}\ \text{By Variable}}{\Gamma \vdash (tl\ l) : \alpha\ list}$$

By Const since α list -> α list is instance of $\forall\alpha.\ \alpha$ list -> α list

Polymorphic Example: (2) by ArithOp

- Let Γ' = {length:$\forall\alpha.\ \alpha$ list -> int}
- Show:

$$\frac{\dfrac{(8)}{\Gamma' \vdash length\ (2 :: []) : int} \qquad \dfrac{(9)}{\Gamma' \vdash length(true :: []) : int}}{\{length:\forall\alpha.\ \alpha\ list \rightarrow int\} \vdash length\ (2 :: []) + length(true :: []) : int}$$

Polymorphic Example: (8)AppRule

- Let Γ' = {length:$\forall\alpha.\ \alpha$ list -> int}
- Show:

$$\frac{\Gamma' \vdash length : int\ list \rightarrow int \qquad \Gamma' \vdash (2 :: []) : int\ list}{\Gamma' \vdash length\ (2 :: []) : int}$$

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Two Problems

- Type checking
  - Question: Does exp. e have type τ in env Γ?
  - Answer: Yes / No
  - Method: Type derivation
- Typability
  - Question Does exp. e have some type in env. Γ? If so, what is it?
  - Answer: Type τ / error
  - Method: Type inference

Type Inference - Outline

- Begin by assigning a type variable as the type of the whole expression
- Decompose the expression into component expressions
- Use typing rules to generate constraints on components and whole
- Recursively find substitution that solves typing judgment of first subcomponent
- Apply substitution to next subcomponent and find substitution solving it; compose with first, etc.
- Apply comp of all substitution to orig. type var. to get answer

Type Inference - Example

- What type can we give to

  (fun x -> fun f -> f (f x))
- Start with a type variable and then look at the way the term is constructed

Type Inference - Example

- First approximate:

  $\{\ \}$ |- (fun x -> fun f -> f (f x)) : α
- Second approximate: use fun rule

$$\frac{\{x : \beta\} \text{ |- (fun f -> f (f x)) : } \gamma}{\{\ \} \text{ |- (fun x -> fun f -> f (f x)) : } \alpha}$$

- Remember constraint $\alpha \equiv (\beta \to \gamma)$

Type Inference - Example

- Third approximate: use fun rule

$$\frac{\dfrac{\{f : \delta\ ;\ x : \beta\} \text{ |- f (f x) : } \varepsilon}{\{x : \beta\} \text{ |- (fun f -> f (f x)) : } \gamma}}{\{\ \} \text{ |- (fun x -> fun f -> f(f x)) : } \alpha}$$

- $\alpha \equiv (\beta \to \gamma)$; $\gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Fourth approximate: use app rule

$$\dfrac{\dfrac{\dfrac{\{f{:}\delta;\ x{:}\beta\}\vdash f : \varphi \to \varepsilon \quad \{f{:}\delta;\ x{:}\beta\}\vdash f\ x : \varphi}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}}{\{x : \beta\} \vdash (\text{fun } f \text{ -> } f\ (f\ x)) : \gamma}}{\{\ \} \vdash (\text{fun } x \text{ -> fun } f \text{ -> } f\ (f\ x)) : \alpha}$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Fifth approximate: use var rule, get constraint $\delta \equiv \varphi \to \varepsilon$, Solve with same
- Apply to next sub-proof

$$\dfrac{\dfrac{\dfrac{\{f{:}\delta;\ x{:}\beta\}\vdash f : \varphi \to \varepsilon \quad \{f{:}\delta;\ x{:}\beta\}\vdash f\ x : \varphi}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}}{\{x : \beta\} \vdash (\text{fun } f \text{ -> } f\ (f\ x)) : \gamma}}{\{\ \} \vdash (\text{fun } x \text{ -> fun } f \text{ -> } f\ (f\ x)) : \alpha}$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst: $\{\delta \equiv \varphi \to \varepsilon\}$

$$\dfrac{\dfrac{\dfrac{\ldots \quad \{f{:}\varphi \to \varepsilon;\ x{:}\beta\}\vdash f\ x : \varphi}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}}{\{x : \beta\} \vdash (\text{fun } f \text{ -> } f\ (f\ x)) : \gamma}}{\{\ \} \vdash (\text{fun } x \text{ -> fun } f \text{ -> } f\ (f\ x)) : \alpha}$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst: $\{\delta \equiv \varphi \to \varepsilon\}$ Use App Rule

$$\dfrac{\dfrac{\dfrac{\{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash f{:}\zeta\to\varphi \quad \{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash x{:}\zeta}{\ldots \quad \{f{:}\varphi \to \varepsilon;\ x{:}\beta\}\vdash f\ x : \varphi}}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}}{\{x : \beta\} \vdash (\text{fun } f \text{ -> } f\ (f\ x)) : \gamma}$$
$$\{\ \} \vdash (\text{fun } x \text{ -> fun } f \text{ -> } f\ (f\ x)) : \alpha$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst: $\{\delta \equiv \varphi \to \varepsilon\}$
- Var rule: Solve $\zeta\to\varphi \equiv \varphi\to\varepsilon$ Unification

$$\dfrac{\dfrac{\dfrac{\{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash f{:}\zeta\to\varphi \quad \{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash x{:}\zeta}{\ldots \quad \{f{:}\varphi \to \varepsilon;\ x{:}\beta\}\vdash f\ x : \varphi}}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}}{\{x : \beta\} \vdash (\text{fun } f \text{ -> } f\ (f\ x)) : \gamma}$$
$$\{\ \} \vdash (\text{fun } x \text{ -> fun } f \text{ -> } f\ (f\ x)) : \alpha$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst: $\{\zeta \equiv \varepsilon,\ \varphi \equiv \varepsilon\}\ o\ \{\delta \equiv \varphi \to \varepsilon\}$
- Var rule: Solve $\zeta\to\varphi \equiv \varphi\to\varepsilon$ Unification

$$\dfrac{\dfrac{\dfrac{\{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash f{:}\zeta\to\varphi \quad \{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash x{:}\zeta}{\ldots \quad \{f{:}\varphi \to \varepsilon;\ x{:}\beta\}\vdash f\ x : \varphi}}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}}{\{x : \beta\} \vdash (\text{fun } f \text{ -> } f\ (f\ x)) : \gamma}$$
$$\{\ \} \vdash (\text{fun } x \text{ -> fun } f \text{ -> } f\ (f\ x)) : \alpha$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst: {ζ≡ε, φ≡ε, δ≡ε→ε}
- Apply to next sub-proof

$$\frac{\dots \qquad \{f{:}\varepsilon\to\varepsilon;\ x{:}\beta\}\vdash x{:}\varepsilon}{\dots \quad \{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash f\ x : \varphi}$$
$$\frac{}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}$$
$$\frac{}{\{x : \beta\} \vdash (\text{fun f -> f (f x)}) : \gamma}$$
$$\{\ \} \vdash (\text{fun x -> fun f -> f (f x)}) : \alpha$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst: {ζ≡ε, φ≡ε, δ≡ε→ε}
- Var rule: ε≡β

$$\frac{\dots \qquad \{f{:}\varepsilon\to\varepsilon;\ x{:}\beta\}\vdash x{:}\varepsilon}{\dots \quad \{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash f\ x : \varphi}$$
$$\frac{}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}$$
$$\frac{}{\{x : \beta\} \vdash (\text{fun f -> f (f x)}) : \gamma}$$
$$\{\ \} \vdash (\text{fun x -> fun f -> f (f x)}) : \alpha$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst: {ε≡β}o{ζ≡ε, φ≡ε, δ≡ε→ε}
- Solves subproof; return one layer

$$\frac{\dots \qquad \{f{:}\varepsilon\to\varepsilon;\ x{:}\beta\}\vdash x{:}\varepsilon}{\dots \quad \{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash f\ x : \varphi}$$
$$\frac{}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}$$
$$\frac{}{\{x : \beta\} \vdash (\text{fun f -> f (f x)}) : \gamma}$$
$$\{\ \} \vdash (\text{fun x -> fun f -> f (f x)}) : \alpha$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst: {ε≡β, ζ≡β, φ≡β, δ≡β→β}
- Solves this subproof; return one layer

$$\frac{\dots}{\dots \quad \{f{:}\varphi\to\varepsilon;\ x{:}\beta\}\vdash f\ x : \varphi}$$
$$\frac{}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}$$
$$\frac{}{\{x : \beta\} \vdash (\text{fun f -> f (f x)}) : \gamma}$$
$$\{\ \} \vdash (\text{fun x -> fun f -> f (f x)}) : \alpha$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst: {ε≡β, ζ≡β, φ≡β, δ≡β→β}
- Need to satisfy constraint $\gamma \equiv (\delta \to \varepsilon)$, given subst, becomes: $\gamma \equiv ((\beta\to\beta) \to \beta)$

$$\frac{\dots}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}$$
$$\frac{}{\{x : \beta\} \vdash (\text{fun f -> f (f x)}) : \gamma}$$
$$\{\ \} \vdash (\text{fun x -> fun f -> f (f x)}) : \alpha$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst:

{γ ≡ ((β→β) → β),ε≡β, ζ≡β, φ≡β, δ≡β→β}

- Solves subproof; return one layer

$$\frac{\dots}{\{f : \delta\ ;\ x : \beta\} \vdash (f\ (f\ x)) : \varepsilon}$$
$$\frac{}{\{x : \beta\} \vdash (\text{fun f -> f (f x)}) : \gamma}$$
$$\{\ \} \vdash (\text{fun x -> fun f -> f (f x)}) : \alpha$$

- $\alpha \equiv (\beta \to \gamma);\ \gamma \equiv (\delta \to \varepsilon)$

Type Inference - Example

- Current subst:

$\{\gamma \equiv ((\beta\to\beta) \to \beta), \varepsilon\equiv\beta, \zeta\equiv\beta, \varphi\equiv\beta, \delta\equiv\beta\to\beta\}$

- Need to satisfy constraint $\alpha \equiv (\beta \to \gamma)$ given subst: $\alpha \equiv (\beta \to ((\beta\to\beta) \to \beta))$

$$\dfrac{\dfrac{\ldots}{\{x : \beta\} \text{ |- } (\text{fun f -> f (f x)}) : \gamma}}{\{\ \} \text{ |- } (\text{fun x -> fun f -> f (f x)}) : \alpha}$$

- $\alpha \equiv (\beta \to \gamma)$;

Type Inference - Example

- Current subst:

$\{\alpha \equiv (\beta \to ((\beta\to\beta) \to \beta)),$
$\gamma \equiv ((\beta\to\beta) \to \beta), \varepsilon\equiv\beta, \zeta\equiv\beta, \varphi\equiv\beta, \delta\equiv\beta\to\beta\}$

- Solves subproof; return on layer

$$\dfrac{\dfrac{}{\{x : \beta\} \text{ |- } (\text{fun f -> f (f x)}) : \gamma}}{\{\ \} \text{ |- } (\text{fun x -> fun f -> f (f x)}) : \alpha}$$

Type Inference - Example

- Current subst:

$\{\alpha \equiv (\beta \to ((\beta\to\beta) \to \beta)),$
$\gamma \equiv ((\beta\to\beta) \to \beta), \varepsilon\equiv\beta, \zeta\equiv\beta, \varphi\equiv\beta, \delta\equiv\beta\to\beta\}$

- Done: $\alpha \equiv (\beta \to ((\beta\to\beta) \to \beta))$

$\{\ \} \text{ |- } (\text{fun x -> fun f -> f (f x)}) : \alpha$

Type Inference Algorithm

Let infer $(\Gamma, e, \tau) = \sigma$

- Γ is a typing environment (giving polymorphic types to expression variables)
- e is an expression
- τ is a type (with type variables),
- σ is a substitution of types for type variables
- Idea: σ is substitution solving the constraints on type variables necessary for $\Gamma \text{ |- } e : \tau$
- Should have $\sigma(\Gamma) \text{ |- } e : \sigma(\tau)$ valid

Type Inference Algorithm

infer (Γ, exp, τ) =

- Case *exp* of
  - Var *v* --> return Unify$\{\tau \equiv$ freshInstance$(\Gamma(v))\}$
    - Replace all quantified type vars by fresh ones
  - Const *c* --> return Unify$\{\tau \equiv$ freshInstance $\varphi\ \}$ where $\Gamma \text{ |- } c : \varphi$ by the constant rules
  - fun *x* -> *e* -->
    - Let α, β be fresh variables
    - Let σ = infer $(\{x: \alpha\} + \Gamma, e, \beta)$
    - Return Unify$(\{\sigma(\tau) \equiv \sigma(\alpha \to \beta)\}) \circ \sigma$

Example of inference with Var Rule

Instance {'a -> 'w} ('w a fresh variable)

{x: All 'a. ('a * 'b) list, y:All. 'b}|- x : (int * string) list

freshInstance(All 'a. ('a * 'b) list) = ('w * 'b) list
Unify {((int*string)list = ('w * 'b) list)} ={'w -> int, 'b -> string}

After substitution:
Instance {'a -> int}

{x:All 'a. ('a * string) list, y:All. string}|- x:(int * string) list

Type Inference Algorithm (cont)

- Case *exp* of
  - App (e_1 e_2) -->
    - Let α be a fresh variable
    - Let σ_1 = infer(Γ, e_1, $\alpha \rightarrow \tau$)
    - Let σ_2 = infer($\sigma_1(\Gamma)$, e_2, $\sigma_1(\alpha)$)
    - Return $\sigma_2 \circ \sigma_1$

Type Inference Algorithm (cont)

- Case *exp* of
  - If e_1 then e_2 else e_3 -->
    - Let σ_1 = infer(Γ, e_1, bool)
    - Let σ_2 = infer($\sigma_1\Gamma$, e_2, $\sigma_1(\tau)$)
    - Let σ_3 = infer($\sigma_2 \circ \sigma_1(\Gamma)$,$e_3$,$\sigma_2 \circ \sigma_1(\tau)$)
    - Return $\sigma_3 \circ \sigma_2 \circ \sigma_1$

Type Inference Algorithm (cont)

- Case *exp* of
  - let $x = e_1$ in e_2 -->
    - Let α be a fresh variable
    - Let σ_1 = infer(Γ, e_1, α)
    - Let σ_2 = infer({x:GEN($\sigma_1(\Gamma)$, $\sigma_1(\alpha)$)} + $\sigma_1(\Gamma)$, e_2, $\sigma_1(\tau)$)
    - Return $\sigma_2 \circ \sigma_1$

Type Inference Algorithm (cont)

- Case *exp* of
  - let rec $x = e_1$ in e_2 -->
    - Let α be a fresh variable
    - Let σ_1 = infer({x: α} + Γ, e_1, α)
    - Let σ_2 = infer({x:GEN($\sigma_1(\Gamma)$,$\sigma_1(\alpha)$)} + $\sigma_1(\Gamma)$}, e_2, $\sigma_1(\tau)$)
    - Return $\sigma_2 \circ \sigma_1$

Type Inference Algorithm (cont)

- To infer a type, introduce type_of
- Let α be a fresh variable
- type_of (Γ, *e*) =
  - Let σ = infer (Γ, *e*, α)
  - Return σ (α)
- Need an algorithm for Unif

Background for Unification

- Terms made from constructors and variables (for the simple first order case)
- Constructors may be applied to arguments (other terms) to make new terms
- Variables and constructors with no arguments are base cases
- Constructors applied to different number of arguments (arity) considered different
- Substitution of terms for variables

Simple Implementation Background

```
type term = Variable of string
          | Const of (string * term list)
let x = Variable "'a";;   let tm = Const ("2",[]);;

let rec subst var_name residue term =
    match term with Variable name ->
        if var_name = name then residue else term
      | Const (c, tys) ->
        Const (c, List.map (subst var_name residue)
                           tys);;
```


Unification Problem

Given a set of pairs of terms ("equations")

$$\{(s_1, t_1), (s_2, t_2), \ldots, (s_n, t_n)\}$$

(the *unification problem*) does there exist a substitution σ (the *unification solution*) of terms for variables such that

$$\sigma(s_i) = \sigma(t_i),$$

for all $i = 1, \ldots, n$?

Uses for Unification

- Type Inference and type checking
- Pattern matching as in OCaml
  - Can use a simplified version of algorithm
- Logic Programming - Prolog
- Simple parsing

Unification Algorithm

- Let $S = \{(s_1 = t_1), (s_2 = t_2), \ldots, (s_n = t_n)\}$ be a unification problem.
- Case $S = \{\}$: $Unif(S)$ = Identity function (i.e., no substitution)
- Case $S = \{(s, t)\} \cup S'$: Four main steps

Unification Algorithm

- Delete: if $s = t$ (they are the same term) then $Unif(S) = Unif(S')$
- Decompose: if $s = f(q_1, \ldots, q_m)$ and $t = f(r_1, \ldots, r_m)$ (same f, same m!), then $Unif(S) = Unif(\{(q_1, r_1), \ldots, (q_m, r_m)\} \cup S')$
- Orient: if $t = x$ is a variable, and s is not a variable, $Unif(S) = Unif(\{(x = s)\} \cup S')$

Unification Algorithm

- Eliminate: if $s = x$ is a variable, and x does not occur in t (the occurs check), then
  - Let $\varphi = \{x \rightarrow t\}$
    - $Unif(S) = Unif(\varphi(S')) \circ \{x \rightarrow t\}$
  - Let $\psi = Unif(\varphi(S'))$
  - $Unif(S) = \{x \rightarrow \psi(t)\} \circ \psi$
    - Note: $\{x \rightarrow a\} \circ \{y \rightarrow b\} = \{y \rightarrow (\{x \rightarrow a\}(b))\} \circ \{x \rightarrow a\}$ if y not in a

Tricks for Efficient Unification

- Don't return substitution, rather do it incrementally
- Make substitution be constant time
  - Requires implementation of terms to use mutable structures (or possibly lazy structures)
  - We won't discuss these

Example

- x,y,z variables, f,g constructors
- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} = ?

Example

- x,y,z variables, f,g constructors
- S = {(f(x) = f(g(f(z),y))), (g(y,y) = x)} is nonempty
- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (g(y,y) = x)
- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (g(y,y)) = x)
- Orient: (x = g(y,y))
- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} =
  Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))}

by Orient

Example

- x,y,z variables, f,g constructors
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} = ?

Example

- x,y,z variables, f,g constructors
- {(f(x) = f(g(f(z),y))), (x = g(y,y))} is non-empty
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (x = g(y,y))
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (x = g(y,y))
- Eliminate x with substitution {x→ g(y,y)}
  - Check: x not in g(y,y)
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (x = g(y,y))
- Eliminate x with substitution {x→ g(y,y)}
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} =
  Unify {(f(g(y,y)) = f(g(f(z),y)))}
  o {x→ g(y,y)}

Example

- x,y,z variables, f,g constructors
- Unify {(f(g(y,y)) = f(g(f(z),y)))}
  o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- {(f(g(y,y)) = f(g(f(z),y)))} is non-empty
- Unify {(f(g(y,y)) = f(g(f(z),y)))}
  o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(g(y,y)) = f(g(f(z),y)))

- Unify {(f(g(y,y)) = f(g(f(z),y)))}
  o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(g(y,y)) = f(g(f(z),y)))
- Decompose:(f(g(y,y)) = f(g(f(z),y))) becomes {(g(y,y) = g(f(z),y))}

- Unify {(f(g(y,y)) = f(g(f(z),y)))}
  o {x→ g(y,y)} =
  Unify {(g(y,y) = g(f(z),y))} o {x→ g(y,y)}

Example

- x,y,z variables, f,g constructors
- {(g(y,y) = g(f(z),y))} is non-empty

- Unify {(g(y,y) = g(f(z),y))}
  o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (g(y,y) = g(f(z),y))

- Unify {(g(y,y) = g(f(z),y))}
  o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(g(y,y)) = f(g(f(z),y)))
- Decompose: (g(y,y)) = g(f(z),y)) becomes {(y = f(z)); (y = y)}

- Unify {(g(y,y) = g(f(z),y))} o {x→ g(y,y)} =
  Unify {(y = f(z)); (y = y)} o {x→ g(y,y)}

Example

- x,y,z variables, f,g constructors

- Unify {(y = f(z)); (y = y)} o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- {(y = f(z)); (y = y)} o {x→ g(y,y) is non-empty
- Unify {(y = f(z)); (y = y)} o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (y = f(z))
- Unify {(y = f(z)); (y = y)} o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (y = f(z))
- Eliminate y with {y → f(z)}
- Unify {(y = f(z)); (y = y)} o {x→ g(y,y)} =
  Unify {(f(z) = f(z))}
  o ({y → f(z)} o {x→ g(y,y)})=
  Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))}

Example

- x,y,z variables, f,g constructors
- Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))} = ?

Example

- x,y,z variables, f,g constructors
- {(f(z) = f(z))} is non-empty
- Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(z) = f(z))
- Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(z) = f(z))
- Delete
- Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))} =
  Unify {} o {y → f(z); x→ g(f(z), f(z))}

Example

- x,y,z variables, f,g constructors
- Unify {} o {y → f(z); x→ g(f(z), f(z))} = ?

Example

- x,y,z variables, f,g constructors
- {} is empty
- Unify {} = identity function
- Unify {} o {y → f(z); x→ g(f(z), f(z))} =
  {y → f(z); x→ g(f(z), f(z))}

Example

- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} =
  {y → f(z); x→ g(f(z), f(z))}

f(x) = f(g(f(z), y))
→ f(g(f(z), f(z))) = f(g(f(z), f(z)))

g(y , y) = x
→ g(f(z),f(z)) = g(f(z), f(z))

Example of Failure: Decompose

- Unify{(f(x,g(y)) = f(h(y),x))}
- Decompose: (f(x,g(y)) = f(h(y),x))
- = Unify {(x = h(y)), (g(y) = x)}
- Orient: (g(y) = x)
- = Unify {(x = h(y)), (x = g(y))}
- Eliminate: (x = h(y))
- Unify {(h(y) = g(y))} o {x → h(y)}
- No rule to apply! Decompose fails!

Example of Failure: Occurs Check

- Unify{(f(x,g(x)) = f(h(x),x))}
- Decompose: (f(x,g(x)) = f(h(x),x))
- = Unify {(x = h(x)), (g(x) = x)}
- Orient: (g(x) = x)
- = Unify {(x = h(x)), (x = g(x))}
- No rules apply.

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Unification Problem

Given a set of pairs of terms ("equations")

$$\{(s_1, t_1), (s_2, t_2), \ldots, (s_n, t_n)\}$$

(the *unification problem*) does there exist a substitution σ (the *unification solution*) of terms for variables such that

$$\sigma(s_i) = \sigma(t_i),$$

for all $i = 1, \ldots, n$?

Uses for Unification

- Type Inference and type checking
- Pattern matching as in OCaml
  - Can use a simplified version of algorithm
- Logic Programming - Prolog
- Simple parsing

Unification Algorithm

- Let $S = \{(s_1 = t_1), (s_2 = t_2), \ldots, (s_n = t_n)\}$ be a unification problem.
- Case $S = \{\}$: Unif(S) = Identity function (i.e., no substitution)
- Case $S = \{(s, t)\} \cup S'$: Four main steps

Unification Algorithm

- Delete: if $s = t$ (they are the same term) then $Unif(S) = Unif(S')$
- Decompose: if $s = f(q_1, \ldots, q_m)$ and $t = f(r_1, \ldots, r_m)$ (same f, same m!), then $Unif(S) = Unif(\{(q_1, r_1), \ldots, (q_m, r_m)\} \cup S')$
- Orient: if $t = x$ is a variable, and s is not a variable, $Unif(S) = Unif(\{(x = s)\} \cup S')$

Unification Algorithm

- Eliminate: if $s = x$ is a variable, and x does not occur in t (the occurs check), then
  - Let $\varphi = \{x \to t\}$
    - $Unif(S) = Unif(\varphi(S')) \circ \{x \to t\}$
  - Let $\psi = Unif(\varphi(S'))$
  - $Unif(S) = \{x \to \psi(t)\} \circ \psi$
    - Note: $\{x \to a\} \circ \{y \to b\} = \{y \to (\{x \to a\}(b))\} \circ \{x \to a\}$ if y not in a

Tricks for Efficient Unification

- Don't return substitution, rather do it incrementally
- Make substitution be constant time
  - Requires implementation of terms to use mutable structures (or possibly lazy structures)
  - We won't discuss these

Example

- x,y,z variables, f,g constructors
- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} = ?

Example

- x,y,z variables, f,g constructors
- S = {(f(x) = f(g(f(z),y))), (g(y,y) = x)} is nonempty
- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (g(y,y) = x)
- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (g(y,y)) = x)
- Orient: (x = g(y,y))
- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} =
  Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))}

by Orient

Example

- x,y,z variables, f,g constructors
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} = ?

Example

- x,y,z variables, f,g constructors
- {(f(x) = f(g(f(z),y))), (x = g(y,y))} is non-empty
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (x = g(y,y))
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (x = g(y,y))
- Eliminate x with substitution {x→ g(y,y)}
  - Check: x not in g(y,y)
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (x = g(y,y))
- Eliminate x with substitution {x→ g(y,y)}
- Unify {(f(x) = f(g(f(z),y))), (x = g(y,y))} =
  Unify {(f(g(y,y)) = f(g(f(z),y)))}
  o {x→ g(y,y)}

Example

- x,y,z variables, f,g constructors
- Unify {(f(g(y,y)) = f(g(f(z),y)))}
  o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- {(f(g(y,y)) = f(g(f(z),y)))} is non-empty
- Unify {(f(g(y,y)) = f(g(f(z),y)))}
  o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(g(y,y)) = f(g(f(z),y)))
- Unify {(f(g(y,y)) = f(g(f(z),y)))} o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(g(y,y)) = f(g(f(z),y)))
- Decompose:(f(g(y,y)) = f(g(f(z),y))) becomes {(g(y,y) = g(f(z),y))}
- Unify {(f(g(y,y)) = f(g(f(z),y)))} o {x→ g(y,y)} = Unify {(g(y,y) = g(f(z),y))} o {x→ g(y,y)}

Example

- x,y,z variables, f,g constructors
- {(g(y,y) = g(f(z),y))} is non-empty
- Unify {(g(y,y) = g(f(z),y))} o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (g(y,y) = g(f(z),y))
- Unify {(g(y,y) = g(f(z),y))} o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(g(y,y)) = f(g(f(z),y)))
- Decompose: (g(y,y)) = g(f(z),y)) becomes {(y = f(z)); (y = y)}
- Unify {(g(y,y) = g(f(z),y))} o {x→ g(y,y)} = Unify {(y = f(z)); (y = y)} o {x→ g(y,y)}

Example

- x,y,z variables, f,g constructors
- Unify {(y = f(z)); (y = y)} o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- {(y = f(z)); (y = y)} o {x→ g(y,y) is non-empty
- Unify {(y = f(z)); (y = y)} o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (y = f(z))
- Unify {(y = f(z)); (y = y)} o {x→ g(y,y)} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (y = f(z))
- Eliminate y with {y → f(z)}
- Unify {(y = f(z)); (y = y)} o {x→ g(y,y)} =
  Unify {(f(z) = f(z))}
  o ({y → f(z)} o {x→ g(y,y)})=
  Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))}

Example

- x,y,z variables, f,g constructors
- Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))} = ?

Example

- x,y,z variables, f,g constructors
- {(f(z) = f(z))} is non-empty
- Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(z) = f(z))
- Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))} = ?

Example

- x,y,z variables, f,g constructors
- Pick a pair: (f(z) = f(z))
- Delete
- Unify {(f(z) = f(z))}
  o {y → f(z); x→ g(f(z), f(z))} =
  Unify {} o {y → f(z); x→ g(f(z), f(z))}

Example

- x,y,z variables, f,g constructors
- Unify {} o {y → f(z); x→ g(f(z), f(z))} = ?

Example

- x,y,z variables, f,g constructors
- {} is empty
- Unify {} = identity function
- Unify {} o {y → f(z); x→ g(f(z), f(z))} =
  {y → f(z); x→ g(f(z), f(z))}

Example

- Unify {(f(x) = f(g(f(z),y))), (g(y,y) = x)} =
  {y → f(z); x→ g(f(z), f(z))}

f(x) = f(g(f(z), y))
→ f(g(f(z), f(z))) = f(g(f(z), f(z)))

g(y , y) = x
→ g(f(z),f(z)) = g(f(z), f(z))

Example of Failure: Decompose

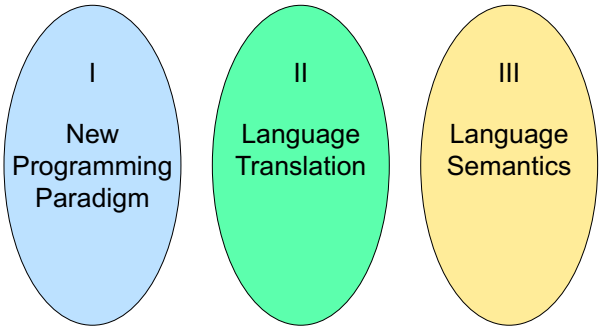
- Unify{(f(x,g(y)) = f(h(y),x))}
- Decompose: (f(x,g(y)) = f(h(y),x))
- = Unify {(x = h(y)), (g(y) = x)}
- Orient: (g(y) = x)
- = Unify {(x = h(y)), (x = g(y))}
- Eliminate: (x = h(y))
- Unify {(h(y) = g(y))} o {x → h(y)}
- No rule to apply! Decompose fails!

Example of Failure: Occurs Check

- Unify{(f(x,g(x)) = f(h(x),x))}
- Decompose: (f(x,g(x)) = f(h(x),x))
- = Unify {(x = h(x)), (g(x) = x)}
- Orient: (g(x) = x)
- = Unify {(x = h(x)), (x = g(x))}
- No rules apply.

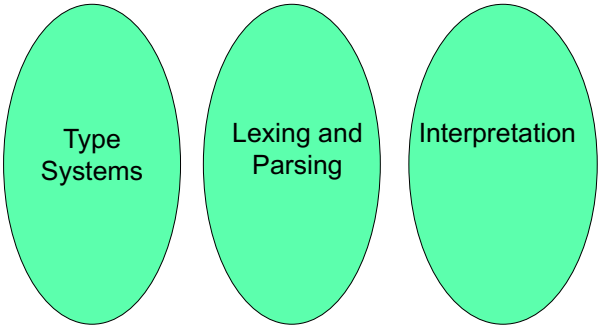
Programming Languages & Compilers

Three Main Topics of the Course

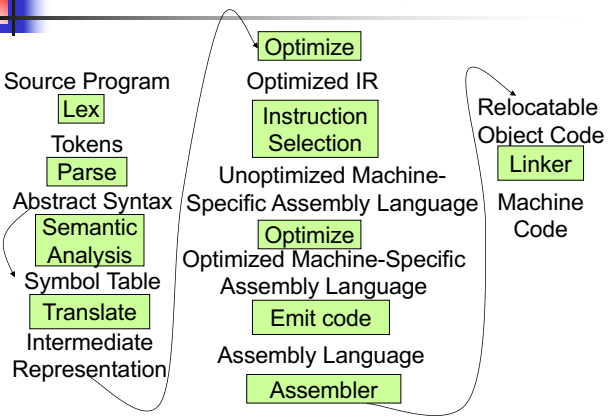


Programming Languages & Compilers

II : Language Translation



Major Phases of a Compiler



Modified from "Modern Compiler Implementation in ML", by Andrew Appel

Where We Are Going Next?

- We want to turn strings (code) into computer instructions
- Done in phases
- Turn strings into abstract syntax trees (parse)
- Translate abstract syntax trees into executable instructions (interpret or compile)

Meta-discourse

- Language Syntax and Semantics
- Syntax
  - Regular Expressions, DFSAs and NDFSAs
  - Grammars
- Semantics
  - Natural Semantics
  - Transition Semantics

Language Syntax

- Syntax is the description of which strings of symbols are meaningful expressions in a language
- It takes more than syntax to understand a language; need meaning (semantics) too
- Syntax is the entry point

Syntax of English Language

- Pattern 1

Subject	Verb
David	*sings*
The dog	*barked*
Susan	*yawned*

- Pattern 2

Subject	Verb	Direct Object
David	*sings*	*ballads*
The professor	*wants*	*to retire*
The jury	*found*	*the defendant guilty*

Elements of Syntax

- Character set – previously always ASCII, now often 64 character sets
- Keywords – usually reserved
- Special constants – cannot be assigned to
- Identifiers – can be assigned to
- Operator symbols
- Delimiters (parenthesis, braces, brackets)
- Blanks (aka white space)

Elements of Syntax

- Expressions
  if ... then begin ... ; ... end else begin ... ; ... end
- Type expressions
  typexpr$_1$ -> *typexpr*$_2$
- Declarations (in functional languages)
  let *pattern* = *expr*
- Statements (in imperative languages)
  a = b + c
- Subprograms
  let *pattern*$_1$ = *expr*$_1$ in *expr*

Elements of Syntax

- Modules
- Interfaces
- Classes (for object-oriented languages)

Lexing and Parsing

- Converting strings to abstract syntax trees done in two phases
  - **Lexing:** Converting string (or streams of characters) into lists (or streams) of tokens (the "words" of the language)
    - Specification Technique: Regular Expressions
  - **Parsing:** Convert a list of tokens into an abstract syntax tree
    - Specification Technique: BNF Grammars

Formal Language Descriptions

- Regular expressions, regular grammars, finite state automata
- Context-free grammars, BNF grammars, syntax diagrams
- Whole family more of grammars and automata – covered in automata theory

Grammars

- Grammars are formal descriptions of which strings over a given character set are in a particular language
- Language designers write grammar
- Language implementers use grammar to know what programs to accept
- Language users use grammar to know how to write legitimate programs

Regular Expressions - Review

- Start with a given character set – **a, b, c**…
- $\mathcal{L}(\varepsilon) = \{\text{“”}\}$
- Each character is a regular expression
  - It represents the set of one string containing just that character
  - $\mathcal{L}(\mathbf{a}) = \{a\}$

Regular Expressions

- If **x** and **y** are regular expressions, then **xy** is a regular expression
  - It represents the set of all strings made from first a string described by **x** then a string described by **y**

If $\mathcal{L}(x)=\{a,ab\}$ and $\mathcal{L}(y)=\{c,d\}$
then $\mathcal{L}(xy)=\{ac,ad,abc,abd\}$

Regular Expressions

- If **x** and **y** are regular expressions, then **x∨y** is a regular expression
  - It represents the set of strings described by either **x** or **y**

If $\mathcal{L}(x)=\{a,ab\}$ and $\mathcal{L}(y)=\{c,d\}$
then $\mathcal{L}(x \vee y)=\{a,ab,c,d\}$

Regular Expressions

- If **x** is a regular expression, then so is **(x)**
  - It represents the same thing as **x**
- If **x** is a regular expression, then so is **x***
  - It represents strings made from concatenating zero or more strings from **x**

If $\mathcal{L}(x) = \{a,ab\}$ then $\mathcal{L}(x^*) = \{\text{“”},a,ab,aa,aab,abab,\ldots\}$

- ε
  - It represents {“”}, set containing the empty string
- ϕ
  - It represents { }, the empty set

Example Regular Expressions

- **(0∨1)*1**
  - The set of all strings of **0**'s and **1**'s ending in 1, **{1, 01, 11,…}**
- **a*b(a*)**
  - The set of all strings of a's and b's with exactly one b
- **((01) ∨(10))***
  - You tell me
- Regular expressions (equivalently, regular grammars) important for lexing, breaking strings into recognized words

Right Regular Grammars

- Subclass of BNF (covered in detail sool)
- Only rules of form
  <nonterminal>::=<terminal><nonterminal> or
  <nonterminal>::=<terminal> or
  <nonterminal>::=ε
- Defines same class of languages as regular expressions
- Important for writing lexers (programs that convert strings of characters into strings of tokens)
- Close connection to nondeterministic finite state automata – nonterminals ≅ states; rule ≅ edge

Example

- Right regular grammar:
  <Balanced> ::= ε
  <Balanced> ::= 0<OneAndMore>
  <Balanced> ::= 1<ZeroAndMore>
  <OneAndMore> ::= 1<Balanced>
  <ZeroAndMore> ::= 0<Balanced>
- Generates even length strings where every initial substring of even length has same number of 0's as 1's

Implementing Regular Expressions

- Regular expressions reasonable way to generate strings in language
- Not so good for recognizing when a string is in language
- Problems with Regular Expressions
  - which option to choose,
  - how many repetitions to make
- Answer: finite state automata
- Should have seen in CS374

Example: Lexing

- Regular expressions good for describing lexemes (words) in a programming language
  - Identifier = (a ∨ b ∨ … ∨ z ∨ A ∨ B ∨ … ∨ Z) (a ∨ b ∨ … ∨ z ∨ A ∨ B ∨ … ∨ Z ∨ 0 ∨ 1 ∨ … ∨ 9)*
  - Digit = (0 ∨ 1 ∨ … ∨ 9)
  - Number = 0 ∨ (1 ∨ … ∨ 9)(0 ∨ … ∨ 9)* ∨ ~ (1 ∨ … ∨ 9)(0 ∨ … ∨ 9)*
  - Keywords: if = if, while = while,…

Lexing

- Different syntactic categories of "words": tokens

Example:

- Convert sequence of characters into sequence of strings, integers, and floating point numbers.
- "asd 123 jkl 3.14" will become:
  [String "asd"; Int 123; String "jkl"; Float 3.14]

Lex, ocamllex

- Could write the reg exp, then translate to DFA by hand
  - A lot of work
- Better: Write program to take reg exp as input and automatically generates automata
- Lex is such a program
- ocamllex version for ocaml

How to do it

- To use regular expressions to parse our input we need:
  - Some way to identify the input string — call it a lexing buffer
  - Set of regular expressions,
  - Corresponding set of actions to take when they are matched.

How to do it

- The lexer will take the regular expressions and generate a state machine.
- The state machine will take our lexing buffer and apply the transitions...
- If we reach an accepting state from which we can go no further, the machine will perform the appropriate action.

Mechanics

- Put table of reg exp and corresponding actions (written in ocaml) into a file *<filename>*.mll
- Call

  ocamllex *<filename>*.mll
- Produces Ocaml code for a lexical analyzer in file *<filename>*.ml

Sample Input

```
rule main = parse
 ['0'-'9']+ { print_string "Int\n"}
 | ['0'-'9']+'.'['0'-'9']+ { print_string "Float\n"}
 | ['a'-'z']+ { print_string "String\n"}
 | _ { main lexbuf }
 {
 let newlexbuf = (Lexing.from_channel stdin) in
 main newlexbuf
}
```


General Input

{ *header* }
let *ident* = *regexp* ...
rule *entrypoint* [*arg1*... *argn*] = parse
    regexp { *action* }
  | ...
  | *regexp* { *action* }
and *entrypoint* [*arg1*... *argn*] = parse ...and
  ...
{ *trailer* }

Ocamllex Input

- *header* and *trailer* contain arbitrary ocaml code put at top an bottom of *<filename>*.ml
- let *ident* = *regexp* ... Introduces *ident* for use in later regular expressions

Ocamllex Input

- *<filename>*.ml contains one lexing function per *entrypoint*
  - Name of function is name given for *entrypoint*
  - Each entry point becomes an Ocaml function that takes *n*+1 arguments, the extra implicit last argument being of type Lexing.lexbuf
- *arg1*... a*rgn* are for use in *action*

Ocamllex Regular Expression

- Single quoted characters for letters: 'a'
- _: (underscore) matches any letter
- Eof: special "end_of_file" marker
- Concatenation same as usual
- "*string*": concatenation of sequence of characters
- e_1 / e_2: choice - was $e_1 \vee e_2$

Ocamllex Regular Expression

- [c_1 - c_2]: choice of any character between first and second inclusive, as determined by character codes
- [^c_1 - c_2]: choice of any character NOT in set
- e*: same as before
- e+: same as $e\ e$*
- e?: option - was $e \vee \varepsilon$
- (e): same as e

Ocamllex Regular Expression

- e_1 # e_2: the characters in e_1 but not in e_2; e_1 and e_2 must describe just sets of characters
- *ident*: abbreviation for earlier reg exp in let *ident* = *regexp*
- e_1 as *id*: binds the result of e_1 to *id* to be used in the associated *action*

Ocamllex Manual

- More details can be found at

Version for ocaml 4.07:

https://v2.ocaml.org/releases/4.07/htmlman/lexyacc.html

Current version (ocaml 4.14)

https://v2.ocaml.org/releases/4.14/htmlman/lexyacc.html

(same, except formatting, I think)

End of Lect 18

Example : test.mll

```
{ type result = Int of int | Float of float |
  String of string }
let digit = ['0'-'9']
let digits = digit +
let lower_case = ['a'-'z']
let upper_case = ['A'-'Z']
let letter = upper_case | lower_case
let letters = letter +
```


Example : test.mll

```
rule main = parse
  (digits)'.'digits as f  { Float (float_of_string f) }
 | digits as n            { Int (int_of_string n) }
 | letters as s           { String s}
 | _ { main lexbuf }
 { let newlexbuf = (Lexing.from_channel stdin) in
 print_newline ();
 main newlexbuf  }
```


Example

```
# #use "test.ml";;
…
val main : Lexing.lexbuf -> result = <fun>
val __ocaml_lex_main_rec : Lexing.lexbuf -> int ->
  result = <fun>
hi there 234 5.2
- : result = String "hi"
```

What happened to the rest?!?

Example

```
# let b = Lexing.from_channel stdin;;
# main b;;
hi 673 there
- : result = String "hi"
# main b;;
- : result = Int 673
# main b;;
- : result = String "there"
```


Your Turn

- Work on MP8
  - Add a few keywords
  - Implement booleans and unit
  - Implement Ints and Floats
  - Implement identifiers

Problem

- How to get lexer to look at more than the first token at one time?
- Answer: *action* has to tell it to -- recursive calls
  - Not what you want to sew this together with ocamlyacc
- Side Benefit: can add “state” into lexing
- Note: already used this with the _ case

Example

```
rule main = parse
   (digits) '.' digits as f { Float
   (float_of_string f) :: main lexbuf}
 | digits as n          { Int (int_of_string n) ::
   main lexbuf }
 | letters as s         { String s :: main
   lexbuf}
 | eof                   { [] }
 | _                     { main lexbuf }
```


Example Results

```
hi there 234 5.2
- : result list = [String "hi"; String "there"; Int
  234; Float 5.2]
#
```

Used Ctrl-d to send the end-of-file signal

Dealing with comments

First Attempt

```
let open_comment = "(*"
let close_comment = "*)"
rule main = parse
   (digits) '.' digits as f { Float (float_of_string
   f) :: main lexbuf}
 | digits as n          { Int (int_of_string n) ::
   main lexbuf }
 | letters as s         { String s :: main lexbuf}
```


Dealing with comments

```
 | open_comment         { comment  lexbuf}
 | eof                  { [] }
 | _ { main lexbuf }
and comment = parse
   close_comment        { main lexbuf }
 | _                    { comment lexbuf }
```


Dealing with nested comments

```
rule main = parse …
 | open_comment         { comment 1 lexbuf}
 | eof                  { [] }
 | _ { main lexbuf }
and comment depth = parse
   open_comment         { comment (depth+1) lexbuf
   }
 | close_comment        { if depth = 1
                        then main lexbuf
                        else comment (depth - 1) lexbuf }
 | _                    { comment depth lexbuf }
```


Dealing with nested comments

```
rule main = parse
   (digits) '.' digits as f { Float (float_of_string f) ::
   main lexbuf}
 | digits as n          { Int (int_of_string n) :: main
   lexbuf }
 | letters as s         { String s :: main lexbuf}
 | open_comment         { (comment 1 lexbuf}
 | eof                  { [] }
 | _ { main lexbuf }
```

Dealing with nested comments

```
and comment depth = parse
  open_comment       { comment (depth+1) lexbuf
  }
| close_comment      { if depth = 1
                       then main lexbuf
                       else comment (depth - 1) lexbuf }
| _                  { comment depth lexbuf }
```


Types of Formal Language Descriptions

- Regular expressions, regular grammars
- Context-free grammars, BNF grammars, syntax diagrams
- Finite state automata
- Pushdown automata
- Whole family more of grammars and automata – covered in automata theory

Sample Grammar

- Language: Parenthesized sums of 0's and 1's
- <Sum> ::= 0
- <Sum >::= 1
- <Sum> ::= <Sum> + <Sum>
- <Sum> ::= (<Sum>)

BNF Grammars

- Start with a set of characters, **a,b,c,...**
  - We call these *terminals*
- Add a set of different characters, **X,Y,Z,...**
  - We call these *nonterminals*
- One special nonterminal **S** called *start symbol*

BNF Grammars

- BNF rules (aka *productions*) have form

  X ::= *y*

  where **X** is any nonterminal and *y* is a string of terminals and nonterminals
- BNF *grammar* is a set of BNF rules such that every nonterminal appears on the left of some rule

Sample Grammar

- Terminals: 0 1 + ( )
- Nonterminals: <Sum>
- Start symbol = <Sum>
- <Sum> ::= 0
- <Sum >::= 1
- <Sum> ::= <Sum> + <Sum>
- <Sum> ::= (<Sum>)
- Can be abbreviated as

  <Sum> ::= 0 | 1
  | <Sum> + <Sum> | (<Sum>)

BNF Deriviations

- Given rules

  X::= *y***Z***w* and **Z**::=*v*

  we may replace **Z** by *v* to say

  X => *y***Z***w* => *yvw*
- Sequence of such replacements called *derivation*
- Derivation called *right-most* if always replace the right-most non-terminal

BNF Derivations

- Start with the start symbol:

<Sum> =>

BNF Derivations

- Pick a non-terminal

<Sum> =>

BNF Derivations

- Pick a rule and substitute:
  - <Sum> ::= <Sum> + <Sum>

<Sum> => <Sum> + <Sum >

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >

BNF Derivations

- Pick a rule and substitute:
  - <Sum> ::= (<Sum>)

<Sum> => <Sum> + <Sum >

=> (<Sum>) + <Sum>

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>

BNF Derivations

- Pick a rule and substitute:
  - <Sum> ::= <Sum> + <Sum>

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>

BNF Derivations

- Pick a rule and substitute:
  - <Sum >::= 1

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>

BNF Derivations

- Pick a rule and substitute:
  - <Sum >::= 0

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>
=> (<Sum> + 1) + 0

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>
=> (<Sum> + 1) + 0

BNF Derivations

- Pick a rule and substitute
  - <Sum> ::= 0

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>
=> (<Sum> + 1) 0
=> (0 + 1) + 0

BNF Derivations

- (0 + 1) + 0 is generated by grammar

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>
=> (<Sum> + 1) + 0
=> (0 + 1) + 0

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Mechanics

- Put table of reg exp and corresponding actions (written in ocaml) into a file *<filename>*.mll
- Call

  ocamllex *<filename>*.mll
- Produces Ocaml code for a lexical analyzer in file *<filename>*.ml

Sample Input

```
rule main = parse
 ['0'-'9']+ { print_string "Int\n"}
 | ['0'-'9']+'.'['0'-'9']+ { print_string "Float\n"}
 | ['a'-'z']+ { print_string "String\n"}
 | _ { main lexbuf }
 {
 let newlexbuf = (Lexing.from_channel stdin) in
 main newlexbuf
}
```


General Input

{ *header* }
let *ident* = *regexp* ...
rule *entrypoint* [*arg1*... *argn*] = parse
    regexp { *action* }
  | ...
  | *regexp* { *action* }
and *entrypoint* [*arg1*... *argn*] = parse ...and
  ...
{ *trailer* }

Ocamllex Input

- *header* and *trailer* contain arbitrary ocaml code put at top an bottom of *<filename>*.ml
- let *ident* = *regexp* ... Introduces *ident* for use in later regular expressions

Ocamllex Input

- *<filename>*.ml contains one lexing function per *entrypoint*
  - Name of function is name given for *entrypoint*
  - Each entry point becomes an Ocaml function that takes *n*+1 arguments, the extra implicit last argument being of type Lexing.lexbuf
- *arg1*... a*rgn* are for use in *action*

Ocamllex Regular Expression

- Single quoted characters for letters: 'a'
- _: (underscore) matches any letter
- Eof: special "end_of_file" marker
- Concatenation same as usual
- "*string*": concatenation of sequence of characters
- e_1 / e_2: choice - what was $e_1 \vee e_2$

Ocamllex Regular Expression

- [c_1 - c_2]: choice of any character between first and second inclusive, as determined by character codes
- [^c_1 - c_2]: choice of any character NOT in set
- e*: same as before
- e+: same as $e\ e$*
- e?: option - was $e \vee \varepsilon$
- (e): same as e

Ocamllex Regular Expression

- e_1 # e_2: the characters in e_1 but not in e_2; e_1 and e_2 must describe just sets of characters
- *ident*: abbreviation for earlier reg exp in let *ident* = *regexp*
- e_1 as *id*: binds the result of e_1 to *id* to be used in the associated *action*

Ocamllex Manual

- More details can be found at

Version for ocaml 4.07:

https://v2.ocaml.org/releases/4.07/htmlman/lexyacc.html

Current version (ocaml 4.14)

https://v2.ocaml.org/releases/4.14/htmlman/lexyacc.html

(same, except formatting, I think)

Example : test.mll

```
{ type result = Int of int | Float of float |
  String of string }
let digit = ['0'-'9']
let digits = digit +
let lower_case = ['a'-'z']
let upper_case = ['A'-'Z']
let letter = upper_case | lower_case
let letters = letter +
```


Example : test.mll

```
rule main = parse
   (digits)'.'digits as f  { Float (float_of_string f) }
 | digits as n             { Int (int_of_string n) }
 | letters as s            { String s}
 | _ { main lexbuf }
 { let newlexbuf = (Lexing.from_channel stdin) in
 print_newline ();
 main newlexbuf  }
```

Example

```
# #use "test.ml";;
…
val main : Lexing.lexbuf -> result = <fun>
val __ocaml_lex_main_rec : Lexing.lexbuf -> int ->
  result = <fun>
hi there 234 5.2
- : result = String "hi"
```

What happened to the rest?!?

Example

```
# let b = Lexing.from_channel stdin;;
# main b;;
hi 673 there
- : result = String "hi"
# main b;;
- : result = Int 673
# main b;;
- : result = String "there"
```


Problem

- How to get lexer to look at more than the first token at one time?
- Answer: *action* has to tell it to -- recursive calls
  - Not what you want to sew this together with ocamlyacc
- Side Benefit: can add "state" into lexing
- Note: already used this with the _ case

Example

```
rule main = parse
   (digits) '.' digits as f { Float
   (float_of_string f) :: main lexbuf}
 | digits as n          { Int (int_of_string n) ::
   main lexbuf }
 | letters as s         { String s :: main
   lexbuf}
 | eof                   { [] }
 | _                     { main lexbuf }
```


Example Results

```
hi there 234 5.2
- : result list = [String "hi"; String "there"; Int
  234; Float 5.2]
#
```

Used Ctrl-d to send the end-of-file signal

Dealing with comments

First Attempt

```
let open_comment = "(*"
let close_comment = "*)"
rule main = parse
   (digits) '.' digits as f { Float (float_of_string
   f) :: main lexbuf}
 | digits as n          { Int (int_of_string n) ::
   main lexbuf }
 | letters as s         { String s :: main lexbuf}
```

Dealing with comments

```
 | open_comment        { comment  lexbuf}
 | eof                 { [] }
 | _ { main lexbuf }
and comment = parse
   close_comment       { main lexbuf }
 | _                   { comment lexbuf }
```


Dealing with nested comments

```
rule main = parse …
 | open_comment        { comment 1 lexbuf}
 | eof                 { [] }
 | _ { main lexbuf }
and comment depth = parse
   open_comment        { comment (depth+1) lexbuf
   }
 | close_comment       { if depth = 1
                         then main lexbuf
                        else comment (depth - 1) lexbuf }
 | _                   { comment depth lexbuf }
```


Dealing with nested comments

```
rule main = parse
   (digits) '.' digits as f { Float (float_of_string f) ::
   main lexbuf}
 | digits as n         { Int (int_of_string n) :: main
   lexbuf }
 | letters as s        { String s :: main lexbuf}
 | open_comment        { (comment 1 lexbuf}
 | eof                 { [] }
 | _ { main lexbuf }
```


Dealing with nested comments

```
and comment depth = parse
   open_comment        { comment (depth+1) lexbuf
   }
 | close_comment       { if depth = 1
                         then main lexbuf
                        else comment (depth - 1) lexbuf }
 | _                   { comment depth lexbuf }
```


Types of Formal Language Descriptions

- Regular expressions, regular grammars
- Context-free grammars, BNF grammars, syntax diagrams
- Finite state automata
- Pushdown automata
- Whole family more of grammars and automata – covered in automata theory

BNF Grammars

- Start with a set of characters, **a,b,c,…**
  - We call these *terminals*
- Add a set of different characters, **X,Y,Z,…**
  - We call these *nonterminals*
- One special nonterminal **S** called *start symbol*

BNF Grammars

- BNF rules (aka *productions*) have form

  X ::= *y*

  where **X** is any nonterminal and *y* is a string of terminals and nonterminals
- BNF *grammar* is a set of BNF rules such that every nonterminal appears on the left of some rule

Sample Grammar

- Terminals: 0 1 + ( )
- Nonterminals: <Sum>
- Start symbol = <Sum>

- <Sum> ::= 0
- <Sum >::= 1
- <Sum> ::= <Sum> + <Sum>
- <Sum> ::= (<Sum>)
- Can be abbreviated as

  <Sum> ::= 0 | 1
  | <Sum> + <Sum> | (<Sum>)

BNF Deriviations

- Given rules

  X::= *y***Z***w* and **Z**::=*v*

  we may replace **Z** by *v* to say

  X => *y***Z***w* => *yvw*
- Sequence of such replacements called *derivation*
- Derivation called *right-most* if always replace the right-most non-terminal

BNF Semantics

- The meaning of a BNF grammar is the set of all strings consisting only of terminals that can be derived from the Start symbol

BNF Derivations

- Start with the start symbol:

<Sum> =>

BNF Derivations

- Pick a non-terminal

<Sum> =>

BNF Derivations

- Pick a rule and substitute:
  - <Sum> ::= <Sum> + <Sum>

<Sum> => <Sum> + <Sum >

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >

BNF Derivations

- Pick a rule and substitute:
  - <Sum> ::= (<Sum>)

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>

BNF Derivations

- Pick a rule and substitute:
  - <Sum> ::= <Sum> + <Sum>

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>

BNF Derivations

- Pick a rule and substitute:
  - <Sum >::= 1

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>

BNF Derivations

- Pick a rule and substitute:
  - <Sum >::= 0

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>
=> (<Sum> + 1) + 0

BNF Derivations

- Pick a non-terminal:

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>
=> (<Sum> + 1) + 0

BNF Derivations

- Pick a rule and substitute
  - <Sum> ::= 0

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>
=> (<Sum> + 1) 0
=> (0 + 1) + 0

BNF Derivations

- (0 + 1) + 0 is generated by grammar

<Sum> => <Sum> + <Sum >
=> (<Sum>) + <Sum>
=> (<Sum> + <Sum>) + <Sum>
=> (<Sum> + 1) + <Sum>
=> (<Sum> + 1) + 0
=> (0 + 1) + 0

Extended BNF Grammars

- Alternatives: allow rules of from X::=*y*/*z*
  - Abbreviates X::= *y*, X::= *z*
- Options: X::=*y*[*v*]*z*
  - Abbreviates X::=*yvz*, X::=*yz*
- Repetition: X::=*y*{*v*}*z*
  - Can be eliminated by adding new nonterminal V and rules X::=*yz*, X::=*yVz*, V::=*v*, V::=*W*

Parse Trees

- Graphical representation of derivation
- Each node labeled with either non-terminal or terminal
- If node is labeled with a terminal, then it is a leaf (no sub-trees)
- If node is labeled with a non-terminal, then it has one branch for each character in the right-hand side of rule used to substitute for it

Example

- Consider grammar:

  <exp> ::= <factor>
  | <factor> + <factor>
  <factor> ::= <bin>
  | <bin> * <exp>
  <bin> ::= 0 | 1

- Problem: Build parse tree for 1 * 1 + 0 as an <exp>

Example cont.

- 1 * 1 + 0: <exp>

<exp> is the start symbol for this parse tree

Example cont.

- 1 * 1 + 0: <exp>
  |
  <factor>

Use rule: <exp> ::= <factor>

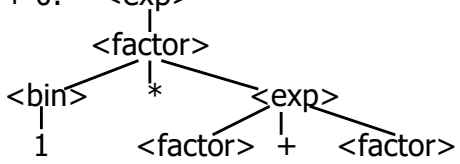
Example cont.

- 1 * 1 + 0: <exp>
  |
  <factor>
  / | \
  <bin> * <exp>

Use rule: <factor> ::= <bin> * <exp>

Example cont.

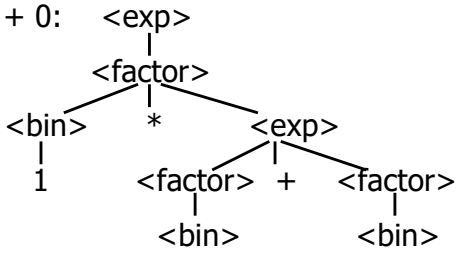
- 1 * 1 + 0: <exp>



Use rules: <bin> ::= 1 and
<exp> ::= <factor> + <factor>

Example cont.

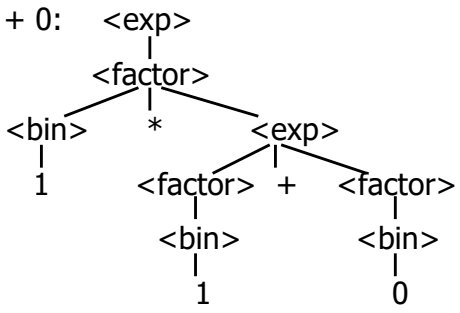
- 1 * 1 + 0: <exp>



Use rule: <factor> ::= <bin>

Example cont.

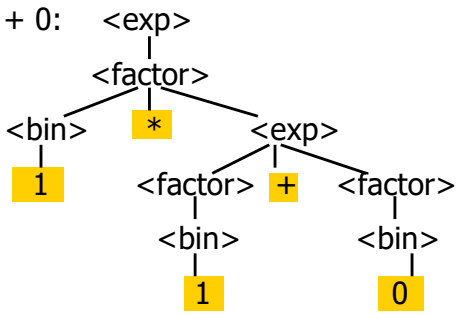
- 1 * 1 + 0: <exp>



Use rules: <bin> ::= 1 | 0

Example cont.

- 1 * 1 + 0: <exp>



Fringe of tree is string generated by grammar

Parse Tree Data Structures

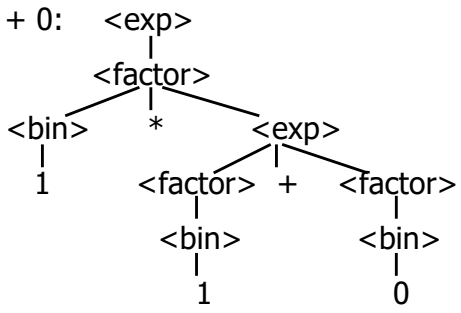
- Parse trees may be represented by OCaml datatypes
- One datatype for each nonterminal
- One constructor for each rule
- Defined as mutually recursive collection of datatype declarations

Example

- Recall grammar:
  <exp> ::= <factor> | <factor> + <factor>
  <factor> ::= <bin> | <bin> * <exp>
  <bin> ::= 0 | 1
- type exp = Factor2Exp of factor
  | Plus of factor * factor
  and factor = Bin2Factor of bin
  | Mult of bin * exp
  and bin = Zero | One

Example cont.

- 1 * 1 + 0:



Example cont.

- Can be represented as

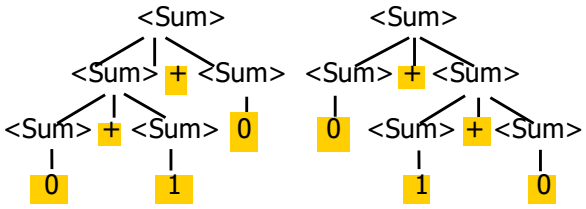
```
Factor2Exp
(Mult(One,
        Plus(Bin2Factor One,
                Bin2Factor Zero)))
```


Ambiguous Grammars and Languages

- A BNF grammar is *ambiguous* if its language contains strings for which there is more than one parse tree
- If all BNF's for a language are ambiguous then the language is *inherently ambiguous*

Example: Ambiguous Grammar

- 0 + 1 + 0



Two Major Sources of Ambiguity

- Lack of determination of operator precedence
- Lack of determination of operator assoicativity
- Not the only sources of ambiguity

Disambiguating a Grammar

- Given ambiguous grammar G, with start symbol S, find a grammar G' with same start symbol, such that

  language of G = language of G'
- Not always possible
- No algorithm in general

Disambiguating a Grammar

- Idea: Each non-terminal represents all strings having some property
- Identify these properties (often in terms of things that can't happen)
- Use these properties to inductively guarantee every string in language has a unique parse

Steps to Grammar Disambiguation

- Identify the rules and a smallest use that display ambiguity
- Decide which parse to keep; why should others be thrown out?
- What syntactic restrictions on subexpressions are needed to throw out the bad (while keeping the good)?
- Add a new non-terminal and rules to describe this set of restricted subexpressions (called stratifying, or refactoring)
- **Characterize each non-terminal by a language invariant**
- Replace old rules to use new non-terminals
- Rinse and repeat

Example

- Ambiguous grammar:
  <exp> ::= 0 | 1 | <exp> + <exp>
  | <exp> * <exp>
- String with more then one parse:
  0 + 1 + 0
  1 * 1 + 1
- Sourceof ambiuity: associativity and precedence

Two Major Sources of Ambiguity

- Lack of determination of operator precedence
- Lack of determination of operator assoicativity
- Not the only sources of ambiguity

How to Enforce Associativity

- Have at most one recursive call per production
- When two or more recursive calls would be natural leave right-most one for right assoicativity, left-most one for left assoiciativity

Example

- <Sum> ::= 0 | 1 | <Sum> + <Sum>
  | (<Sum>)
- Becomes
  - <Sum> ::= <Num> | <Num> + <Sum>
  - <Num> ::= 0 | 1 | (<Sum>)

<Sum> + <Sum> + <Sum>

Operator Precedence

- Operators of highest precedence evaluated first (bind more tightly).
- Precedence for infix binary operators given in following table
- Needs to be reflected in grammar

Precedence Table - Sample

	Fortan	Pascal	C/C++	Ada	SML
highest	**	*, /, div, mod	++, --	**	div, mod, /, *
	*, /	+, -	*, /, %	*, /, mod	+, -, ^
	+, -		+, -	+, -	::

Predence in Grammar

- Higher precedence translates to longer derivation chain
- Example:

<exp> ::= 0 | 1 | <exp> + <exp>
| <exp> * <exp>

- Becomes

<exp> ::= <mult_exp>
| <exp> + <mult_exp>
<mult_exp> ::= <id> | <mult_exp> * <id>
<id> ::= 0 | 1

Parser Code

- <*grammar*>.mly defines one parsing function per entry point
- Parsing function takes a lexing function (lexer buffer to token) and a lexer buffer as arguments
- Returns semantic attribute of corresponding entry point

Ocamlyacc Input

- File format:

%{
<*header*>
%}
<*declarations*>
%%
<*rules*>
%%
<*trailer*>

Ocamlyacc <*header*>

- Contains arbitrary Ocaml code
- Typically used to give types and functions needed for the semantic actions of rules and to give specialized error recovery
- May be omitted
- <*footer*> similar. Possibly used to call parser

Ocamlyacc <declarations>

- %token *symbol … symbol*
- Declare given symbols as tokens
- %token <*type*> *symbol … symbol*
- Declare given symbols as token constructors, taking an argument of type <*type*>
- %start *symbol … symbol*
- Declare given symbols as entry points; functions of same names in <*grammar*>.ml

Ocamlyacc <*declarations*>

- %type <*type*> *symbol … symbol*

  Specify type of attributes for given symbols. Mandatory for start symbols
- %left *symbol … symbol*
- %right *symbol … symbol*
- %nonassoc *symbol … symbol*

  Associate precedence and associativity to given symbols. Same line,same precedence; earlier line, lower precedence (broadest scope)

Ocamlyacc <*rules*>

- *nonterminal* :
  symbol ... symbol { *semantic_action* }
  | ...
  | *symbol ... symbol* { *semantic_action* }
  ;
- Semantic actions are arbitrary Ocaml expressions
- Must be of same type as declared (or inferred) for *nonterminal*
- Access semantic attributes (values) of symbols by position: $1 for first symbol, $2 to second …

Example - Base types

```
(* File: expr.ml *)
type expr =
    Term_as_Expr of term
  | Plus_Expr of (term * expr)
  | Minus_Expr of (term * expr)
and term =
    Factor_as_Term of factor
  | Mult_Term of (factor * term)
  | Div_Term of (factor * term)
and factor =
    Id_as_Factor of string
  | Parenthesized_Expr_as_Factor of expr
```


Example - Lexer (exprlex.mll)

```
{ (*open Exprparse*) }
let numeric = ['0' - '9']
let letter =['a' - 'z' 'A' - 'Z']
rule token = parse
  | "+" {Plus_token}
  | "-"  {Minus_token}
  | "*"  {Times_token}
  | "/"  {Divide_token}
  | "("  {Left_parenthesis}
  | ")"  {Right_parenthesis}
  | letter (letter|numeric|"_")* as id  {Id_token id}
  | [' ' '\t' '\n'] {token lexbuf}
  | eof {EOL}
```


Example - Parser (exprparse.mly)

```
%{ open Expr
%}
%token <string> Id_token
%token Left_parenthesis Right_parenthesis
%token Times_token Divide_token
%token Plus_token Minus_token
%token EOL
%start main
%type <expr> main
%%
```

Example - Parser (exprparse.mly)

```
expr:
   term
       { Term_as_Expr $1 }
 | term Plus_token expr
       { Plus_Expr ($1, $3) }
 | term Minus_token expr
       { Minus_Expr ($1, $3) }
```


Example - Parser (exprparse.mly)

```
term:
   factor
       { Factor_as_Term $1 }
 | factor Times_token term
       { Mult_Term ($1, $3) }
 | factor Divide_token term
       { Div_Term ($1, $3) }
```


Example - Parser (exprparse.mly)

```
factor:
   Id_token
       { Id_as_Factor $1 }
 | Left_parenthesis expr Right_parenthesis
       {Parenthesized_Expr_as_Factor $2 }
main:
 | expr EOL
       { $1 }
```


Example - Using Parser

```
# #use "expr.ml";;
...
# #use "exprparse.ml";;
...
# #use "exprlex.ml";;
...
# let test s =
  let lexbuf = Lexing.from_string (s^"\n") in
      main token lexbuf;;
```


Example - Using Parser

```
# test "a + b";;
- : expr =
Plus_Expr
 (Factor_as_Term (Id_as_Factor "a"),
  Term_as_Expr (Factor_as_Term
   (Id_as_Factor "b")))
```


LR Parsing

- Read tokens left to right (L)
- Create a rightmost derivation (R)
- How is this possible?
- Start at the bottom (left) and work your way up
- Last step has only one non-terminal to be replaced so is right-most
- Working backwards, replace mixed strings by non-terminals
- Always proceed so that there are no non-terminals to the right of the string to be replaced

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= <Sum> + ● 0 shift
= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>
=> <Sum> + 0 ● reduce
= <Sum> + ● 0 shift
= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> => <Sum> + <Sum > ● reduce
=> <Sum> + 0 ● reduce
= <Sum> + ● 0 shift
= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> ● => <Sum> + <Sum > ● reduce
=> <Sum> + 0 ● reduce
= <Sum> + ● 0 shift
= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example

(0 + 1) + 0

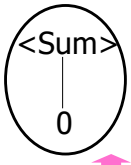
Example

(0 + 1) + 0

Example

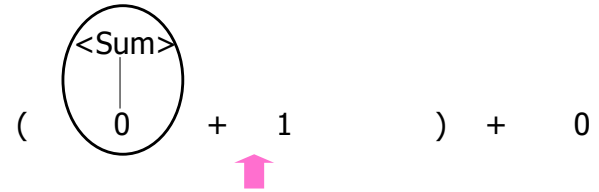
(0 + 1) + 0

Example

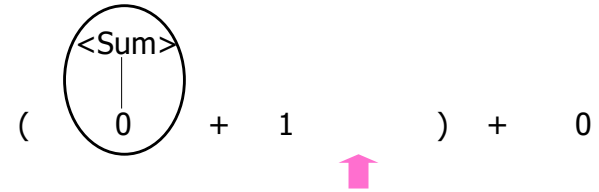


(0 + 1) + 0

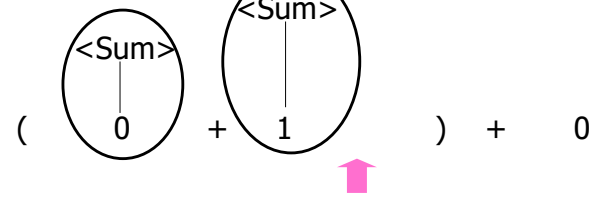
Example



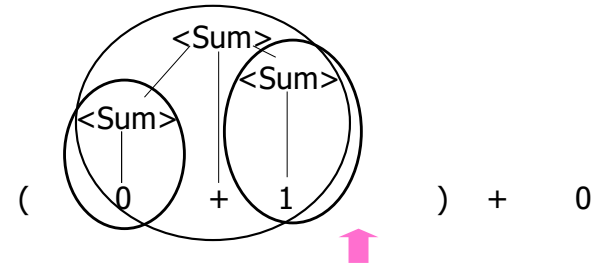
Example



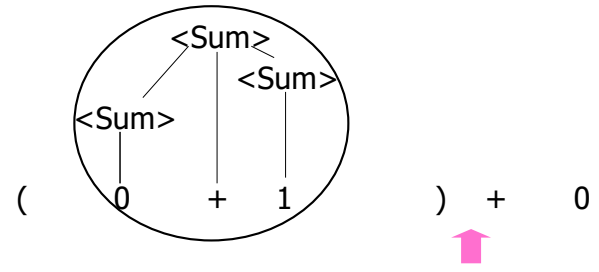
Example



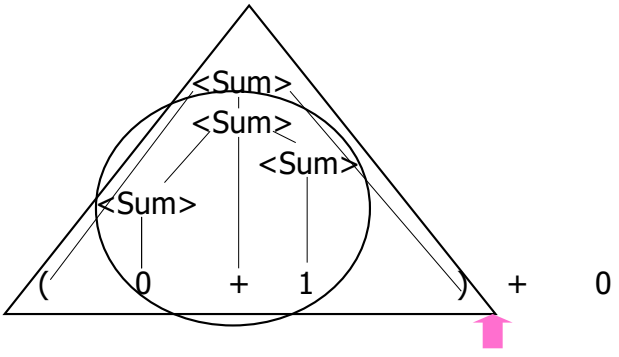
Example



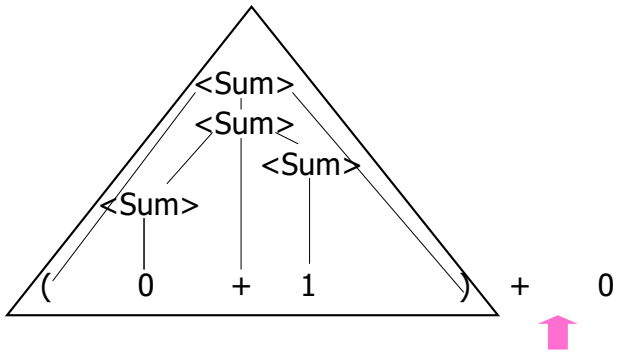
Example



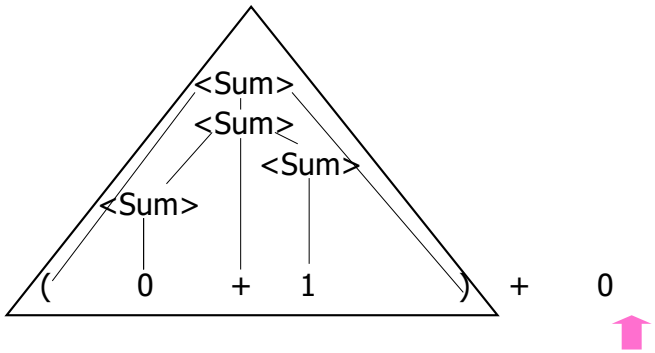
Example



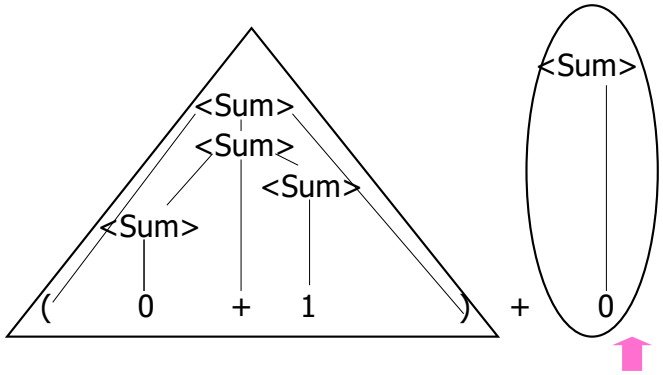
Example



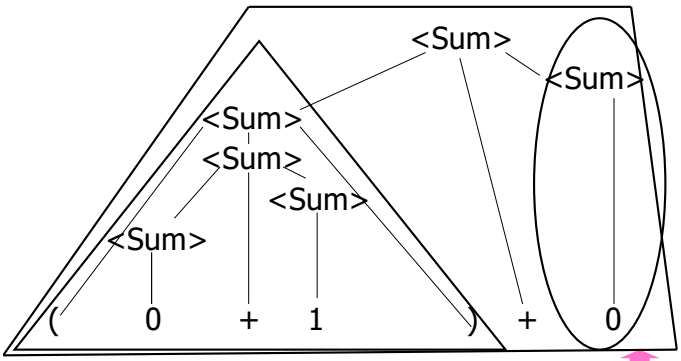
Example



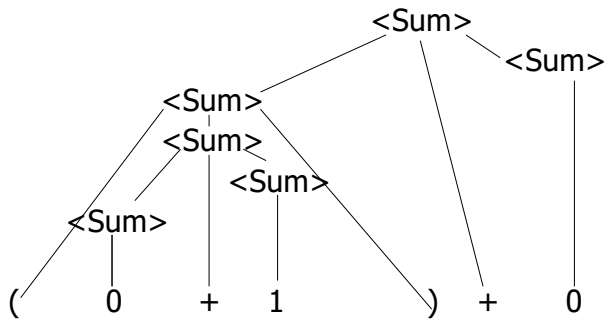
Example



Example



Example



LR Parsing Tables

- Build a pair of tables, Action and Goto, from the grammar
  - This is the hardest part, we omit here
  - Rows labeled by states
  - For Action, columns labeled by terminals and "end-of-tokens" marker
    - (more generally strings of terminals of fixed length)
  - For Goto, columns labeled by non-terminals

Action and Goto Tables

- Given a state and the next input, Action table says either
  - **shift** and go to state *n*, or
  - **reduce** by production *k* (explained in a bit)
  - **accept** or **error**
- Given a state and a non-terminal, Goto table says
  - go to state *m*

LR(i) Parsing Algorithm

- Based on push-down automata
- Uses states and transitions (as recorded in Action and Goto tables)
- Uses a stack containing states, terminals and non-terminals

LR(i) Parsing Algorithm

0. Insure token stream ends in special "end-of-tokens" symbol
1. Start in state 1 with an empty stack
2. Push **state**(1) onto stack
3. Look at next *i* tokens from token stream (*toks*) (don't remove yet)
4. If top symbol on stack is **state**(*n*), look up action in Action table at (*n*, *toks*)

LR(i) Parsing Algorithm

5. If action = **shift** *m*,
   a) Remove the top token from token stream and push it onto the stack
   b) Push **state**(*m*) onto stack
   c) Go to step 3

LR(i) Parsing Algorithm

6. If action = **reduce** *k* where production *k* is E ::= u
   a) Remove 2 * length(u) symbols from stack (u and all the interleaved states)
   b) If new top symbol on stack is **state**(*m*), look up new state *p* in Goto(*m*,E)
   c) Push E onto the stack, then push **state**(*p*) onto the stack
   d) Go to step 3

LR(i) Parsing Algorithm

7. If action = **accept**
   - Stop parsing, return success
8. If action = **error**,
   - Stop parsing, return failure

Adding Synthesized Attributes

- Add to each **reduce** a rule for calculating the new synthesized attribute from the component attributes
- Add to each non-terminal pushed onto the stack, the attribute calculated for it
- When performing a **reduce**,
  - gather the recorded attributes from each non-terminal popped from stack
  - Compute new attribute for non-terminal pushed onto stack

Shift-Reduce Conflicts

- **Problem**: can't decide whether the action for a state and input character should be **shift** or **reduce**
- Caused by ambiguity in grammar
- Usually caused by lack of associativity or precedence information in grammar

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

● 0 + 1 + 0	shift
-> 0 ● + 1 + 0	reduce
-> <Sum> ● + 1 + 0	shift
-> <Sum> + ● 1 + 0	shift
-> <Sum> + 1 ● + 0	reduce
-> <Sum> + <Sum> ● + 0	

Example - cont

- **Problem:** shift or reduce?
- You can shift-shift-reduce-reduce or reduce-shift-shift-reduce
- Shift first - right associative
- Reduce first- left associative

Reduce - Reduce Conflicts

- **Problem:** can't decide between two different rules to reduce by
- Again caused by ambiguity in grammar
- **Symptom:** RHS of one production suffix of another
- Requires examining grammar and rewriting it
- Harder to solve than shift-reduce errors

Example

- S ::= A | aB A ::= abc B ::= bc

● abc	shift
a ● bc	shift
ab ● c	shift
abc ●	

- Problem: reduce by B ::= bc then by S ::= aB, or by A::= abc then S::A?

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Disambiguating a Grammar

- Idea: Each non-terminal represents all strings having some property
  - In fact each string of terminals and non-terminals represents all strings having some property (Remember reg exp semantics)
- Identify these properties (often in terms of things that can't happen)
- Use these properties to inductively guarantee every string in language has a unique parse

Steps to Grammar Disambiguation

- Identify the rules and a smallest use that display ambiguity
- Decide which parse to keep; why should others be thrown out?
- What syntactic restrictions on subexpressions are needed to throw out the bad (while keeping the good)?
- Add a new non-terminal and rules to describe this set of restricted subexpressions (called stratifying, or refactoring)
- **Characterize each non-terminal by a language invariant**
- Replace old rules to use new non-terminals
- Rinse and repeat

Operator Precedence

- Operators of highest precedence evaluated first (bind more tightly).
- Precedence for infix binary operators given in following table
- Needs to be reflected in grammar
  - Saw how last class

Precedence Table - Sample

	Fortan	Pascal	C/C++	Ada	SML
highest	**	*, /, div, mod	++, --	**	div, mod, /, *
	*, /	+, -	*, /, %	*, /, mod	+, -, ^
	+, -		+, -	+, -	::

More Disambiguating Grammars

- M ::= M * M | (M) | M ++ | 6
- Ambiguous because of associativity of *
- because of conflict between * and ++:
- 6 * 6 ++ 6 * 6 ++



M ::= M * M | (M) | M ++ | 6

- How to disambiguate?
- Choose associativity for *
- Choose precedence between * and ++
- Four possibilities
- Four different approaches
- Some easier than others
- Will do --- My choice, then yours if time

M ::= M * M | (M) | M ++ | 6

- Think about 6 * 6 ++ * 6 * 6 ++
- Let's start with observations
- If * binds less tightly than ++, then no * can be the immediate subtree to a ++.
  - We would need a language for things that don't parse as *
- If * binds more tightly than ++, then …
- The right subtree to * can't be a ++
- But the left can!
  - Need different languages of the left and right

M ::= M * M | (M) | M ++ | 6

- * higher prec than ++
  - 6 * 6 ++ 6 ++ * 6
- M :: = M++ | StarExp | (M) | 6
- What is StarExp
- It is everything that parses as a * and can't parse as a ++
- But what is the associativity of *?
- I'll choose left

M ::= M * M | (M) | M ++ | 6

- * higher prec than ++
  - 6 * 6 ++ * 6 ++ * 6
- * Left assoc
- M :: = M++ | StarExp | (M) | 6
- StarExp ::= PossStar * NotStarNotPlusPlus
- What is PossStar? It could it be a *, but it also doesn't have to be.
- Can it be ++? YES! It can be anything
- It is M !

M ::= M * M | (M) | M ++ | 6

- * higher prec than ++
  - 6 * 6 ++ 6 ++ * 6
- * Left assoc
- M :: = M++ | StarExp | (M) | 6
- StarExp ::= M * NotStarNotPlusPlus

M ::= M * M | (M) | M ++ | 6

- * higher prec than ++
  - 6 * 6 ++ 6 ++ * 6
- * Left assoc
- M :: = M++ | StarExp | (M) | 6
- StarExp ::= M * NotStarNotPlusPlus
- But what is NotStarNotPlusPlus?
- Well, the other two original rules: (M) | 6

M ::= M * M | (M) | M ++ | 6

- * higher prec than ++
  - 6 * 6 ++ 6 ++ * 6
- * Left assoc
- M :: = M++ | StarExp | (M) | 6
- StarExp ::= M * NotStarNotPlusPlus
- NotStarNotPlusPlus ::= (M) | 6
- But we have (M) | 6 twice, and it's the same language each time. Let's have one

M ::= M * M | (M) | M ++ | 6

- * higher prec than ++
  - 6 * 6 ++ 6 ++ * 6
- * Left assoc
- M :: = M++ | StarExp | NotStarNotPlusPlus
- StarExp ::= M * NotStarNotPlusPlus
- NotStarNotPlusPlus ::= (M) | 6

Parser Code

- <*grammar*>.ml defines one parsing function per entry point
- Parsing function takes a lexing function (lexer buffer to token) and a lexer buffer as arguments
- Returns semantic attribute of corresponding entry point

Ocamlyacc Input

- File format:

%{
  <*header*>
%}
  <*declarations*>
%%
  <*rules*>
%%
  <*trailer*>

Ocamlyacc <*header*>

- Contains arbitrary Ocaml code
- Typically used to give types and functions needed for the semantic actions of rules and to give specialized error recovery
- May be omitted
- <*footer*> similar. Possibly used to call parser

Ocamlyacc <declarations>

- %token *symbol … symbol*
- Declare given symbols as tokens
- %token <*type*> *symbol … symbol*
- Declare given symbols as token constructors, taking an argument of type <*type*>
- %start *symbol … symbol*
- Declare given symbols as entry points; functions of same names in <*grammar*>.ml

Ocamlyacc <*declarations*>

- %type <*type*> *symbol … symbol*

  Specify type of attributes for given symbols. Mandatory for start symbols
- %left *symbol … symbol*
- %right *symbol … symbol*
- %nonassoc *symbol … symbol*

  Associate precedence and associativity to given symbols. Same line,same precedence; earlier line, lower precedence (broadest scope)

Ocamlyacc <*rules*>

- *nonterminal* :
  symbol ... symbol { *semantic_action* }
  | ...
  | *symbol ... symbol* { *semantic_action* }
  ;
- Semantic actions are arbitrary Ocaml expressions
- Must be of same type as declared (or inferred) for *nonterminal*
- Access semantic attributes (values) of symbols by position: $1 for first symbol, $2 to second …

Example - Base types

```
(* File: expr.ml *)
type expr =
    Term_as_Expr of term
  | Plus_Expr of (term * expr)
  | Minus_Expr of (term * expr)
and term =
    Factor_as_Term of factor
  | Mult_Term of (factor * term)
  | Div_Term of (factor * term)
and factor =
    Id_as_Factor of string
  | Parenthesized_Expr_as_Factor of expr
```


Example - Lexer (exprlex.mll)

```
{ (*open Exprparse*) }
let numeric = ['0' - '9']
let letter =['a' - 'z' 'A' - 'Z']
rule token = parse
  | "+" {Plus_token}
  | "-"  {Minus_token}
  | "*"  {Times_token}
  | "/"  {Divide_token}
  | "("  {Left_parenthesis}
  | ")"  {Right_parenthesis}
  | letter (letter|numeric|"_")* as id  {Id_token id}
  | [' ' '\t' '\n'] {token lexbuf}
  | eof {EOL}
```


Example - Parser (exprparse.mly)

```
%{ open Expr
%}
%token <string> Id_token
%token Left_parenthesis Right_parenthesis
%token Times_token Divide_token
%token Plus_token Minus_token
%token EOL
%start main
%type <expr> main
%%
```


Example - Parser (exprparse.mly)

```
expr:
   term
       { Term_as_Expr $1 }
 | term Plus_token expr
       { Plus_Expr ($1, $3) }
 | term Minus_token expr
       { Minus_Expr ($1, $3) }
```

Example - Parser (exprparse.mly)

```
term:
   factor
       { Factor_as_Term $1 }
 | factor Times_token term
       { Mult_Term ($1, $3) }
 | factor Divide_token term
       { Div_Term ($1, $3) }
```


Example - Parser (exprparse.mly)

```
factor:
    Id_token
        { Id_as_Factor $1 }
  | Left_parenthesis expr Right_parenthesis
        {Parenthesized_Expr_as_Factor $2 }
main:
  | expr EOL
        { $1 }
```


Example - Using Parser

```
# #use "expr.ml";;
...
# #use "exprparse.ml";;
...
# #use "exprlex.ml";;
...
# let test s =
  let lexbuf = Lexing.from_string (s^"\n") in
      main token lexbuf;;
```


Example - Using Parser

```
# test "a + b";;
- : expr =
Plus_Expr
 (Factor_as_Term (Id_as_Factor "a"),
  Term_as_Expr (Factor_as_Term
   (Id_as_Factor "b")))
```


LR Parsing

- Read tokens left to right (L)
- Create a rightmost derivation (R)
- How is this possible?
- Start at the bottom (left) and work your way up
- Last step has only one non-terminal to be replaced so is right-most
- Working backwards, replace mixed strings by non-terminals
- Always proceed so that there are no non-terminals to the right of the string to be replaced

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>

= <Sum> + ● 0 shift
= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> =>
=> <Sum> + 0 ● reduce
= <Sum> + ● 0 shift
= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> => <Sum> + <Sum > ● reduce
=> <Sum> + 0 ● reduce
= <Sum> + ● 0 shift
= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

<Sum> ● => <Sum> + <Sum > ● reduce
=> <Sum> + 0 ● reduce
= <Sum> + ● 0 shift
= <Sum> ● + 0 shift
=> (<Sum>) ● + 0 reduce
= (<Sum> ●) + 0 shift
=> (<Sum> + <Sum> ●) + 0 reduce
=> (<Sum> + 1 ●) + 0 reduce
= (<Sum> + ● 1) + 0 shift
= (<Sum> ● + 1) + 0 shift
=> (0 ● + 1) + 0 reduce
= (● 0 + 1) + 0 shift
= ● (0 + 1) + 0 shift

Example

(0 + 1) + 0

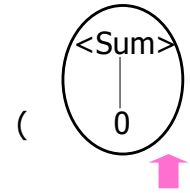
Example

(0 + 1) + 0

Example

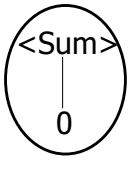
(0 + 1) + 0

Example



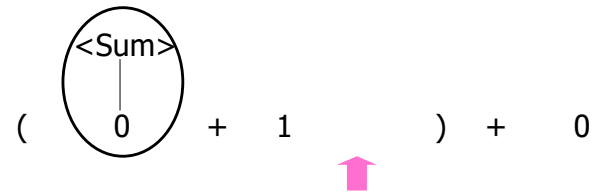
(+ 1) + 0

Example

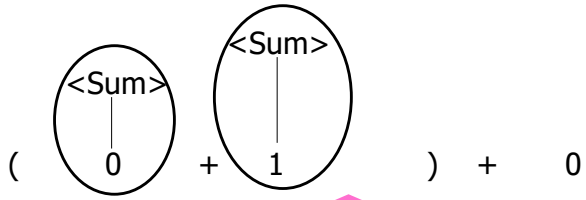


(+ 1) + 0

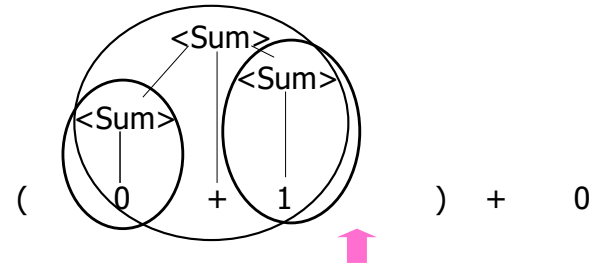
Example



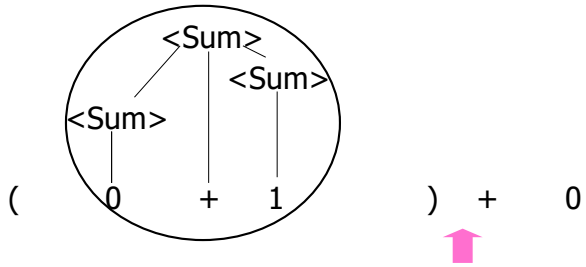
Example



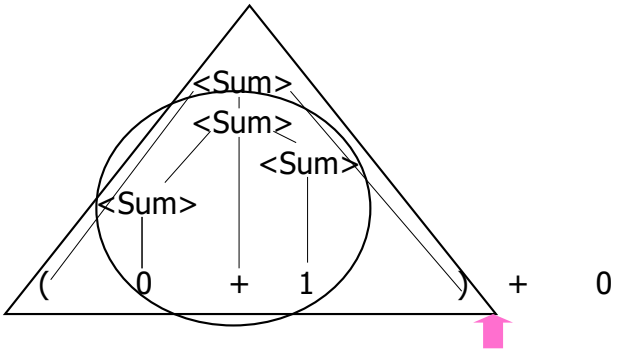
Example



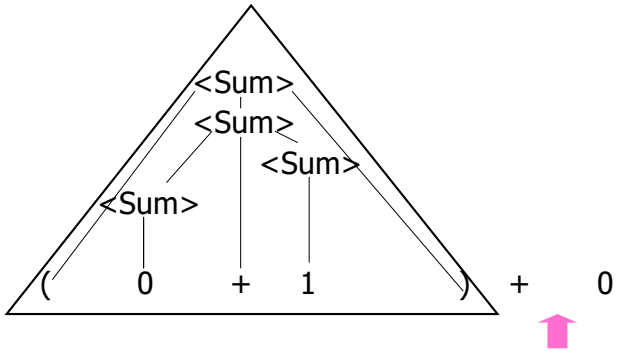
Example



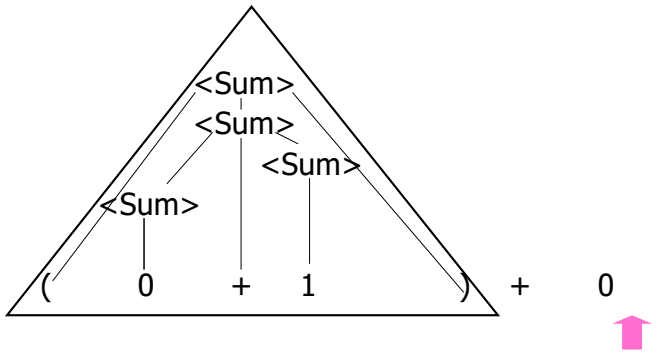
Example



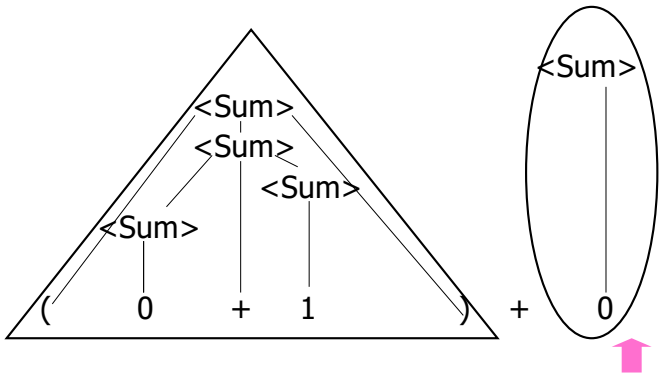
Example



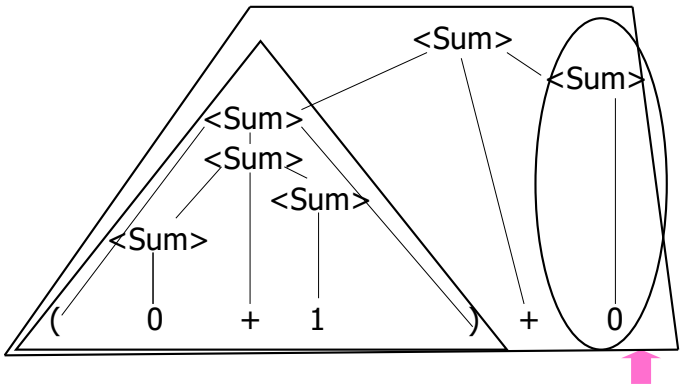
Example



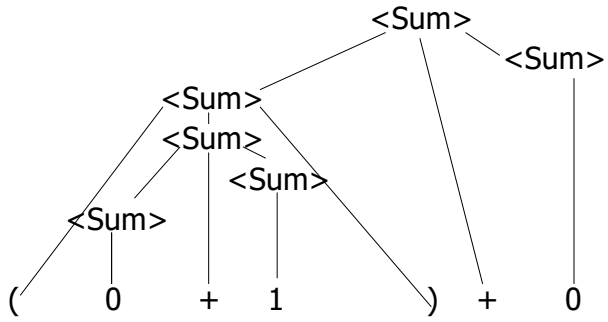
Example



Example



Example



LR Parsing Tables

- Build a pair of tables, Action and Goto, from the grammar
  - This is the hardest part, we omit here
  - Rows labeled by states
  - For Action, columns labeled by terminals and "end-of-tokens" marker
    - (more generally strings of terminals of fixed length)
  - For Goto, columns labeled by non-terminals

Action and Goto Tables

- Given a state and the next input, Action table says either
  - **shift** and go to state *n*, or
  - **reduce** by production *k* (explained in a bit)
  - **accept** or **error**
- Given a state and a non-terminal, Goto table says
  - go to state *m*

LR(i) Parsing Algorithm

- Based on push-down automata
- Uses states and transitions (as recorded in Action and Goto tables)
- Uses a stack containing states, terminals and non-terminals

LR(i) Parsing Algorithm

0. Insure token stream ends in special "end-of-tokens" symbol
1. Start in state 1 with an empty stack
2. Push **state**(1) onto stack
3. Look at next *i* tokens from token stream (*toks*) (don't remove yet)
4. If top symbol on stack is **state**(*n*), look up action in Action table at (*n*, *toks*)

LR(i) Parsing Algorithm

5. If action = **shift** *m*,
   a) Remove the top token from token stream and push it onto the stack
   b) Push **state**(*m*) onto stack
   c) Go to step 3

LR(i) Parsing Algorithm

6. If action = **reduce** *k* where production *k* is E ::= u
   a) Remove 2 * length(u) symbols from stack (u and all the interleaved states)
   b) If new top symbol on stack is **state**(*m*), look up new state *p* in Goto(*m*,E)
   c) Push E onto the stack, then push **state**(*p*) onto the stack
   d) Go to step 3

LR(i) Parsing Algorithm

7. If action = **accept**
   - Stop parsing, return success
8. If action = **error**,
   - Stop parsing, return failure

Adding Synthesized Attributes

- Add to each **reduce** a rule for calculating the new synthesized attribute from the component attributes
- Add to each non-terminal pushed onto the stack, the attribute calculated for it
- When performing a **reduce**,
  - gather the recorded attributes from each non-terminal popped from stack
  - Compute new attribute for non-terminal pushed onto stack

Shift-Reduce Conflicts

- **Problem**: can't decide whether the action for a state and input character should be **shift** or **reduce**
- Caused by ambiguity in grammar
- Usually caused by lack of associativity or precedence information in grammar

Example: <Sum> = 0 | 1 | (<Sum>) | <Sum> + <Sum>

● 0 + 1 + 0	shift
-> 0 ● + 1 + 0	reduce
-> <Sum> ● + 1 + 0	shift
-> <Sum> + ● 1 + 0	shift
-> <Sum> + 1 ● + 0	reduce
-> <Sum> + <Sum> ● + 0	

Example - cont

- **Problem:** shift or reduce?
- You can shift-shift-reduce-reduce or reduce-shift-shift-reduce
- Shift first - right associative
- Reduce first- left associative

Reduce - Reduce Conflicts

- **Problem:** can't decide between two different rules to reduce by
- Again caused by ambiguity in grammar
- **Symptom:** RHS of one production suffix of another
- Requires examining grammar and rewriting it
- Harder to solve than shift-reduce errors

Example

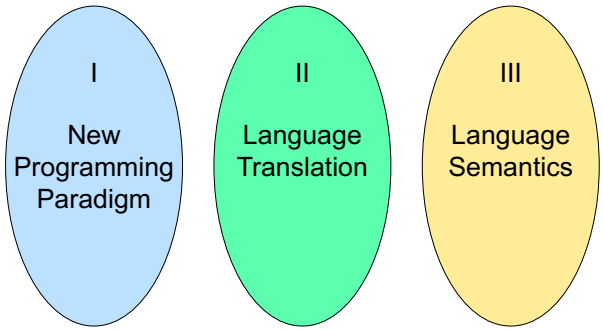
- S ::= A | aB A ::= abc B ::= bc

● abc	shift
a ● bc	shift
ab ● c	shift
abc ●	

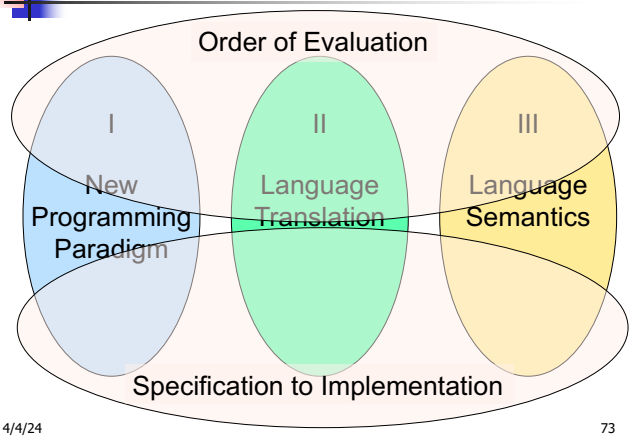
- Problem: reduce by B ::= bc then by S ::= aB, or by A::= abc then S::A?

Programming Languages & Compilers

Three Main Topics of the Course

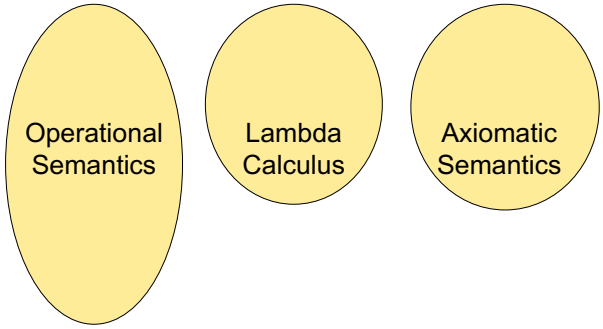


Programming Languages & Compilers

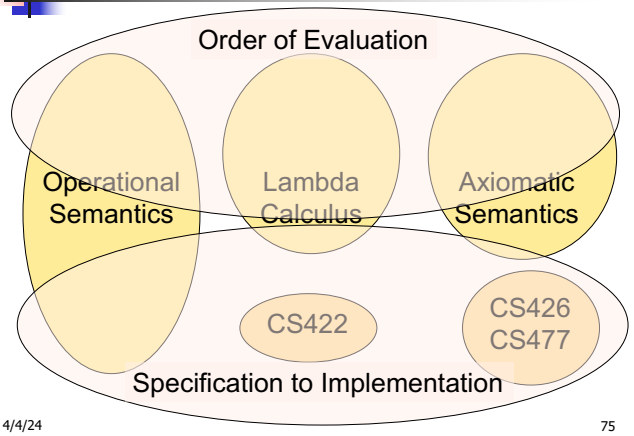


Programming Languages & Compilers

III : Language Semantics



Programming Languages & Compilers



Semantics

- Expresses the meaning of syntax
- Static semantics
  - Meaning based only on the form of the expression without executing it
  - Usually restricted to type checking / type inference

Dynamic semantics

- Method of describing meaning of executing a program
- Several different types:
  - Operational Semantics
  - Axiomatic Semantics
  - Denotational Semantics

Dynamic Semantics

- Different languages better suited to different types of semantics
- Different types of semantics serve different purposes

Operational Semantics

- Start with a simple notion of machine
- Describe how to execute (implement) programs of language on virtual machine, by describing how to execute each program statement (ie, following the *structure* of the program)
- Meaning of program is how its execution changes the state of the machine
- Useful as basis for implementations

Axiomatic Semantics

- Also called Floyd-Hoare Logic
- Based on formal logic (first order predicate calculus)
- Axiomatic Semantics is a logical system built from *axioms* and *inference rules*
- Mainly suited to simple imperative programming languages

Axiomatic Semantics

- Used to formally prove a property (*post-condition*) of the *state* (the values of the program variables) after the execution of program, assuming another property (*pre-condition*) of the state before execution
- Written :

  {Precondition} Program {Postcondition}
- Source of idea of *loop invariant*

Denotational Semantics

- Construct a function $\mathcal{M}$ assigning a mathematical meaning to each program construct
- Lambda calculus often used as the range of the meaning function
- Meaning function is compositional: meaning of construct built from meaning of parts
- Useful for proving properties of programs

Natural Semantics

- Aka "Big Step Semantics"
- Provide value for a program by rules and derivations, similar to type derivations
- Rule conclusions look like

$$(C, m) \Downarrow m'$$

or

$$(E, m) \Downarrow v$$

Simple Imperative Programming Language

- $I \in$ *Identifiers*
- $N \in$ *Numerals*
- *B* ::= true | false | *B* & *B* | *B* or *B* | not *B* | *E* < *E* | *E* = *E*
- *E*::= *N* | *I* | *E* + *E* | *E* * *E* | *E* - *E* | - *E*
- *C*::= skip | *C*;*C* | *I* ::= *E* | if *B* then *C* else *C* fi | while *B* do *C* od

Natural Semantics of Atomic Expressions

- Identifiers: $(I,m) \Downarrow m(I)$
- Numerals are values: $(N,m) \Downarrow N$
- Booleans: $(\text{true},m) \Downarrow \text{true}$
  $(\text{false},m) \Downarrow \text{false}$

Booleans:

$$\frac{(B, m) \Downarrow \text{false}}{(B \,\&\, B', m) \Downarrow \text{false}} \qquad \frac{(B, m) \Downarrow \text{true} \quad (B', m) \Downarrow b}{(B \,\&\, B', m) \Downarrow b}$$

$$\frac{(B, m) \Downarrow \text{true}}{(B \text{ or } B', m) \Downarrow \text{true}} \qquad \frac{(B, m) \Downarrow \text{false} \quad (B', m) \Downarrow b}{(B \text{ or } B', m) \Downarrow b}$$

$$\frac{(B, m) \Downarrow \text{true}}{(\text{not } B, m) \Downarrow \text{false}} \qquad \frac{(B, m) \Downarrow \text{false}}{(\text{not } B, m) \Downarrow \text{true}}$$

Relations

$$\frac{(E, m) \Downarrow U \quad (E', m) \Downarrow V \quad U \sim V = b}{(E \sim E', m) \Downarrow b}$$

- By $U \sim V = b$, we mean does (the meaning of) the relation $\sim$ hold on the meaning of U and V
- May be specified by a mathematical expression/equation or rules matching U and V

Arithmetic Expressions

$$\frac{(E, m) \Downarrow U \quad (E', m) \Downarrow V \quad U \; op \; V = N}{(E \; op \; E', m) \Downarrow N}$$

where N is the specified value for $U \; op \; V$

Commands

Skip: $(\text{skip}, m) \Downarrow m$

Assignment: $$\frac{(E,m) \Downarrow V}{(I ::= E, m) \Downarrow m[I <\!\!-\!\!- V]}$$

Sequencing: $$\frac{(C,m) \Downarrow m' \quad (C',m') \Downarrow m''}{(C;C', m) \Downarrow m''}$$

If Then Else Command

$$\frac{(B,m) \Downarrow \text{true} \quad (C,m) \Downarrow m'}{(\text{if } B \text{ then } C \text{ else } C' \text{ fi}, m) \Downarrow m'}$$

$$\frac{(B,m) \Downarrow \text{false} \quad (C',m) \Downarrow m'}{(\text{if } B \text{ then } C \text{ else } C' \text{ fi}, m) \Downarrow m'}$$

While Command

$$\frac{(B,m) \Downarrow \text{false}}{(\text{while } B \text{ do } C \text{ od}, m) \Downarrow m}$$

$$\frac{(B,m) \Downarrow \text{true} \quad (C,m) \Downarrow m' \quad (\text{while } B \text{ do } C \text{ od}, m') \Downarrow m''}{(\text{while } B \text{ do C od}, m) \Downarrow m''}$$

Example: If Then Else Rule

$$\frac{}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \Downarrow ?}$$

Example: If Then Else Rule

$$\frac{\dfrac{}{(x > 5, \{x \to 7\}) \Downarrow ?}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \Downarrow ?}$$

Example: Arith Relation

$$\frac{? > ? = ? \qquad \dfrac{(x,\{x \to 7\}) \Downarrow ? \quad (5,\{x \to 7\}) \Downarrow ?}{(x > 5, \{x \to 7\}) \Downarrow ?}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \Downarrow ?}$$

Example: Identifier(s)

$$\frac{7 > 5 = \text{true} \qquad \dfrac{(x,\{x \to 7\}) \Downarrow 7 \quad (5,\{x \to 7\}) \Downarrow 5}{(x > 5, \{x \to 7\}) \Downarrow ?}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \Downarrow ?}$$

Example: Arith Relation

$$\frac{7 > 5 = \text{true} \qquad \dfrac{(x,\{x \to 7\}) \Downarrow 7 \quad (5,\{x \to 7\}) \Downarrow 5}{(x > 5, \{x \to 7\}) \Downarrow \text{true}}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \Downarrow ?}$$

Example: If Then Else Rule

$$\frac{7 > 5 = \text{true} \quad (x,\{x\text{->}7\})\Downarrow 7 \quad (5,\{x\text{->}7\})\Downarrow 5 \quad (x > 5,\{x \text{->} 7\})\Downarrow \text{true} \qquad (y\text{:=}\ 2 + 3,\{x\text{->}\ 7\}) \Downarrow\ ?}{(\text{if } x > 5 \text{ then } y\text{:=}\ 2 + 3 \text{ else } y\text{:=}3 + 4 \text{ fi},\ \{x \text{->}\ 7\}) \Downarrow\ ?}\ .$$

Example: Assignment

$$\frac{7 > 5 = \text{true} \quad (x,\{x\text{->}7\})\Downarrow 7 \quad (5,\{x\text{->}7\})\Downarrow 5 \quad (x > 5,\{x \text{->} 7\})\Downarrow \text{true} \qquad \dfrac{(2+3,\ \{x\text{->}7\})\Downarrow ?}{(y\text{:=}\ 2 + 3,\{x\text{->}\ 7\}) \Downarrow\ ?}}{(\text{if } x > 5 \text{ then } y\text{:=}\ 2 + 3 \text{ else } y\text{:=}3 + 4 \text{ fi},\ \{x \text{->}\ 7\}) \Downarrow\ ?}$$

Example: Arith Op

$$\frac{7 > 5 = \text{true} \quad (x,\{x\text{->}7\})\Downarrow 7 \quad (5,\{x\text{->}7\})\Downarrow 5 \quad (x > 5,\{x \text{->} 7\})\Downarrow \text{true} \qquad \dfrac{? + ? = ? \quad (2,\{x\text{->}7\})\Downarrow ? \quad (3,\{x\text{->}7\})\ \Downarrow ? \qquad (2+3,\ \{x\text{->}7\})\Downarrow ?}{(y\text{:=}\ 2 + 3,\{x\text{->}\ 7\}) \Downarrow ?}}{(\text{if } x > 5 \text{ then } y\text{:=}\ 2 + 3 \text{ else } y\text{:=}3 + 4 \text{ fi},\ \{x \text{->}\ 7\}) \Downarrow\ ?}\ .$$

Example: Numerals

$$\frac{7 > 5 = \text{true} \quad (x,\{x\text{->}7\})\Downarrow 7 \quad (5,\{x\text{->}7\})\Downarrow 5 \quad (x > 5,\{x \text{->} 7\})\Downarrow \text{true} \qquad \dfrac{2 + 3 = 5 \quad (2,\{x\text{->}7\})\Downarrow 2 \quad (3,\{x\text{->}7\})\ \Downarrow 3 \qquad (2+3,\ \{x\text{->}7\})\Downarrow ?}{(y\text{:=}\ 2 + 3,\{x\text{->}\ 7\}) \Downarrow\ ?}}{(\text{if } x > 5 \text{ then } y\text{:=}\ 2 + 3 \text{ else } y\text{:=}3 + 4 \text{ fi},\ \{x \text{->}\ 7\}) \Downarrow\ ?}$$

Example: Arith Op

$$\frac{7 > 5 = \text{true} \quad (x,\{x\text{->}7\})\Downarrow 7 \quad (5,\{x\text{->}7\})\Downarrow 5 \quad (x > 5,\{x \text{->} 7\})\Downarrow \text{true} \qquad \dfrac{2 + 3 = 5 \quad (2,\{x\text{->}7\})\Downarrow 2 \quad (3,\{x\text{->}7\})\ \Downarrow 3 \qquad (2+3,\ \{x\text{->}7\})\Downarrow 5}{(y\text{:=}\ 2 + 3,\{x\text{->}\ 7\}) \Downarrow ?}}{(\text{if } x > 5 \text{ then } y\text{:=}\ 2 + 3 \text{ else } y\text{:=}3 + 4 \text{ fi},\ \{x \text{->}\ 7\}) \Downarrow\ ?}$$

Example: Assignment

$$\frac{7 > 5 = \text{true} \quad (x,\{x\text{->}7\})\Downarrow 7 \quad (5,\{x\text{->}7\})\Downarrow 5 \quad (x > 5,\{x \text{->} 7\})\Downarrow \text{true} \qquad \dfrac{2 + 3 = 5 \quad (2,\{x\text{->}7\})\Downarrow 2 \quad (3,\{x\text{->}7\})\ \Downarrow 3 \qquad (2+3,\ \{x\text{->}7\})\Downarrow 5}{(y\text{:=}\ 2 + 3,\{x\text{->}\ 7\}) \Downarrow \{x\text{->}7,\ y\text{->}5\}}}{(\text{if } x > 5 \text{ then } y\text{:=}\ 2 + 3 \text{ else } y\text{:=}3 + 4 \text{ fi},\ \{x \text{->}\ 7\}) \Downarrow ?}$$

Example: If Then Else Rule

$$\dfrac{\dfrac{7 > 5 = \text{true} \quad (x,\{x\text{->}7\}) \Downarrow 7 \quad (5,\{x\text{->}7\}) \Downarrow 5}{(x > 5, \{x \text{ -> } 7\}) \Downarrow \text{true}} \qquad \dfrac{2 + 3 = 5 \quad (2,\{x\text{->}7\}) \Downarrow 2 \quad (3,\{x\text{->}7\}) \Downarrow 3}{(2+3, \{x\text{->}7\}) \Downarrow 5}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \text{ -> } 7\}) \Downarrow \{x\text{->}7, y\text{->}5\}}$$

Let in Command

$$\dfrac{(E,m) \Downarrow v \quad (C, m[I \text{<-} v]) \Downarrow m'}{(\text{let } I = E \text{ in } C, m) \Downarrow m''}$$

Where $m''(y) = m'(y)$ for $y \neq I$ and $m''(I) = m(I)$ if $m(I)$ is defined, and $m''(I)$ is undefined otherwise

Example

$$\dfrac{(5,\{x\text{->}17\}) \Downarrow 5 \qquad \dfrac{(x,\{x\text{->}5\}) \Downarrow 5 \quad (3,\{x\text{->}5\}) \Downarrow 3}{(x+3,\{x\text{->}5\}) \Downarrow 8} \quad (x:=x+3,\{x\text{->}5\}) \Downarrow \{x\text{->}8\}}{(\text{let } x = 5 \text{ in } (x:=x+3), \{x \text{ -> } 17\}) \Downarrow ?}$$

Example

$$\dfrac{(5,\{x\text{->}17\}) \Downarrow 5 \qquad \dfrac{(x,\{x\text{->}5\}) \Downarrow 5 \quad (3,\{x\text{->}5\}) \Downarrow 3}{(x+3,\{x\text{->}5\}) \Downarrow 8} \quad (x:=x+3,\{x\text{->}5\}) \Downarrow \{x\text{->}8\}}{(\text{let } x = 5 \text{ in } (x:=x+3), \{x \text{ -> } 17\}) \Downarrow \{x\text{->}17\}}$$

Comment

- Simple Imperative Programming Language introduces variables *implicitly* through assignment
- The let-in command introduces scoped variables *explictly*
- Clash of constructs apparent in awkward semantics

Interpretation Versus Compilation

- A **compiler** from language L1 to language L2 is a program that takes an L1 program and for each piece of code in L1 generates a piece of code in L2 of same meaning
- An **interpreter** of L1 in L2 is an L2 program that executes the meaning of a given L1 program
- Compiler would examine the body of a loop once; an interpreter would examine it every time the loop was executed

Interpreter

- An *Interpreter* represents the operational semantics of a language L1 (source language) in the language of implementation L2 (target language)
- Built incrementally
  - Start with literals
  - Variables
  - Primitive operations
  - Evaluation of expressions
  - Evaluation of commands/declarations

Interpreter

- Takes abstract syntax trees as input
  - In simple cases could be just strings
- One procedure for each syntactic category (nonterminal)
  - eg one for expressions, another for commands
- If Natural semantics used, tells how to compute final value from code
- If Transition semantics used, tells how to compute next "state"
  - To get final value, put in a loop

Natural Semantics Example

- compute_exp (Var(v), m) = look_up v m
- compute_exp (Int(n), _) = Num (n)
- …
- compute_com(IfExp(b,c1,c2),m) =
  if compute_exp (b,m) = Bool(true)
  then compute_com (c1,m)
  else compute_com (c2,m)

Natural Semantics Example

- compute_com(While(b,c), m) =
  if compute_exp (b,m) = Bool(false)
  then m
  else compute_com
  (While(b,c), compute_com(c,m))
- May fail to terminate - exceed stack limits
- Returns no useful information then

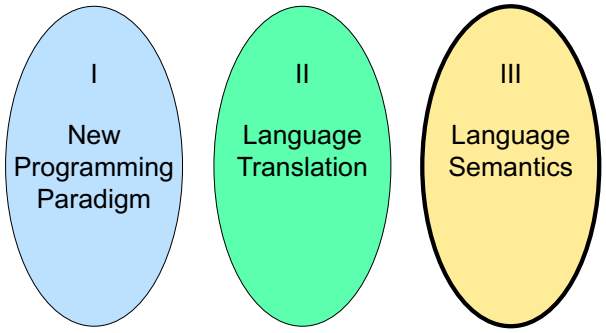
Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

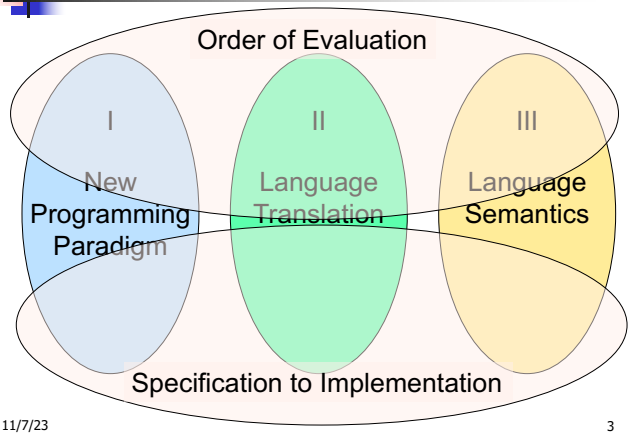
Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Programming Languages & Compilers

Three Main Topics of the Course

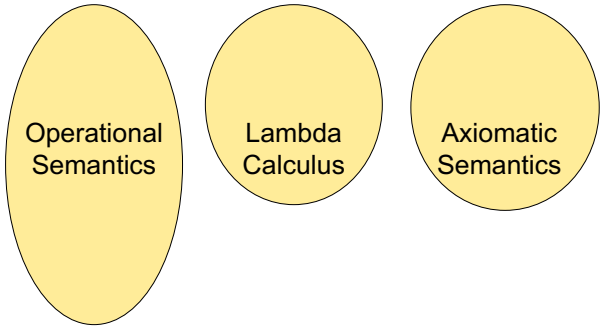


Programming Languages & Compilers

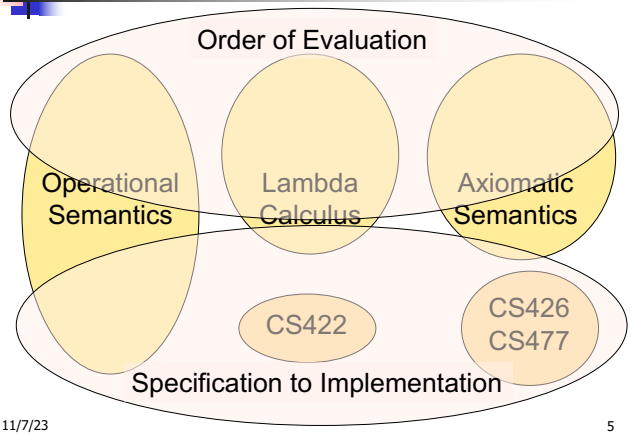


Programming Languages & Compilers

III : Language Semantics



Programming Languages & Compilers



Semantics

- Expresses the meaning of syntax
- Static semantics
  - Meaning based only on the form of the expression without executing it
  - Usually restricted to type checking / type inference

Dynamic semantics

- Method of describing meaning of executing a program
- Several different types:
  - Operational Semantics
  - Axiomatic Semantics
  - Denotational Semantics

Dynamic Semantics

- Different languages better suited to different types of semantics
- Different types of semantics serve different purposes

Operational Semantics

- Start with a simple notion of machine
- Describe how to execute (implement) programs of language on virtual machine, by describing how to execute each program statement (ie, following the *structure* of the program)
- Meaning of program is how its execution changes the state of the machine
- Useful as basis for implementations

Axiomatic Semantics

- Also called Floyd-Hoare Logic
- Based on formal logic (first order predicate calculus)
- Axiomatic Semantics is a logical system built from *axioms* and *inference rules*
- Mainly suited to simple imperative programming languages

Axiomatic Semantics

- Used to formally prove a property (*post-condition*) of the *state* (the values of the program variables) after the execution of program, assuming another property (*pre-condition*) of the state before execution
- Written :

  {Precondition} Program {Postcondition}
- Source of idea of *loop invariant*

Denotational Semantics

- Construct a function $\mathcal{M}$ assigning a mathematical meaning to each program construct
- Lambda calculus often used as the range of the meaning function
- Meaning function is compositional: meaning of construct built from meaning of parts
- Useful for proving properties of programs

Natural Semantics

- Aka Structural Operational Semantics, aka "Big Step Semantics"
- Provide value for a program by rules and derivations, similar to type derivations
- Rule conclusions look like

$$(C, m) \Downarrow m'$$

or

$$(E, m) \Downarrow v$$

Simple Imperative Programming Language

- $I \in$ *Identifiers*
- $N \in$ *Numerals*
- B ::= true | false | B & B | B or B | not B | $E < E$ | $E = E$
- E::= N | I | $E + E$ | $E * E$ | $E - E$ | $- E$ | (E)
- C::= skip | $C;C$ | $I := E$ | if B then C else C fi | while B do C od

Natural Semantics of Atomic Expressions

- Identifiers: $(I,m) \Downarrow m(I)$
- Numerals are values: $(N,m) \Downarrow N$
- Booleans: (true,m) $\Downarrow$ true
  (false ,m) $\Downarrow$ false

Booleans:

$$\frac{(B, m) \Downarrow \text{false}}{(B \,\&\, B', m) \Downarrow \text{false}} \qquad \frac{(B, m) \Downarrow \text{true} \quad (B', m) \Downarrow b}{(B \,\&\, B', m) \Downarrow b}$$

$$\frac{(B, m) \Downarrow \text{true}}{(B \text{ or } B', m) \Downarrow \text{true}} \qquad \frac{(B, m) \Downarrow \text{false} \quad (B', m) \Downarrow b}{(B \text{ or } B', m) \Downarrow b}$$

$$\frac{(B, m) \Downarrow \text{true}}{(\text{not } B, m) \Downarrow \text{false}} \qquad \frac{(B, m) \Downarrow \text{false}}{(\text{not } B, m) \Downarrow \text{true}}$$

Relations

$$\frac{(E, m) \Downarrow U \quad (E', m) \Downarrow V \quad U \sim V = b}{(E \sim E', m) \Downarrow b}$$

- By $U \sim V = b$, we mean does (the meaning of) the relation ~ hold on the meaning of U and V
- May be specified by a mathematical expression/equation or rules matching U and V

Arithmetic Expressions

$$\frac{(E, m) \Downarrow U \quad (E', m) \Downarrow V \quad U \mathbin{op} V = N}{(E \mathbin{op} E', m) \Downarrow N}$$

where N is the specified value for U *op* V

Commands

Skip: $(\text{skip}, m) \Downarrow m$

Assignment: $$\frac{(E,m) \Downarrow V}{(I{:=}E,m) \Downarrow m[I \text{ <-- } V] \ (=\{I \text{ -> } V\}+m)}$$

Sequencing: $$\frac{(C,m) \Downarrow m' \quad (C',m') \Downarrow m''}{(C;C', m) \Downarrow m''}$$

If Then Else Command

$$\frac{(B,m) \Downarrow \text{true} \quad (C,m) \Downarrow m'}{(\text{if } B \text{ then } C \text{ else } C' \text{ fi}, m) \Downarrow m'}$$

$$\frac{(B,m) \Downarrow \text{false} \quad (C',m) \Downarrow m'}{(\text{if } B \text{ then } C \text{ else } C' \text{ fi}, m) \Downarrow m'}$$

While Command

$$\frac{(B,m) \Downarrow \text{false}}{(\text{while } B \text{ do } C \text{ od}, m) \Downarrow m}$$

$$\frac{(B,m) \Downarrow \text{true} \quad (C,m) \Downarrow m' \quad (\text{while } B \text{ do } C \text{ od}, m') \Downarrow m''}{(\text{while } B \text{ do C od}, m) \Downarrow m''}$$

Example: If Then Else Rule

$$\frac{}{(\text{if } x > 5 \text{ then } y{:=}\ 2 + 3 \text{ else } y{:=}3 + 4 \text{ fi}, \{x \text{ -> } 7\}) \Downarrow ?}$$

Example: If Then Else Rule

$$\frac{\dfrac{}{(x > 5, \{x \text{ -> } 7\}) \Downarrow ?}}{(\text{if } x > 5 \text{ then } y{:=}\ 2 + 3 \text{ else } y{:=}3 + 4 \text{ fi}, \{x \text{ -> } 7\}) \Downarrow\ ?}$$

Example: Arith Relation

$$\frac{? > ? = ? \qquad \dfrac{(x,\{x\text{->}7\}) \Downarrow ? \quad (5,\{x\text{->}7\}) \Downarrow ?}{(x > 5, \{x \text{ -> } 7\}) \Downarrow ?}}{(\text{if } x > 5 \text{ then } y{:=}\ 2 + 3 \text{ else } y{:=}3 + 4 \text{ fi}, \{x \text{ -> } 7\}) \Downarrow ?}$$

Example: Identifier(s)

$$\frac{7 > 5 = \text{true} \qquad (x,\{x\text{->}7\}) \Downarrow 7 \quad (5,\{x\text{->}7\}) \Downarrow 5 \qquad (x > 5,\ \{x \text{ -> } 7\}) \Downarrow ?}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi},\ \{x \text{ -> } 7\}) \Downarrow ?}$$

Example: Arith Relation

$$\frac{7 > 5 = \text{true} \qquad (x,\{x\text{->}7\}) \Downarrow 7 \quad (5,\{x\text{->}7\}) \Downarrow 5 \qquad (x > 5,\ \{x \text{ -> } 7\}) \Downarrow \text{true}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi},\ \{x \text{ -> } 7\}) \Downarrow ?}$$

Example: If Then Else Rule

$$\frac{7 > 5 = \text{true} \qquad (x,\{x\text{->}7\}) \Downarrow 7 \quad (5,\{x\text{->}7\}) \Downarrow 5 \qquad (x > 5,\ \{x \text{ -> } 7\}) \Downarrow \text{true} \qquad \dfrac{}{(y := 2 + 3,\ \{x\text{->}\ 7\} \Downarrow ? \quad .}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi},\ \{x \text{ -> } 7\}) \Downarrow ?}$$

Example: Assignment

$$\frac{\dfrac{}{7 > 5 = \text{true}} \qquad (x,\{x\text{->}7\}) \Downarrow 7 \quad (5,\{x\text{->}7\}) \Downarrow 5 \qquad (x > 5,\ \{x \text{ -> } 7\}) \Downarrow \text{true} \qquad \dfrac{(2+3,\ \{x\text{->}7\}) \Downarrow ?}{(y := 2 + 3,\ \{x\text{->}\ 7\} \Downarrow ?}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi},\ \{x \text{ -> } 7\}) \Downarrow ?}$$

Example: Arith Op

$$\frac{\dfrac{}{7 > 5 = \text{true}} \qquad (x,\{x\text{->}7\}) \Downarrow 7 \quad (5,\{x\text{->}7\}) \Downarrow 5 \qquad (x > 5,\ \{x \text{ -> } 7\}) \Downarrow \text{true} \qquad \dfrac{\dfrac{? + ? = ?}{(2,\{x\text{->}7\}) \Downarrow ? \quad (3,\{x\text{->}7\}) \Downarrow ?} \qquad (2+3,\ \{x\text{->}7\}) \Downarrow ?}{(y := 2 + 3,\ \{x\text{->}\ 7\} \Downarrow ? \quad .}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi},\ \{x \text{ -> } 7\}) \Downarrow ?}$$

Example: Numerals

$$\frac{\dfrac{}{7 > 5 = \text{true}} \qquad (x,\{x\text{->}7\}) \Downarrow 7 \quad (5,\{x\text{->}7\}) \Downarrow 5 \qquad (x > 5,\ \{x \text{ -> } 7\}) \Downarrow \text{true} \qquad \dfrac{\dfrac{2 + 3 = 5}{(2,\{x\text{->}7\}) \Downarrow 2 \quad (3,\{x\text{->}7\}) \Downarrow 3} \qquad (2+3,\ \{x\text{->}7\}) \Downarrow ?}{(y := 2 + 3,\ \{x\text{->}\ 7\} \Downarrow ?}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi},\ \{x \text{ -> } 7\}) \Downarrow ?}$$

Example: Arith Op

$$\dfrac{\dfrac{(x,\{x\to 7\})\Downarrow 7 \quad (5,\{x\to 7\})\Downarrow 5 \quad 7>5=\text{true}}{(x>5,\{x\to 7\})\Downarrow \text{true}} \quad \dfrac{\dfrac{(2,\{x\to 7\})\Downarrow 2 \quad (3,\{x\to 7\})\Downarrow 3 \quad 2+3=5}{(2+3,\{x\to 7\})\Downarrow 5}}{(y:=2+3,\{x\to 7\})\Downarrow ?}$$

$$(\text{if } x>5 \text{ then } y:=2+3 \text{ else } y:=3+4 \text{ fi}, \{x\to 7\})\Downarrow\ ?$$

Example: Assignment

$$\dfrac{\dfrac{(x,\{x\to 7\})\Downarrow 7 \quad (5,\{x\to 7\})\Downarrow 5 \quad 7>5=\text{true}}{(x>5,\{x\to 7\})\Downarrow \text{true}} \quad \dfrac{\dfrac{(2,\{x\to 7\})\Downarrow 2 \quad (3,\{x\to 7\})\Downarrow 3 \quad 2+3=5}{(2+3,\{x\to 7\})\Downarrow 5}}{(y:=2+3,\{x\to 7\})\Downarrow \{x\to 7, y\to 5\}}$$

$$(\text{if } x>5 \text{ then } y:=2+3 \text{ else } y:=3+4 \text{ fi}, \{x\to 7\})\Downarrow ?$$

Example: If Then Else Rule

$$\dfrac{\dfrac{(x,\{x\to 7\})\Downarrow 7 \quad (5,\{x\to 7\})\Downarrow 5 \quad 7>5=\text{true}}{(x>5,\{x\to 7\})\Downarrow \text{true}} \quad \dfrac{\dfrac{(2,\{x\to 7\})\Downarrow 2 \quad (3,\{x\to 7\})\Downarrow 3 \quad 2+3=5}{(2+3,\{x\to 7\})\Downarrow 5}}{(y:=2+3,\{x\to 7\})\Downarrow \{x\to 7, y\to 5\}}$$

$$(\text{if } x>5 \text{ then } y:=2+3 \text{ else } y:=3+4 \text{ fi}, \{x\to 7\})\Downarrow \{x\to 7, y\to 5\}$$

Comment

- Simple Imperative Programming Language introduces variables *implicitly* through assignment
- The let-in command introduces scoped variables *explictly*
- Clash of constructs apparent in awkward semantics

Interpretation Versus Compilation

- A **compiler** from language L1 to language L2 is a program that takes an L1 program and for each piece of code in L1 generates a piece of code in L2 of same meaning
- An **interpreter** of L1 in L2 is an L2 program that executes the meaning of a given L1 program
- Compiler would examine the body of a loop once; an interpreter would examine it every time the loop was executed

Interpreter

- An *Interpreter* represents the operational semantics of a language L1 (source language) in the language of implementation L2 (target language)
- Built incrementally
  - Start with literals
  - Variables
  - Primitive operations
  - Evaluation of expressions
  - Evaluation of commands/declarations

Interpreter

- Takes abstract syntax trees as input
  - In simple cases could be just strings
- One procedure for each syntactic category (nonterminal)
  - eg one for expressions, another for commands
- If Natural semantics used, tells how to compute final value from code
- If Transition semantics used, tells how to compute next "state"
  - To get final value, put in a loop

Natural Semantics Example

- compute_exp (Var(v), m) = look_up v m
- compute_exp (Int(n), _) = Num (n)
- …
- compute_com(IfExp(b,c1,c2),m) =
  if compute_exp (b,m) = Bool(true)
  then compute_com (c1,m)
  else compute_com (c2,m)

Natural Semantics Example

- compute_com(While(b,c), m) =
  if compute_exp (b,m) = Bool(false)
  then m
  else compute_com
  (While(b,c), compute_com(c,m))
- May fail to terminate - exceed stack limits
- Returns no useful information then

Transition Semantics

- Form of operational semantics
- Describes how each program construct transforms machine state by *transitions*
- Rules look like
  (C, *m*) --> (C', *m'*) or (C,*m*) --> *m'*
- *C, C'* is code remaining to be executed
- m, m' represent the state/store/memory/environment
  - Partial mapping from identifiers to values
  - Sometimes *m* (or *C*) not needed
- Indicates exactly one step of computation

Expressions and Values

- *C, C'* used for commands; *E, E' for* expressions; *U,V* for values
- Special class of expressions designated as *values*
  - Eg 2, 3 are values, but 2+3 is only an expression
- Memory only holds values
  - Other possibilities exist

Evaluation Semantics

- Transitions successfully stops when E/*C* is a value/memory
- Evaluation fails if no transition possible, but not at value/memory
- Value/memory is the final *meaning* of original expression/command (in the given state)
- Coarse semantics: final value / memory
- More fine grained: whole transition sequence

Simple Imperative Programming Language

- $I \in$ *Identifiers*
- $N \in$ *Numerals*
- B ::= true | false | B & B | B or B | not B | $E < E$ | $E = E$
- E::= N | I | $E + E$ | $E * E$ | $E - E$ | $- E$
- C::= skip | $C;C$ | $I ::= E$ | if B then C else C fi | while B do C od

Transitions for Expressions

- Numerals are values
- Boolean values = {true, false}
- Identifiers: (I,m) --> $(m(I), m)$

Boolean Operations:

- Operators: (short-circuit)

(false & B, m) --> (false,m)

(true & B, m) --> (B,m)

$$\frac{(B, m) \text{ --> } (B'', m)}{(B \,\&\, B', m) \text{ --> } (B'' \,\&\, B', m)}$$

(true or B, m) --> (true,m)

(false or B, m) --> (B,m)

$$\frac{(B, m) \text{ --> } (B'', m)}{(B \text{ or } B', m) \text{ --> } (B'' \text{ or } B', m)}$$

(not true, m) --> (false,m)

(not false, m) --> (true,m)

$$\frac{(B, m) \text{ --> } (B', m)}{(\text{not } B, m) \text{ --> } (\text{not } B', m)}$$

Relations

$$\frac{(E, m) \text{ --> } (E'', m)}{(E \sim E', m) \text{ --> } (E'' \sim E', m)}$$

$$\frac{(E, m) \text{ --> } (E', m)}{(V \sim E, m) \text{ --> } (V \sim E', m)}$$

($U \sim V$, m) --> (true,m) or (false,m) depending on whether $U \sim V$ holds or not

Arithmetic Expressions

$$\frac{(E, m) \text{ --> } (E'', m)}{(E \text{ op } E', m) \text{ --> } (E'' \text{ op } E', m)}$$

$$\frac{(E, m) \text{ --> } (E', m)}{(V \text{ op } E, m) \text{ --> } (V \text{ op } E', m)}$$

(U op V, m) --> (N,m) where N is the specified value for U op V

Commands - in English

- skip means done evaluating
- When evaluating an assignment, evaluate the expression first
- If the expression being assigned is already a value, update the memory with the new value for the identifier
- When evaluating a sequence, work on the first command in the sequence first
- If the first command evaluates to a new memory (ie completes), evaluate remainder with new memory

Commands

$$(\text{skip}, m) \dashrightarrow m$$

$$\frac{(E,m) \dashrightarrow (E',m)}{(I::=E,m) \dashrightarrow (I::=E',m)}$$

$$(I::=V,m) \dashrightarrow m[I \leftarrow V]$$

$$\frac{(C,m) \dashrightarrow (C'',m')}{(C;C',m) \dashrightarrow (C'';C',m')} \qquad \frac{(C,m) \dashrightarrow m'}{(C;C',m) \dashrightarrow (C',m')}$$

If Then Else Command - in English

- If the boolean guard in an if_then_else is true, then evaluate the first branch
- If it is false, evaluate the second branch
- If the boolean guard is not a value, then start by evaluating it first.

If Then Else Command

(if true then C else C' fi, m) --> (C, m)

(if false then C else C' fi, m) --> (C', m)

$$\frac{(B,m) \dashrightarrow (B',m)}{(\text{if } B \text{ then } C \text{ else } C' \text{ fi}, m) \dashrightarrow (\text{if } B' \text{ then } C \text{ else } C' \text{ fi}, m)}$$

What should while transition to?

(while B do C od, m) → ?

Wrong! BAD

$$\frac{(B, m) \rightarrow (B', m)}{(\text{while B do C od, m}) \dashrightarrow (\text{while B' do C od, m})}$$

While Command

(while B do C od, m) -->
(if B then C; while B do C od else skip fi, m)

In English: Expand a While into a test of the boolean guard, with the true case being to do the body and then try the while loop again, and the false case being to stop.

Example Evaluation

- First step:

$$\frac{}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \to ?}$$

Example Evaluation

- First step:

$$\frac{(x > 5, \{x \to 7\}) \to ?}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \to ?}$$

Example Evaluation

- First step:

$$\frac{\dfrac{(x,\{x \to 7\}) \to (7, \{x \to 7\})}{(x > 5, \{x \to 7\}) \to ?}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \to ?}$$

Example Evaluation

- First step:

$$\frac{\dfrac{(x,\{x \to 7\}) \to (7, \{x \to 7\})}{(x > 5, \{x \to 7\}) \to (7 > 5, \{x \to 7\})}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \to ?}$$

Example Evaluation

- First step:

$$\frac{\dfrac{(x,\{x \to 7\}) \to (7, \{x \to 7\})}{(x > 5, \{x \to 7\}) \to (7 > 5, \{x \to 7\})}}{(\text{if } x > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \to (\text{if } 7 > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\})}$$

Example Evaluation

- Second Step:

$$\frac{(7 > 5, \{x \to 7\}) \to (\text{true}, \{x \to 7\})}{(\text{if } 7 > 5 \text{ then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \to (\text{if true then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\})}$$

- Third Step:

$$(\text{if true then } y := 2 + 3 \text{ else } y := 3 + 4 \text{ fi}, \{x \to 7\}) \to (y := 2 + 3, \{x \to 7\})$$

Example Evaluation

- Fourth Step:

$$\frac{(2+3, \{x\text{-}> 7\}) \text{ --> } (5, \{x \text{ -> } 7\})}{(y:=2+3, \{x\text{->}7\}) \text{ --> } (y:=5, \{x\text{->}7\})}$$

- Fifth Step:

(y:=5, {x->7}) --> {y -> 5, x -> 7}

Example Evaluation

- Bottom Line:

(if x > 5 then y:= 2 + 3 else y:=3 + 4 fi, {x -> 7})
--> (if 7 > 5 then y:=2 + 3 else y:=3 + 4 fi, {x -> 7})
-->(if true then y:=2 + 3 else y:=3 + 4 fi, {x -> 7})
-->(y:=2+3, {x->7})
--> (y:=5, {x->7}) --> {y -> 5, x -> 7}

Transition Semantics Evaluation

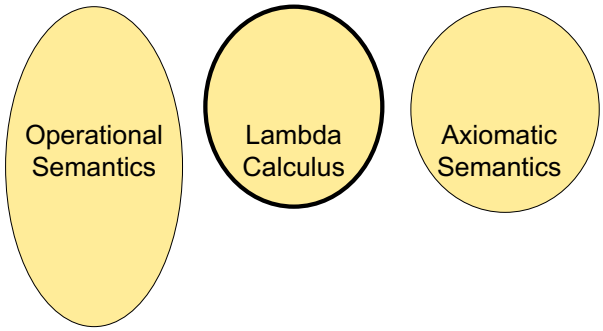
- A sequence of steps with trees of justification for each step

(C_1,m_1) --> (C_2,m_2) --> (C_3,m_3) --> ... --> m

- Let -->* be the transitive closure of -->
- Ie, the smallest transitive relation containing -->

Programming Languages & Compilers

III : Language Semantics



Lambda Calculus - Motivation

- Aim is to capture the essence of functions, function applications, and evaluation
- λ–calculus is a theory of computation
- "The Lambda Calculus: Its Syntax and Semantics". H. P. Barendregt. North Holland, 1984

Lambda Calculus - Motivation

- All *sequential programs* may be viewed as functions from input (initial state and input values) to output (resulting state and output values).
- λ-calculus is a mathematical formalism of functions and functional computations
- Two flavors: typed and untyped

Untyped λ-Calculus

- Only three kinds of expressions:
  - Variables: x, y, z, w, …
  - Abstraction: λ x. e
    (Function creation, think fun x -> e)
  - Application: e_1 e_2
  - Parenthesized expression: (e)

Untyped λ-Calculus Grammar

- Formal BNF Grammar:
  - <expression> ::= <variable>
    | <abstraction>
    | <application>
    | (<expression>)
  - <abstraction>
    ::= λ<variable>.<expression>
  - <application>
    ::= <expression> <expression>

Untyped λ-Calculus Terminology

- Occurrence: a location of a subterm in a term
- Variable binding: λ x. e is a binding of x in e
- Bound occurrence: all occurrences of x in λ x. e
- Free occurrence: one that is not bound
- Scope of binding: in λ x. e, all occurrences in e not in a subterm of the form λ x. e' (same x)
- Free variables: all variables having free occurrences in a term

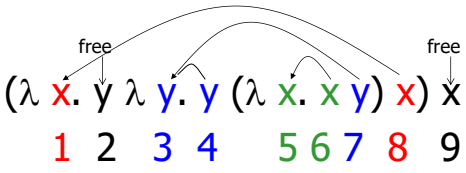
Example

- Label occurrences and scope:

(λ x. y λ y. y (λ x. x y) x) x
1 2 3 4 5 6 7 8 9

Example

- Label occurrences and scope:
-



Untyped λ-Calculus

- How do you compute with the λ-calculus?
- Roughly speaking, by substitution:
- $(\lambda x.\ e_1)\ e_2 \Rightarrow^* e_1[e_2 / x]$
- * Modulo all kinds of subtleties to avoid free variable capture

Transition Semantics for λ-Calculus

$$\frac{E \text{ --> } E''}{E\ E' \text{ --> } E''\ E'}$$

- Application (version 1 - Lazy Evaluation)

$$(\lambda\, x\,.\, E)\ E' \text{ --> } E[E'/x]$$

- Application (version 2 - Eager Evaluation)

$$\frac{E' \text{ --> } E''}{(\lambda\, x\,.\, E)\ E' \text{ --> } (\lambda\, x\,.\, E)\ E''}$$

$$\frac{}{(\lambda\, x\,.\, E)\ V \text{ --> } E[V/x]}$$

V - variable or abstraction (value)

How Powerful is the Untyped λ-Calculus?

- The untyped λ-calculus is Turing Complete
  - Can express any sequential computation
- Problems:
  - How to express basic data: booleans, integers, etc?
  - How to express recursion?
  - Constants, if_then_else, etc, are conveniences; can be added as syntactic sugar

Typed vs Untyped λ-Calculus

- The *pure* λ-calculus has no notion of type: (f f) is a legal expression
- Types restrict which applications are valid
- Types are not syntactic sugar! They disallow some terms
- Simply typed λ-calculus is less powerful than the untyped λ-Calculus: NOT Turing Complete (no recursion)

α Conversion

1. α-conversion:
2. λ x. exp --α--> λ y. (exp [y/x])
3. Provided that
   1. y is not free in exp
   2. No free occurrence of x in exp becomes bound in exp when replaced by y

λ x. x (λ y. x y) -×-> λ y. y(λ y.y y)

α Conversion Non-Examples

1. Error: y is not free in term second

λ x. x y --α--> λ y. y y

2. Error: free occurrence of x becomes bound in wrong way when replaced by y

λ x. λ y. x y --α--> λ y. λ y. y y

exp exp[y/x]

But λ x. (λ y. y) x --α--> λ y. (λ y. y) y

And λ y. (λ y. y) y --α--> λ x. (λ y. y) x

Congruence

- Let ~ be a relation on lambda terms. ~ is a congruence if
- it is an equivalence relation
- If $e_1 \sim e_2$ then
  - $(e\ e_1) \sim (e\ e_2)$ and $(e_1 e) \sim (e_2\ e)$
  - $\lambda\, x.\ e_1 \sim \lambda\, x.\ e_2$

α Equivalence

- α equivalence is the smallest congruence containing α conversion
- One usually treats α-equivalent terms as equal - i.e. use α equivalence classes of terms

Example

Show: λ x. (λ y. y x) x ~α~ λ y. (λ x. x y) y

- λ x. (λ y. y x) x --α--> λ z. (λ y. y z) z so
  λ x. (λ y. y x) x ~α~ λ z. (λ y. y z) z
- (λ y. y z) --α--> (λ x. x z) so
  (λ y. y z) ~α~ (λ x. x z) so
  (λ y. y z) z ~α~ (λ x. x z) z so
  λ z. (λ y. y z) z ~α~ λ z. (λ x. x z) z
- λ z. (λ x. x z) z --α--> λ y. (λ x. x y) y so
  λ z. (λ x. x z) z ~α~ λ y. (λ x. x y) y
- λ x. (λ y. y x) x ~α~ λ y. (λ x. x y) y

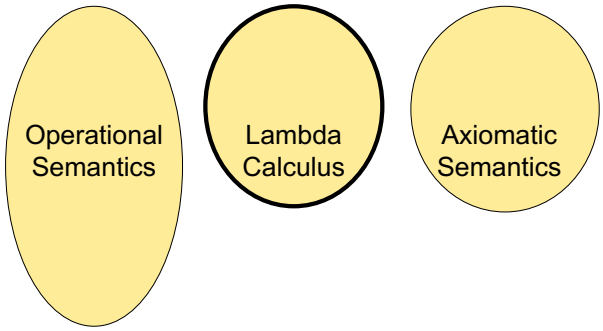
Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Programming Languages & Compilers

III : Language Semantics



Lambda Calculus - Motivation

- Aim is to capture the essence of functions, function applications, and evaluation
- λ–calculus is a theory of computation
- "The Lambda Calculus: Its Syntax and Semantics". H. P. Barendregt. North Holland, 1984

Lambda Calculus - Motivation

- All *sequential programs* may be viewed as functions from input (initial state and input values) to output (resulting state and output values).
- λ-calculus is a mathematical formalism of functions and functional computations
- Two flavors: typed and untyped

Untyped λ-Calculus

- Only three kinds of expressions:
  - Variables: x, y, z, w, …
  - Abstraction: λ x. e
    (Function creation, think fun x -> e)
  - Application: e_1 e_2
  - Parenthesized expression: (e)

Untyped λ-Calculus Grammar

- Formal BNF Grammar:
  - <expression> ::= <variable>
    | <abstraction>
    | <application>
    | (<expression>)
  - <abstraction>
    ::= λ<variable>.<expression>
  - <application>
    ::= <expression> <expression>

Untyped λ-Calculus Terminology

- Occurrence: a location of a subterm in a term
- Variable binding: λ x. e is a binding of x in e
- Bound occurrence: all occurrences of x in λ x. e
- Free occurrence: one that is not bound
- Scope of binding: in λ x. e, all occurrences in e not in a subterm of the form λ x. e' (same x)
- Free variables: all variables having free occurrences in a term

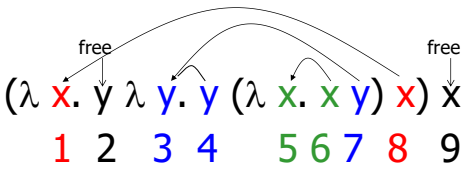
Example

- Label occurrences and scope:

(λ x. y λ y. y (λ x. x y) x) x
1 2 3 4 5 6 7 8 9

Example

- Label occurrences and scope:
-



Untyped λ-Calculus

- How do you compute with the λ-calculus?
- Roughly speaking, by substitution:
- $(\lambda\ x.\ e_1)\ e_2 \Rightarrow^* e_1\,[e_2 / x]$
- * Modulo all kinds of subtleties to avoid free variable capture

Transition Semantics for λ-Calculus

$$\frac{E \text{ --> } E''}{E\ E' \text{ --> } E''\ E'}$$

- Application (version 1 - Lazy Evaluation)

$$(\lambda\ x\ .\ E)\ E' \text{ --> } E[E'/x]$$

- Application (version 2 - Eager Evaluation)

$$\frac{E' \text{ --> } E''}{(\lambda\ x\ .\ E)\ E' \text{ --> } (\lambda\ x\ .\ E)\ E''}$$

$$\frac{}{(\lambda\ x\ .\ E)\ V \text{ --> } E[V/x]}$$

V - variable or abstraction (value)

How Powerful is the Untyped λ-Calculus?

- The untyped λ-calculus is Turing Complete
  - Can express any sequential computation
- Problems:
  - How to express basic data: booleans, integers, etc?
  - How to express recursion?
  - Constants, if_then_else, etc, are conveniences; can be added as syntactic sugar

Typed vs Untyped λ-Calculus

- The *pure* λ-calculus has no notion of type: (f f) is a legal expression
- Types restrict which applications are valid
- Types are not syntactic sugar! They disallow some terms
- Simply typed λ-calculus is less powerful than the untyped λ-Calculus: NOT Turing Complete (no recursion)

α Conversion

1. α-conversion:
2. λ x. exp --α--> λ y. (exp [y/x])
3. Provided that
   1. y is not free in exp
   2. No free occurrence of x in exp becomes bound in exp when replaced by y

λ x. x (λ y. x y) -×-> λ y. y(λ y.y y)

α Conversion Non-Examples

1. Error: y is not free in term second

   λ x. x y --α--> λ y. y y

2. Error: free occurrence of x becomes bound in wrong way when replaced by y

   λ x. λ y. x y --α--> λ y. λ y. y y

   exp exp[y/x]

But λ x. (λ y. y) x --α--> λ y. (λ y. y) y

And λ y. (λ y. y) y --α--> λ x. (λ y. y) x

Congruence

- Let ~ be a relation on lambda terms. ~ is a congruence if
- it is an equivalence relation
- If $e_1 \sim e_2$ then
  - $(e\ e_1) \sim (e\ e_2)$ and $(e_1 e) \sim (e_2\ e)$
  - $\lambda x.\ e_1 \sim \lambda x.\ e_2$

α Equivalence

- α equivalence is the smallest congruence containing α conversion
- One usually treats α-equivalent terms as equal - i.e. use α equivalence classes of terms

Example

Show: λ x. (λ y. y x) x ~α~ λ y. (λ x. x y) y

- λ x. (λ y. y x) x --α--> λ z. (λ y. y z) z so
  λ x. (λ y. y x) x ~α~ λ z. (λ y. y z) z
- (λ y. y z) --α--> (λ x. x z) so
  (λ y. y z) ~α~ (λ x. x z) so
  (λ y. y z) z ~α~ (λ x. x z) z so
  λ z. (λ y. y z) z ~α~ λ z. (λ x. x z) z
- λ z. (λ x. x z) z --α--> λ y. (λ x. x y) y so
  λ z. (λ x. x z) z ~α~ λ y. (λ x. x y) y
- λ x. (λ y. y x) x ~α~ λ y. (λ x. x y) y

Substitution

- Defined on α-equivalence classes of terms
- P [N / x] means replace every free occurrence of x in P by N
  - P called *redex*; N called *residue*
- Provided that no variable free in P becomes bound in P [N / x]
  - Rename bound variables in P to avoid capturing free variables of N

Substitution

- x [N / x] = N
- y [N / x] = y if $y \neq x$
- $(e_1\ e_2)$ [N / x] = $((e_1$ [N / x]) $(e_2$ [N / x]))
- $(\lambda$ x. e) [N / x] = $(\lambda$ x. e)
- $(\lambda$ y. e) [N / x] = λ y. (e [N / x]) provided $y \neq x$ and y not free in N
  - Rename y in redex if necessary

Example

$(\lambda$ y. y z) [$(\lambda$ x. x y) / z] = ?

- Problems?
  - z in redex in scope of y binding
  - y free in the residue
- $(\lambda$ y. y z) [$(\lambda$ x. x y) / z] --α--> $(\lambda$ w.w z) [$(\lambda$ x. x y) / z] = λ w. w $(\lambda$ x. x y)

Example

- Only replace free occurrences
- $(\lambda$ y. y z $(\lambda$ z. z)) [$(\lambda$ x. x) / z] = λ y. y $(\lambda$ x. x) $(\lambda$ z. z)

Not

λ y. y $(\lambda$ x. x) $(\lambda$ z. $(\lambda$ x. x))

β reduction

- β Rule: $(\lambda$ x. P) N --β--> P [N /x]
- Essence of computation in the lambda calculus
- Usually defined on α-equivalence classes of terms

Example

- $(\lambda$ z. $(\lambda$ x. x y) z) $(\lambda$ y. y z)
  --β--> $(\lambda$ x. x y) $(\lambda$ y. y z)
  --β--> $(\lambda$ y. y z) y --β--> y z
- $(\lambda$ x. x x) $(\lambda$ x. x x)
  --β--> $(\lambda$ x. x x) $(\lambda$ x. x x)
  --β--> $(\lambda$ x. x x) $(\lambda$ x. x x) --β--> ....

α β Equivalence

- α β equivalence is the smallest congruence containing α equivalence and β reduction
- A term is in *normal form* if no subterm is α equivalent to a term that can be β reduced
- Hard fact (Church-Rosser): if e_1 and e_2 are αβ-equivalent and both are normal forms, then they are α equivalent

Order of Evaluation

- Not all terms reduce to normal forms
- Not all reduction strategies will produce a normal form if one exists

Lazy evaluation:

- Always reduce the left-most application in a top-most series of applications (i.e. Do not perform reduction inside an abstraction)
- Stop when term is not an application, or left-most application is not an application of an abstraction to a term

Example 1

- (λ z. (λ x. x)) ((λ y. y y) (λ y. y y))
- Lazy evaluation:
- Reduce the left-most application:
- (λ z. (λ x. x)) ((λ y. y y) (λ y. y y)) --β--> (λ x. x)

Eager evaluation

- (Eagerly) reduce left of top application to an abstraction
- Then (eagerly) reduce argument
- Then β-reduce the application

Example 1

- (λ z. (λ x. x))((λ y. y y) (λ y. y y))
- Eager evaluation:
- Reduce the rator of the top-most application to an abstraction: Done.
- Reduce the argument:
- (λ z. (λ x. x))((λ y. y y) (λ y. y y))

--β--> (λ z. (λ x. x))((λ y. y y) (λ y. y y))

--β--> (λ z. (λ x. x))((λ y. y y) (λ y. y y))…

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z)

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

--β--> (λ z. z) ((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

--β--> (λ z. z) ((λ y. y y) (λ z. z)) --β-->

(λ y. y y) (λ z. z)

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

--β--> (λ z. z) ((λ y. y y) (λ z. z)) --β-->

(λ y. y y) (λ z. z)

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

--β--> (λ z. z) ((λ y. y y) (λ z. z)) --β-->

(λ y. y y) (λ z. z) ~β~ λ z. z

Example 2

- $(\lambda\ x.\ x\ x)((\lambda\ y.\ y\ y)\ (\lambda\ z.\ z))$
- Eager evaluation:

$(\lambda\ x.\ x\ \ x)\ ((\lambda\ y.\ y\ y)\ (\lambda\ z.\ z))$ --β-->

$(\lambda\ x.\ x\ \ x)\ ((\ \lambda\ z.\ z\ )\ (\lambda\ z.\ z))$ --β-->

$(\lambda\ x.\ x\ \ x)\ (\lambda\ z.\ z)$ --β-->

$(\lambda\ z.\ z)\ (\lambda\ z.\ z)$ --β--> $\lambda\ z.\ z$

Extra Material

Untyped λ-Calculus

- Only three kinds of expressions:
  - Variables: x, y, z, w, …
  - Abstraction: λ x. e
    (Function creation)
  - Application: $e_1\ e_2$

How to Represent (Free) Data Structures (First Pass - Enumeration Types)

- Suppose τ is a type with *n* constructors: $C_1,\ldots,C_n$ (no arguments)
- Represent each term as an abstraction:
- Let $C_i \rightarrow \lambda\ x_1\ \ldots\ x_n.\ x_i$
- Think: you give me what to return in each case (think match statement) and I'll return the case for the *i*th constructor

How to Represent Booleans

- bool = True | False
- True $\rightarrow \lambda\ x_1.\ \lambda\ x_2.\ x_1 \quad \equiv_\alpha \quad \lambda\ x.\ \lambda\ y.\ x$
- False $\rightarrow \lambda\ x_1.\ \lambda\ x_2.\ x_2 \quad \equiv_\alpha \quad \lambda\ x.\ \lambda\ y.\ y$

- Notation
  - Will write

    $\lambda\ x_1\ \ldots\ x_n.\ e$ for $\lambda\ x_1.\ \ldots\ \lambda x_n.\ e$

    $e_1\ e_2\ \ldots\ e_n$ for $(\ldots(e_1\ e_2)\ldots\ e_n)$

Functions over Enumeration Types

- Write a "match" function
- match e with C_1 -> x_1

  | …

  | C_n -> x_n

  $\rightarrow\ \ \lambda\ x_1\ \ldots\ x_n\ e.\ e\ x_1 \ldots x_n$

- Think: give me what to do in each case and give me a case, and I' ll apply that case

Functions over Enumeration Types

- type $\tau = C_1|\ldots|C_n$
- match e with C_1 -> x_1
  | …
  | C_n -> x_n
- $match\tau = \lambda\ x_1\ \ldots\ x_n\ e.\ e\ x_1 \ldots x_n$
- e = expression (single constructor)
  x_i is returned if e = C_i

match for Booleans

- bool = True | False
- True $\rightarrow \lambda\ x_1\ x_2.\ x_1 \quad \equiv_\alpha \quad \lambda\ x\ y.\ x$
- False $\rightarrow \lambda\ x_1\ x_2.\ x_2 \quad \equiv_\alpha \quad \lambda\ x\ y.\ y$
- $match_{bool}$ = ?

match for Booleans

- bool = True | False
- True $\rightarrow \lambda\ x_1\ x_2.\ x_1 \quad \equiv_\alpha \quad \lambda\ x\ y.\ x$
- False $\rightarrow \lambda\ x_1\ x_2.\ x_2 \quad \equiv_\alpha \quad \lambda\ x\ y.\ y$
- $match_{bool} = \lambda\ x_1\ x_2\ e.\ e\ x_1\ x_2$
  $\equiv_\alpha \quad \lambda\ x\ y\ b.\ b\ x\ y$

How to Write Functions over Booleans

- if b then x_1 else $x_2 \rightarrow$
- if_then_else b x_1 x_2 = b x_1 x_2
- if_then_else $\equiv \lambda\ b\ x_1\ x_2\ .\ b\ x_1\ x_2$

How to Write Functions over Booleans

- Alternately:
- if b then x_1 else x_2 =
  match b with True -> x_1 | False -> x_2 $\rightarrow$
  $match_{bool}\ x_1\ x_2\ b$ =
  $(\lambda\ x_1\ x_2\ b\ .\ b\ x_1\ x_2\ )\ x_1\ x_2\ b = b\ x_1\ x_2$
- if_then_else
  $\equiv \lambda\ b\ x_1\ x_2.\ (match_{bool}\ x_1\ x_2\ b)$
  $= \lambda\ b\ x_1\ x_2.\ (\lambda\ x_1\ x_2\ b\ .\ b\ x_1\ x_2\ )\ x_1\ x_2\ b$
  $= \lambda\ b\ x_1\ x_2.\ b\ x_1\ x_2$

Example:

not b
= match b with True -> False | False -> True
$\rightarrow (match_{bool})$ False True b
$= (\lambda\ x_1\ x_2\ b\ .\ b\ x_1\ x_2\ )\ (\lambda\ x\ y.\ y)\ (\lambda\ x\ y.\ x)\ b$
$= b\ (\lambda\ x\ y.\ y)(\lambda\ x\ y.\ x)$

- not $\equiv \lambda\ b.\ b\ (\lambda\ x\ y.\ y)(\lambda\ x\ y.\ x)$
- Try and, or

and or

How to Represent (Free) Data Structures (Second Pass - Union Types)

- Suppose τ is a type with n constructors:
  type $\tau = C_1\ t_{11} \ldots t_{1k} | \ldots | C_n\ t_{n1} \ldots t_{nm}$,
- Represent each term as an abstraction:
- $C_i\ t_{i1} \ldots t_{ij} \rightarrow \lambda\ x_1 \ldots x_n.\ x_i\ t_{i1} \ldots t_{ij}$
- $C_i \rightarrow \lambda\ t_{i1} \ldots t_{ij}\ x_1 \ldots x_n\ .\ x_i\ t_{i1} \ldots t_{ij}$
- Think: you need to give each constructor its arguments fisrt

How to Represent Pairs

- Pair has one constructor (comma) that takes two arguments
- type (α,β)pair = (,) α β
- (a , b) --> λ x . x a b
- (_ , _) --> λ a b x . x a b

Functions over Union Types

- Write a "match" function
- match e with $C_1\ y_1 \ldots y_{m1}$ -> $f_1\ y_1 \ldots y_{m1}$
  | ...
  | $C_n\ y_1 \ldots y_{mn}$ -> $f_n\ y_1 \ldots y_{mn}$
- $match\tau \rightarrow \lambda\ f_1 \ldots f_n\ e.\ e\ f_1 \ldots f_n$
- Think: give me a function for each case and give me a case, and I' ll apply that case to the appropriate fucntion with the data in that case

Functions over Pairs

- $match_{pair} = \lambda\ f\ p.\ p\ f$
- fst p = match p with (x,y) -> x
- fst $\rightarrow \lambda\ p.\ match_{pair}\ (\lambda\ x\ y.\ x)$
  $= (\lambda\ f\ p.\ p\ f)\ (\lambda\ x\ y.\ x) = \lambda\ p.\ p\ (\lambda\ x\ y.\ x)$
- snd $\rightarrow \lambda\ p.\ p\ (\lambda\ x\ y.\ y)$

How to Represent (Free) Data Structures (Third Pass - Recursive Types)

- Suppose τ is a type with n constructors:
  type $\tau = C_1\ t_{11} \ldots t_{1k} | \ldots | C_n\ t_{n1} \ldots t_{nm}$,
- Suppose $t_{ih} : \tau$ (ie. is recursive)
- In place of a value t_{ih} have a function to compute the recursive value $r_{ih}\ x_1 \ldots x_n$
- $C_i\ t_{i1} \ldots r_{ih} \ldots t_{ij} \rightarrow \lambda\ x_1 \ldots x_n\ .\ x_i\ t_{i1} \ldots (r_{ih}\ x_1 \ldots x_n) \ldots t_{ij}$
- $C_i \rightarrow \lambda\ t_{i1} \ldots r_{ih} \ldots t_{ij}\ x_1 \ldots x_n\ .x_i\ t_{i1} \ldots (r_{ih}\ x_1 \ldots x_n) \ldots t_{ij}$

How to Represent Natural Numbers

- nat = Suc nat | 0
- $\overline{\text{Suc}} = \lambda\ n\ f\ x.\ f\ (n\ f\ x)$
- $\overline{\text{Suc}}\ n = \lambda\ f\ x.\ f\ (n\ f\ x)$
- $\overline{0} = \lambda\ f\ x.\ x$
- Such representation called *Church Numerals*

Some Church Numerals

- $\overline{\text{Suc}\ 0} = (\lambda\ n\ f\ x.\ f\ (n\ f\ x))\ (\lambda\ f\ x.\ x)$ -->
  $\lambda\ f\ x.\ f\ ((\lambda\ f\ x.\ x)\ f\ x)$ -->
  $\lambda\ f\ x.\ f\ ((\lambda\ x.\ x)\ x)$ --> $\lambda\ f\ x.\ f\ x$

Apply a function to its argument once

Some Church Numerals

- $\overline{\text{Suc}(\text{Suc}\ 0)} = (\lambda\ n\ f\ x.\ f\ (n\ f\ x))\ (\text{Suc}\ 0)$ -->
  $(\lambda\ n\ f\ x.\ f\ (n\ f\ x))\ (\lambda\ f\ x.\ f\ x)$ -->
  $\lambda\ f\ x.\ f\ ((\lambda\ f\ x.\ f\ x)\ f\ x))$ -->
  $\lambda\ f\ x.\ f\ ((\lambda\ x.\ f\ x)\ x))$ --> $\lambda\ f\ x.\ f\ (f\ x)$

Apply a function twice

In general $\overline{n} = \lambda\ f\ x.\ f\ (\ \ldots\ (f\ x)\ldots)$ with n applications of f

Primitive Recursive Functions

- Write a "fold" function
- fold $f_1 \ldots f_n$ = match e
  with $C_1\ y_1 \ldots y_{m1}$ -> $f_1\ y_1 \ldots y_{m1}$
  | ...
  | $C_i\ y_1 \ldots r_{ij} \ldots y_{in}$ -> $f_n\ y_1 \ldots$ (fold $f_1 \ldots f_n\ r_{ij}$) $\ldots y_{mn}$
  | ...
  | $C_n\ y_1 \ldots y_{mn}$ -> $f_n\ y_1 \ldots y_{mn}$
- $fold_\tau \rightarrow\ \lambda\ f_1 \ldots f_n\ e.\ e\ f_1 \ldots f_n$
- Match in non recursive case a degenerate version of fold

Primitive Recursion over Nat

- fold f z n=
- match n with 0 -> z
- | Suc m -> f (fold f z m)
- $\overline{\text{fold}} \equiv \lambda\ f\ z\ n.\ n\ f\ z$
- $\overline{\text{is_zero}}\ \overline{n} = \overline{\text{fold}}\ (\lambda\ r.\ \overline{\text{False}})\ \overline{\text{True}}\ \overline{n}$
- $= (\lambda\ f\ x.\ f^{\,n}\ x)\ (\lambda\ r.\ \text{False})\ \text{True}$
- $= ((\lambda\ r.\ \text{False})^{\,n})\ \text{True}$
- $\equiv$ if n = 0 then True else False

Adding Church Numerals

- $\overline{n} \equiv \lambda\ f\ x.\ f^{\,n}\ x$ and $m \equiv \lambda\ f\ x.\ f^{\,m}\ x$
- $\overline{n + m} = \lambda\ f\ x.\ f^{\,(n+m)}\ x$
  $= \lambda\ f\ x.\ f^{\,n}\ (f^{\,m}\ x) = \lambda\ f\ x.\ \overline{n}\ f\ (\overline{m}\ f\ x)$
- $\overline{+} \equiv \lambda\ n\ m\ f\ x.\ n\ f\ (m\ f\ x)$
- Subtraction is harder

Multiplying Church Numerals

- $\overline{n} \equiv \lambda f\ x.\ f^n\ x$ and $m \equiv \lambda f\ x.\ f^m\ x$
- $\overline{n * m} = \lambda f\ x.\ (f^{n*m})\ x = \lambda f\ x.\ (f^m)^n\ x$
  $= \lambda f\ x.\ \overline{n}\ (\overline{m}\ f)\ x$
- $\overline{*} \equiv \lambda n\ m\ f\ x.\ n\ (m\ f)\ x$

Predecessor

- let pred_aux n =
  match n with 0 -> (0,0)
  | Suc m
  -> (Suc(fst(pred_aux m)), fst(pred_aux m)
  = fold (λ r. (Suc(fst r), fst r)) (0,0) n
- pred ≡ λ n. snd (pred_aux n) n =
  λ n. snd (fold (λ r.(Suc(fst r), fst r)) (0,0) n)

Recursion

- Want a λ-term Y such that for all term R we have
- Y R = R (Y R)
- Y needs to have replication to "remember" a copy of R
- Y = λ y. (λ x. y(x x)) (λ x. y(x x))
- Y R = (λ x. R(x x)) (λ x. R(x x))
  = R ((λ x. R(x x)) (λ x. R(x x)))
- Notice: Requires lazy evaluation

Factorial

- Let F = λ f n. if n = 0 then 1 else n * f (n - 1)

Y F 3 = F (Y F) 3
= if 3 = 0 then 1 else 3 * ((Y F)(3 - 1))
= 3 * (Y F) 2 = 3 * (F(Y F) 2)
= 3 * (if 2 = 0 then 1 else 2 * (Y F)(2 - 1))
= 3 * (2 * (Y F)(1)) = 3 * (2 * (F(Y F) 1)) =…
= 3 * 2 * 1 * (if 0 = 0 then 1 else 0*(Y F)(0 -1))
= 3 * 2 * 1 * 1 = 6

Y in OCaml

```
# let rec y f = f (y f);;
val y : ('a -> 'a) -> 'a = <fun>
# let mk_fact =
  fun f n -> if n = 0 then 1 else n * f(n-1);;
val mk_fact : (int -> int) -> int -> int = <fun>
# y mk_fact;;
Stack overflow during evaluation (looping
  recursion?).
```


Eager Eval Y in Ocaml

```
# let rec y f x = f (y f) x;;
val y : (('a -> 'b) -> 'a -> 'b) -> 'a -> 'b
  = <fun>
# y mk_fact;;
- : int -> int = <fun>
# y mk_fact 5;;
- : int = 120
```

- Use recursion to get recursion

Some Other Combinators

- For your general exposure

- I = λ x . x
- K = λ x. λ y. x
- K$_*$ = λ x. λ y. y
- S = λ x. λ y. λ z. x z (y z)

End of Extra Material

Programming Languages and Compilers (CS 421)

Elsa L Gunter
2112 SC, UIUC
http://courses.engr.illinois.edu/cs421

Based in part on slides by Mattox Beckman, as updated by Vikram Adve and Gul Agha

Untyped λ-Calculus

- How do you compute with the λ-calculus?
- Roughly speaking, by substitution:
- $(\lambda\ x.\ e_1)\ e_2 \Rightarrow^* e_1[e_2 / x]$
- * Modulo all kinds of subtleties to avoid free variable capture

Transition Semantics for λ-Calculus

$$\frac{E \text{ --> } E''}{E\ E' \text{ --> } E''\ E'}$$

- Application (version 1 - Lazy Evaluation)

$$(\lambda\ x\ .\ E)\ E' \text{ --> } E[E'/x]$$

- Application (version 2 - Eager Evaluation)

$$\frac{E' \text{ --> } E''}{(\lambda\ x\ .\ E)\ E' \text{ --> } (\lambda\ x\ .\ E)\ E''}$$

$$\frac{}{(\lambda\ x\ .\ E)\ V \text{ --> } E[V/x]}$$

V - variable or abstraction (value)

How Powerful is the Untyped λ-Calculus?

- The untyped λ-calculus is Turing Complete
  - Can express any sequential computation
- Problems:
  - How to express basic data: booleans, integers, etc?
  - How to express recursion?
  - Constants, if_then_else, etc, are conveniences; can be added as syntactic sugar

Typed vs Untyped λ-Calculus

- The *pure* λ-calculus has no notion of type: (f f) is a legal expression
- Types restrict which applications are valid
- Types are not syntactic sugar! They disallow some terms
- Simply typed λ-calculus is less powerful than the untyped λ-Calculus: NOT Turing Complete (no general recursion)

α Conversion

- α-conversion:

  λ x. exp --α--> λ y. (exp [y/x])
- Provided that
  1. y is not free in exp
  2. No free occurrence of x in exp becomes bound in exp when replaced by y

λ x. x (λ y. x y) -X-> λ y. y(λ y.y y)

α Conversion Non-Examples

1. Error: y is not free in termsecond

   λ x. x y --α--> λ y. y y

2. Error: free occurrence of x becomes bound in wrong way when replaced by y

   λ x. λ y. x y --α--> λ y. λ y. y y

   exp exp[y/x]

But λ x. (λ y. y) x --α--> λ y. (λ y. y) y

And λ y. (λ y. y) y --α--> λ x. (λ y. y) x

Congruence

- Let ~ be a relation on lambda terms. ~ is a congruence if:

1. It is an equivalence relation
2. If $e_1 \sim e_2$ then
   - $(e\ e_1) \sim (e\ e_2)$ and $(e_1 e) \sim (e_2\ e)$
   - $\lambda x.\ e_1 \sim \lambda x.\ e_2$

α Equivalence

- α equivalence is the smallest congruence containing α conversion
- One usually treats α-equivalent terms as equal - i.e. use α equivalence classes of terms

Example

Show: λ x. (λ y. y x) x ~α~ λ y. (λ x. x y) y

- λ x. (λ y. y x) x --α--> λ z. (λ y. y z) z so
  λ x. (λ y. y x) x ~α~ λ z. (λ y. y z) z
- (λ y. y z) --α--> (λ x. x z) so
  (λ y. y z) ~α~ (λ x. x z) so
  λ z. (λ y. y z) z ~α~ λ z. (λ x. x z) z
- λ z. (λ x. x z) z --α--> λ y. (λ x. x y) y so
  λ z. (λ x. x z) z ~α~ λ y. (λ x. x y) y
- λ x. (λ y. y x) x ~α~ λ y. (λ x. x y) y

Substitution

- Defined on α-equivalence classes of terms
- P [N / x] means replace every free occurrence of x in P by N
  - P called *redex*; N called *residue*
- Provided that no variable free in P becomes bound in P [N / x]
  - Rename bound variables in P to avoid capturing free variables of N

Substitution

- x [N / x] = N
- y [N / x] = y if $y \neq x$
- $(e_1\ e_2)$ [N / x] = $((e_1$ [N / x]) $(e_2$ [N / x]))
- (λ x. e) [N / x] = (λ x. e)
- (λ y. e) [N / x] = λ y. (e [N / x]) provided $y \neq x$ and y not free in N
  - Rename y in redex if necessary

Example

$(\lambda\ y.\ y\ z)\ [(\lambda\ x.\ x\ y)\ /\ z] = ?$

- Problems?
  - z in redex in scope of y binding
  - y free in the residue
- $(\lambda\ y.\ y\ z)\ [(\lambda\ x.\ x\ y)\ /\ z]$ --α--> $(\lambda\ w.w\ z)\ [(\lambda\ x.\ x\ y)\ /\ z] =$ $\lambda\ w.\ w\ (\lambda\ x.\ x\ y)$

Example

- Only replace free occurrences
- $(\lambda\ y.\ y\ z\ (\lambda\ z.\ z))\ [(\lambda\ x.\ x)\ /\ z] =$ $\lambda\ y.\ y\ (\lambda\ x.\ x)\ (\lambda\ z.\ z)$

Not

$\lambda\ y.\ y\ (\lambda\ x.\ x)\ (\lambda\ z.\ (\lambda\ x.\ x))$

β reduction

- β Rule: $(\lambda\ x.\ P)\ N$ --β--> $P\ [N\ /x]$
- Essence of computation in the lambda calculus
- Usually defined on α-equivalence classes of terms

Example

- $(\lambda\ z.\ (\lambda\ x.\ x\ y)\ z)\ (\lambda\ y.\ y\ z)$
  --β--> $(\lambda\ x.\ x\ y)\ (\lambda\ y.\ y\ z)$
  --β--> $(\lambda\ y.\ y\ z)\ y$ --β--> $y\ z$
- $(\lambda\ x.\ x\ x)\ (\lambda\ x.\ x\ x)$
  --β--> $(\lambda\ x.\ x\ x)\ (\lambda\ x.\ x\ x)$
  --β--> $(\lambda\ x.\ x\ x)\ (\lambda\ x.\ x\ x)$ --β--> ....

$\alpha\ \beta$ Equivalence

- $\alpha\ \beta$ equivalence is the smallest congruence containing α equivalence and β reduction
- A term is in *normal form* if no subterm is α equivalent to a term that can be β reduced
- Hard fact (Church-Rosser): if e_1 and e_2 are $\alpha\beta$-equivalent and both are normal forms, then they are α equivalent

Order of Evaluation

- Not all terms reduce to normal forms
- Not all reduction strategies will produce a normal form if one exists

Lazy evaluation:

- Always reduce the left-most application in a top-most series of applications (i.e. Do not perform reduction inside an abstraction)
- Stop when term is not an application, or left-most application is not an application of an abstraction to a term

Example 1

- (λ z. (λ x. x)) ((λ y. y y) (λ y. y y))
- Lazy evaluation:
- Reduce the left-most application:
- (λ z. (λ x. x)) ((λ y. y y) (λ y. y y)) --β--> (λ x. x)

Eager evaluation

- (Eagerly) reduce left of top application to an abstraction
- Then (eagerly) reduce argument
- Then β-reduce the application

Example 1

- (λ z. (λ x. x))((λ y. y y) (λ y. y y))
- Eager evaluation:
- Reduce the rator of the top-most application to an abstraction: Done.
- Reduce the argument:
- (λ z. (λ x. x))((λ y. y y) (λ y. y y))

--β--> (λ z. (λ x. x))((λ y. y y) (λ y. y y))

--β--> (λ z. (λ x. x))((λ y. y y) (λ y. y y))…

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->

((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))

--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->
((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))
--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))
--β--> (λ z. z) ((λ y. y y) (λ z. z))

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->
((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))
--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))
--β--> (λ z. z) ((λ y. y y) (λ z. z)) --β-->
(λ y. y y) (λ z. z)

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->
((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))
--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))
--β--> (λ z. z) ((λ y. y y) (λ z. z)) --β-->
(λ y. y y) (λ z. z)

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Lazy evaluation:

(λ x. x x)((λ y. y y) (λ z. z)) --β-->
((λ y. y y) (λ z. z)) ((λ y. y y) (λ z. z))
--β--> ((λ z. z) (λ z. z))((λ y. y y) (λ z. z))
--β--> (λ z. z) ((λ y. y y) (λ z. z)) --β-->
(λ y. y y) (λ z. z) ~β~ λ z. z

Example 2

- (λ x. x x)((λ y. y y) (λ z. z))
- Eager evaluation:

(λ x. x x) ((λ y. y y) (λ z. z)) --β-->
(λ x. x x) ((λ z. z) (λ z. z)) --β-->
(λ x. x x) (λ z. z) --β-->
(λ z. z) (λ z. z) --β--> λ z. z

η (Eta) Reduction

- η Rule: λ x. f x --η--> f if x not free in f
  - Can be useful in each direction
  - Not valid in Ocaml
    - recall lambda-lifting and side effects
  - Not equivalent to (λ x. f) x --> f (inst of β)
- Example: λ x. (λ y. y) x --η--> λ y. y

Untyped λ-Calculus

- Only three kinds of expressions:
  - Variables: x, y, z, w, …
  - Abstraction: λ x. e
    (Function creation)
  - Application: $e_1\ e_2$

How to Represent (Free) Data Structures (First Pass - Enumeration Types)

- Suppose τ is a type with *n* constructors: $C_1,\ldots,C_n$ (no arguments)
- Represent each term as an abstraction:
- Let $C_i \rightarrow \lambda\ x_1\ \ldots\ x_n.\ x_i$
- Think: you give me what to return in each case (think match statement) and I'll return the case for the *i*th constructor

How to Represent Booleans

- bool = True | False
- True $\rightarrow \lambda\ x_1.\ \lambda\ x_2.\ x_1 \quad \equiv_\alpha \quad \lambda\ x.\ \lambda\ y.\ x$
- False $\rightarrow \lambda\ x_1.\ \lambda\ x_2.\ x_2 \quad \equiv_\alpha \quad \lambda\ x.\ \lambda\ y.\ y$

- Notation
  - Will write
    $\lambda\ x_1\ \ldots\ x_n.\ e$ for $\lambda\ x_1.\ \ldots\ \lambda x_n.\ e$
    $e_1\ e_2\ \ldots\ e_n$ for $(\ldots(e_1\ e_2)\ldots\ e_n)$

Functions over Enumeration Types

- Write a "match" function
- match e with C_1 -> x_1
  | …
  | C_n -> x_n

  → $\lambda\ x_1\ \ldots\ x_n\ e.\ e\ x_1 \ldots x_n$

- Think: give me what to do in each case and give me a case, and I' ll apply that case

Functions over Enumeration Types

- type $\tau = C_1|\ldots|C_n$
- match e with C_1 -> x_1
  | …
  | C_n -> x_n

- $match_\tau = \lambda\ x_1\ \ldots\ x_n\ e.\ e\ x_1 \ldots x_n$

- e = expression (single constructor)
  x_i is returned if e = C_i

match for Booleans

- bool = True | False
- True $\rightarrow \lambda\ x_1\ x_2.\ x_1 \quad \equiv_\alpha \quad \lambda\ x\ y.\ x$
- False $\rightarrow \lambda\ x_1\ x_2.\ x_2 \quad \equiv_\alpha \quad \lambda\ x\ y.\ y$

- $match_{bool}$ = ?

match for Booleans

- bool = True | False
- True $\rightarrow \lambda\, x_1\, x_2.\, x_1 \quad \equiv_\alpha \quad \lambda\, x\, y.\, x$
- False $\rightarrow \lambda\, x_1\, x_2.\, x_2 \quad \equiv_\alpha \quad \lambda\, x\, y.\, y$

- $match_{bool} = \lambda\, x_1\, x_2\, e.\, e\, x_1\, x_2$
  $\equiv_\alpha \quad \lambda\, x\, y\, b.\, b\, x\, y$

How to Write Functions over Booleans

- if b then x_1 else $x_2 \rightarrow$
- if_then_else b x_1 x_2 = b x_1 x_2
- if_then_else $\equiv \lambda$ b x_1 x_2 . b x_1 x_2

How to Write Functions over Booleans

- Alternately:
- if b then x_1 else x_2 =
  match b with True -> x_1 | False -> x_2 $\rightarrow$
  $match_{bool}\, x_1\, x_2\, b$ =
  $(\lambda\, x_1\, x_2\, b\, .\, b\, x_1\, x_2\,)\, x_1\, x_2\, b = b\, x_1\, x_2$
- if_then_else
  $\equiv \lambda\, b\, x_1\, x_2.\, (match_{bool}\, x_1\, x_2\, b)$
  $= \lambda\, b\, x_1\, x_2.\, (\lambda\, x_1\, x_2\, b\, .\, b\, x_1\, x_2\,)\, x_1\, x_2\, b$
  $= \lambda\, b\, x_1\, x_2.\, b\, x_1\, x_2$

Example:

not b
= match b with True -> False | False -> True
$\rightarrow (match_{bool})$ False True b
$= (\lambda\, x_1\, x_2\, b\, .\, b\, x_1\, x_2\,)\, (\lambda\, x\, y.\, y)\, (\lambda\, x\, y.\, x)\, b$
$= b\, (\lambda\, x\, y.\, y)(\lambda\, x\, y.\, x)$

- not $\equiv \lambda$ b. b $(\lambda\, x\, y.\, y)(\lambda\, x\, y.\, x)$
- Try and, or

and or

How to Represent (Free) Data Structures (Second Pass - Union Types)

- Suppose τ is a type with *n* constructors:
  type $\tau = C_1\, t_{11} \ldots t_{1k} \mid \ldots \mid C_n\, t_{n1} \ldots t_{nm,}$
- Represent each term as an abstraction:
- $C_i\, t_{i1} \ldots t_{ij,} \rightarrow \lambda\, x_1 \ldots x_n.\, x_i\, t_{i1} \ldots t_{ij,}$
- $C_i \rightarrow \lambda\, t_{i1} \ldots t_{ij,}\, x_1 \ldots x_n\, .\, x_i\, t_{i1} \ldots t_{ij,}$
- Think: you need to give each constructor its arguments fisrt

How to Represent Pairs

- Pair has one constructor (comma) that takes two arguments
- type (α,β)pair = (,) α β
- (a , b) --> λ x . x a b
- (_ , _) --> λ a b x . x a b

Functions over Union Types

- Write a "match" function
- match e with C_1 y_1 … y_{m1} -> f_1 y_1 … y_{m1}
  | …
  | C_n y_1 … y_{mn} -> f_n y_1 … y_{mn}
- $match\tau \rightarrow \lambda f_1 \ldots f_n$ e. e $f_1 \ldots f_n$
- Think: give me a function for each case and give me a case, and I' ll apply that case to the appropriate fucntion with the data in that case

Functions over Pairs

- $match_{pair} = \lambda$ f p. p f
- fst p = match p with (x,y) -> x
- fst $\rightarrow \lambda$ p. $match_{pair}$ (λ x y. x)
  = (λ f p. p f) (λ x y. x) = λ p. p (λ x y. x)
- snd $\rightarrow \lambda$ p. p (λ x y. y)

How to Represent (Free) Data Structures (Third Pass - Recursive Types)

- Suppose τ is a type with *n* constructors:
  type $\tau = C_1 t_{11} \ldots t_{1k} | \ldots | C_n t_{n1} \ldots t_{nm,}$
- Suppose t_{ih}: τ (ie. is recursive)
- In place of a value t_{ih} have a function to compute the recursive value r_{ih} x_1 … x_n
- $C_i t_{i1} \ldots r_{ih} \ldots t_{ij} \rightarrow \lambda x_1 \ldots x_n . x_i\ t_{i1} \ldots (r_{ih} x_1 \ldots x_n) \ldots t_{ij}$
- $C_i \rightarrow \lambda\ t_{i1} \ldots r_{ih} \ldots t_{ij}\ x_1 \ldots x_n . x_i\ t_{i1} \ldots (r_{ih} x_1 \ldots x_n) \ldots t_{ij,}$

How to Represent Natural Numbers

- nat = Suc nat | 0
- $\overline{\text{Suc}} = \lambda$ n f x. f (n f x)
- $\overline{\text{Suc n}} = \lambda$ f x. f (n f x)
- $\overline{0} = \lambda$ f x. x
- Such representation called *Church Numerals*

Some Church Numerals

- $\overline{\text{Suc 0}}$ = (λ n f x. f (n f x)) (λ f x. x) -->
  λ f x. f ((λ f x. x) f x) -->
  λ f x. f ((λ x. x) x) --> λ f x. f x

Apply a function to its argument once

Some Church Numerals

- $\overline{Suc(Suc\ 0)}$ = (λ n f x. f (n f x)) (Suc 0) -->
(λ n f x. f (n f x)) (λ f x. f x) -->
λ f x. f ((λ f x. f x) f x)) -->
λ f x. f ((λ x. f x) x)) --> λ f x. f (f x)

Apply a function twice

In general $\overline{n}$ = λ f x. f (… (f x)…) with n applications of f

Primitive Recursive Functions

- Write a "fold" function
- fold f_1 … f_n = match e
with C_1 y_1 … y_{m1} -> f_1 y_1 … y_{m1}
| …
| Ci y_1 … r_{ij} …y_{in} -> f_n y_1 … (fold f_1 … f_n r_{ij}) …y_{mn}
| …
| C_n y_1 … y_{mn} -> f_n y_1 … y_{mn}
- $fold_\tau \rightarrow$ λ f_1 … f_n e. e f_1…f_n
- Match in non recursive case a degenerate version of fold

Primitive Recursion over Nat

- fold f z n=
- match n with 0 -> z
- | Suc m -> f (fold f z m)
- $\overline{fold}$ ≡ λ f z n. n f z
- $\overline{is_zero}$ $\overline{n}$ = $\overline{fold}$ (λ r. $\overline{False}$) $\overline{True}$ $\overline{n}$
- = (λ f x. f^n x) (λ r. False) True
- = ((λ r. False) ^{n}) True
- ≡ if n = 0 then True else False

Adding Church Numerals

- $\overline{n}$ ≡ λ f x. f^n x and m ≡ λ f x. f^m x
- $\overline{n + m}$ = λ f x. $f^{(n+m)}$ x
= λ f x. f^n (f^m x) = λ f x. $\overline{n}$ f ($\overline{m}$ f x)
- $\overline{+}$ ≡ λ n m f x. n f (m f x)
- Subtraction is harder

Multiplying Church Numerals

- $\overline{n}$ ≡ λ f x. f^n x and m ≡ λ f x. f^m x
- $\overline{n * m}$ = λ f x. (f^{n*m}) x = λ f x. $(f^m)^n$ x
= λ f x. $\overline{n}$ ($\overline{m}$ f) x

$\overline{*}$ ≡ λ n m f x. n (m f) x

Predecessor

- let pred_aux n =
match n with 0 -> (0,0)
| Suc m
-> (Suc(fst(pred_aux m)), fst(pred_aux m)
= fold (λ r. (Suc(fst r), fst r)) (0,0) n
- pred ≡ λ n. snd (pred_aux n) n =
λ n. snd (fold (λ r.(Suc(fst r), fst r)) (0,0) n)

Recursion

- Want a λ-term Y such that for all term R we have
- Y R = R (Y R)
- Y needs to have replication to "remember" a copy of R
- Y = λ y. (λ x. y(x x)) (λ x. y(x x))
- Y R = (λ x. R(x x)) (λ x. R(x x))
  = R ((λ x. R(x x)) (λ x. R(x x)))
- Notice: Requires lazy evaluation

Factorial

- Let F = λ f n. if n = 0 then 1 else n * f (n - 1)

Y F 3 = F (Y F) 3
= if 3 = 0 then 1 else 3 * ((Y F)(3 - 1))
= 3 * (Y F) 2 = 3 * (F(Y F) 2)
= 3 * (if 2 = 0 then 1 else 2 * (Y F)(2 - 1))
= 3 * (2 * (Y F)(1)) = 3 * (2 * (F(Y F) 1)) =…
= 3 * 2 * 1 * (if 0 = 0 then 1 else 0*(Y F)(0 -1))
= 3 * 2 * 1 * 1 = 6

Y in OCaml

```
# let rec y f = f (y f);;
val y : ('a -> 'a) -> 'a = <fun>
# let mk_fact =
  fun f n -> if n = 0 then 1 else n * f(n-1);;
val mk_fact : (int -> int) -> int -> int = <fun>
# y mk_fact;;
Stack overflow during evaluation (looping
  recursion?).
```


Eager Eval Y in Ocaml

```
# let rec y f x = f (y f) x;;
val y : (('a -> 'b) -> 'a -> 'b) -> 'a -> 'b
  = <fun>
# y mk_fact;;
- : int -> int = <fun>
# y mk_fact 5;;
- : int = 120
```

- Use recursion to get recursion

Some Other Combinators

- For your general exposure

- I = λ x . x
- K = λ x. λ y. x
- K* = λ x. λ y. y
- S = λ x. λ y. λ z. x z (y z)